



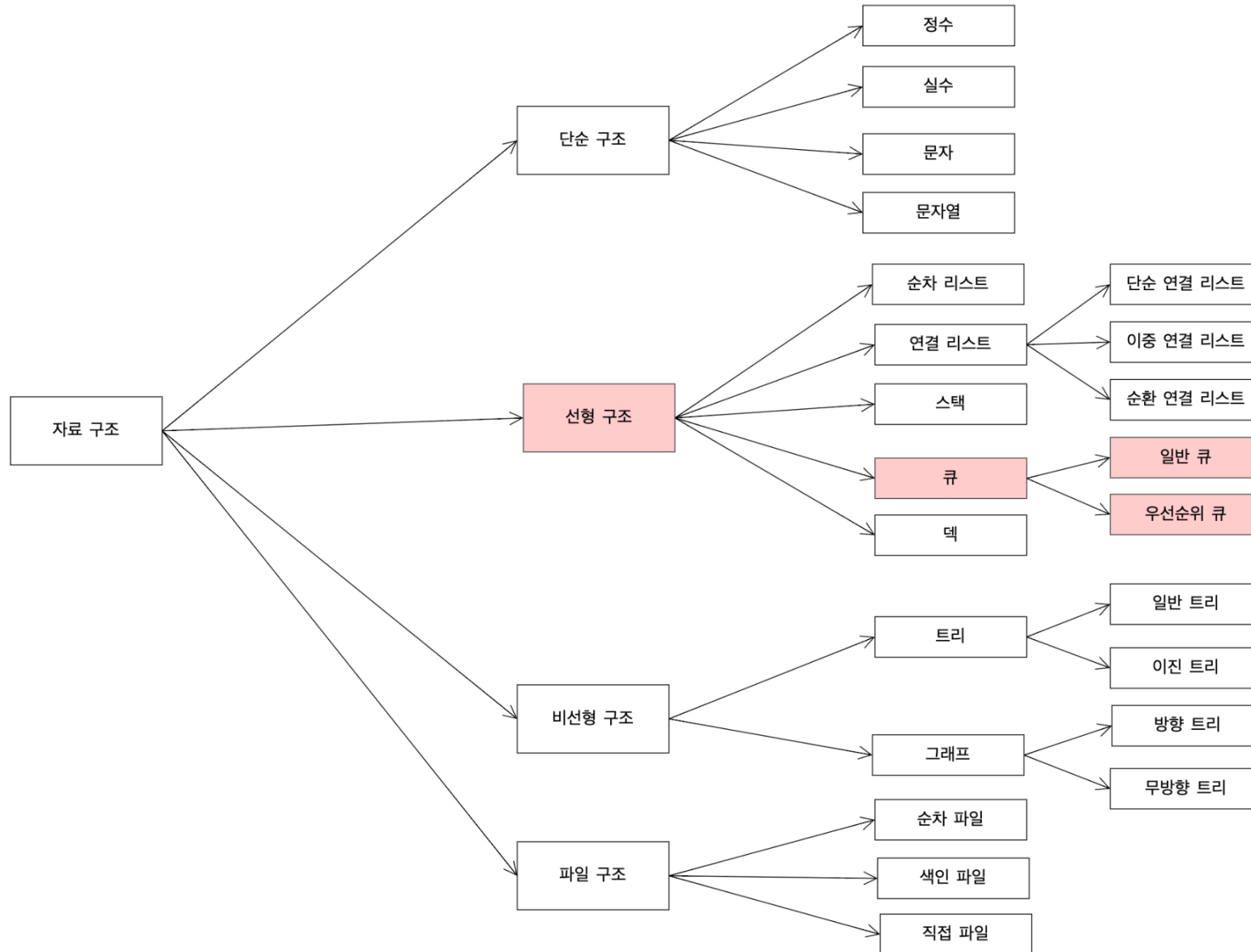
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론

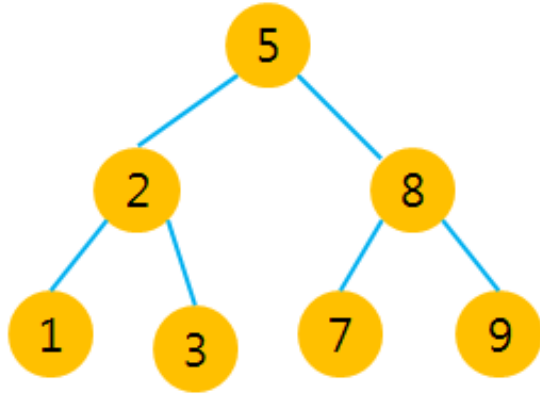


Data structure

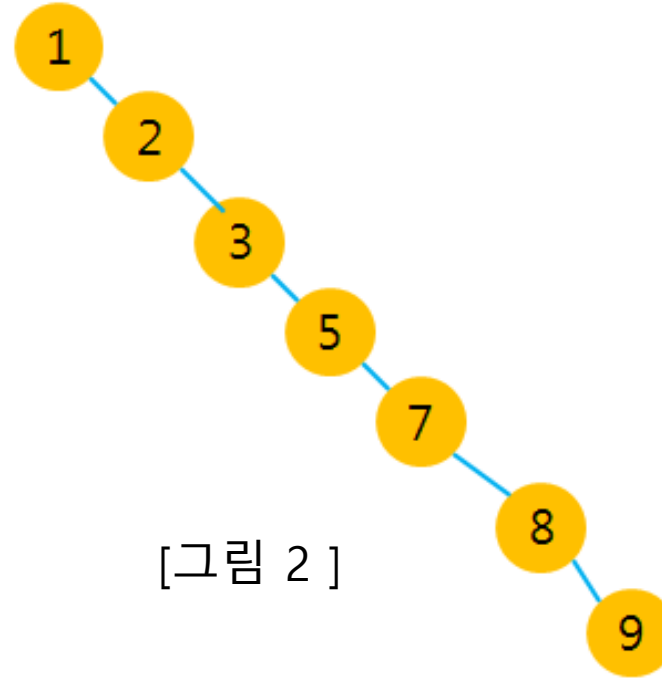


Tree

Tree의 균형



[그림 1]

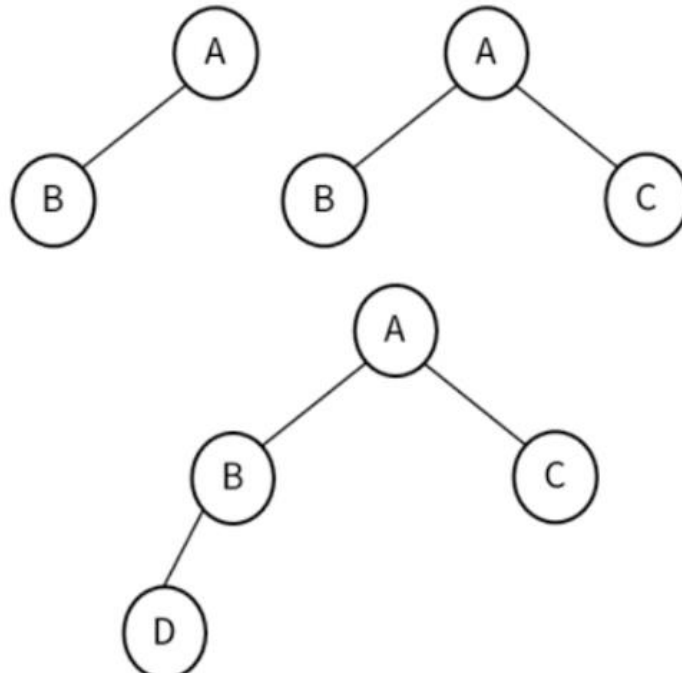


[그림 2]

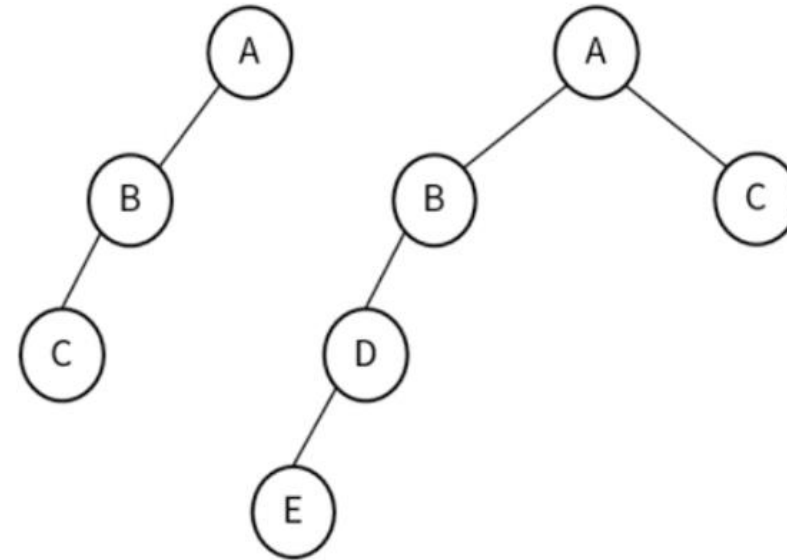
- 위의 [그림 1]의 이진 탐색 트리에서 최대 비교 횟수는 3회인 반면, [그림 2]의 트리에서는 7회가 된다.
- 이진 탐색 트리가 [그림 2]와 같이 경사 트리가 되면 탐색 시간은 순차 탐색과 같게 되어, 효율이 떨어진다.
- 따라서, 이진 탐색 트리에서 균형을 유지하는 것이 무엇보다 중요하다.

균형 이진 트리

- 모든 노드의 좌우 서브 트리 높이가 1이상 차이 나지 않는 트리



균형 이진 트리



균형 이진 트리 X

균형 검색 트리

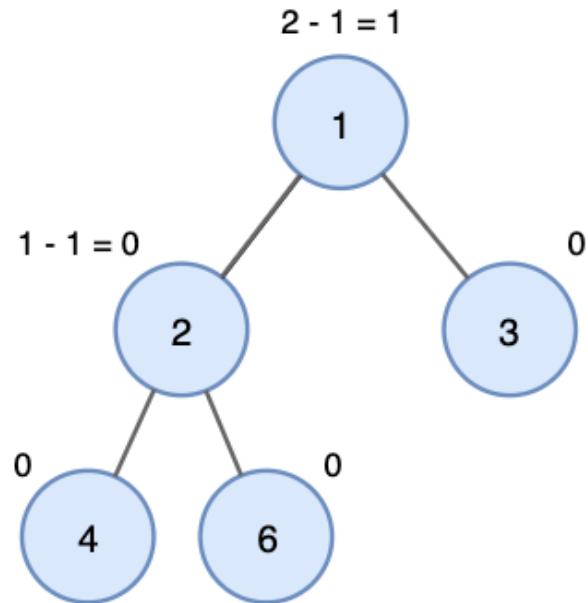
- 검색 트리의 작업들은 트리의 높이(깊이)와 밀접한 관련이 있다
 - 트리의 균형이 필요하다
- 균형 이진 검색 트리
 - $O(\log n)$ -시간 검색, 삽입, 삭제를 보장한다
 - AVL 트리, 레드-블랙 트리
- 균형 k-진 트리
 - $O(\log n)$ -시간 검색, 삽입, 삭제를 보장한다
 - 2-3 트리, 2-3-4 트리, B-트리, ...

균형 트리 (Balanced Tree)

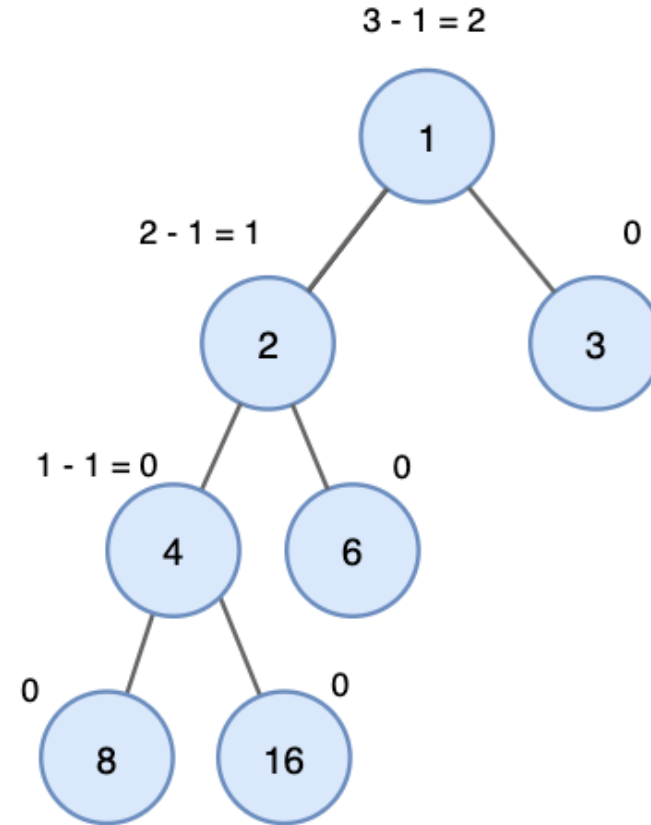
- 모든 서브트리의 '높이 차이가 최대 하나'인 트리를 의미합니다. 이러한 특성으로 인해 균형 트리는 검색 연산의 시간 복잡도를 최적화할 수 있다.
- 대표적인 균형 트리의 종류로는 AVL 트리, 레드-블랙 트리, B-트리 등이 있다.
- 이러한 균형 트리들은 삽입, 삭제 연산이 발생할 때 트리의 균형을 유지하면서 효율적인 검색을 제공한다.
- AVL 트리, 레드-블랙 트리, B-트리는 '균형 트리' 중 하나다.
- AVL 트리, 레드-블랙 트리는 '이진 탐색 트리' 중 하나다.

균형 트리 (Balanced Tree)

- 균형 인수(Balance Factor)는 왼쪽 서브 트리의 높이에서 오른쪽 서브 트리의 높이를 뺀 값이다.



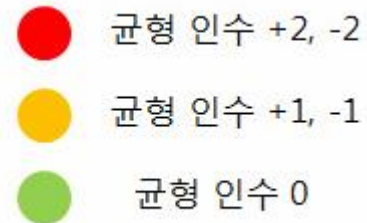
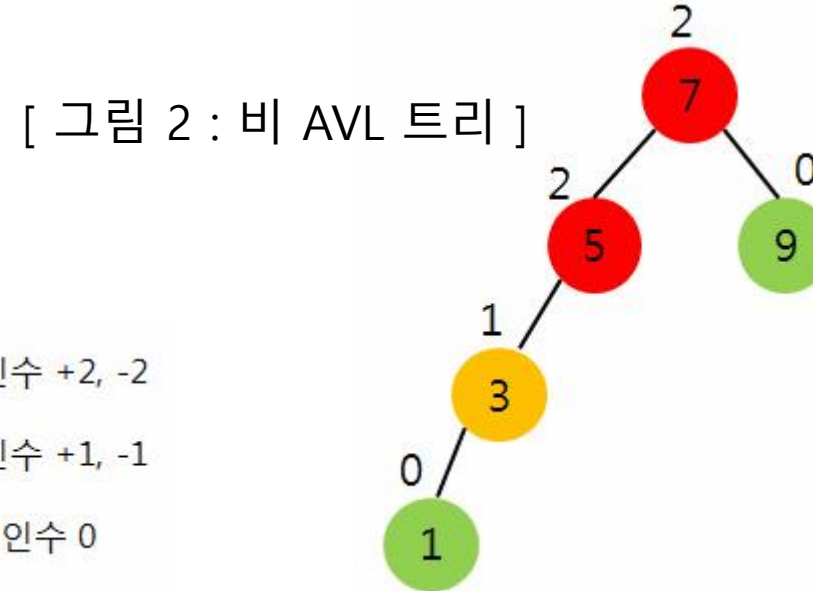
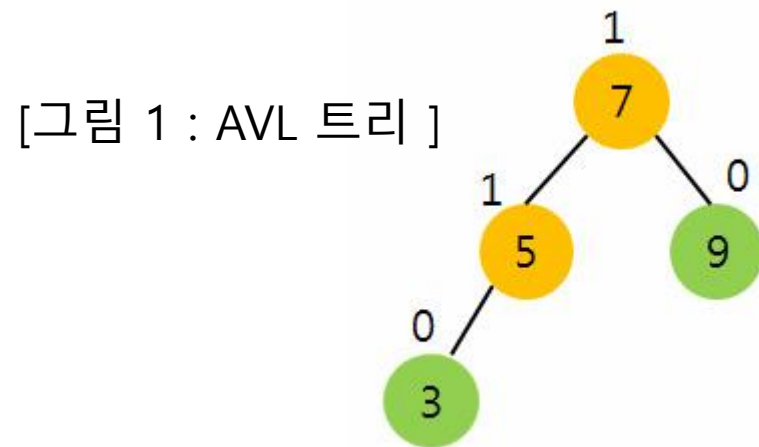
균형 이진 트리가 맞음



균형 이진 트리가 아님
왼쪽 서브트리와 오른쪽 서브트리의 높이 차이가 2이기 때문

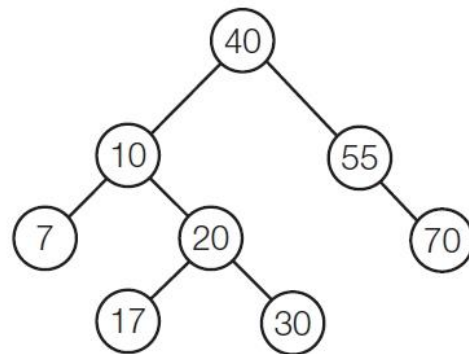
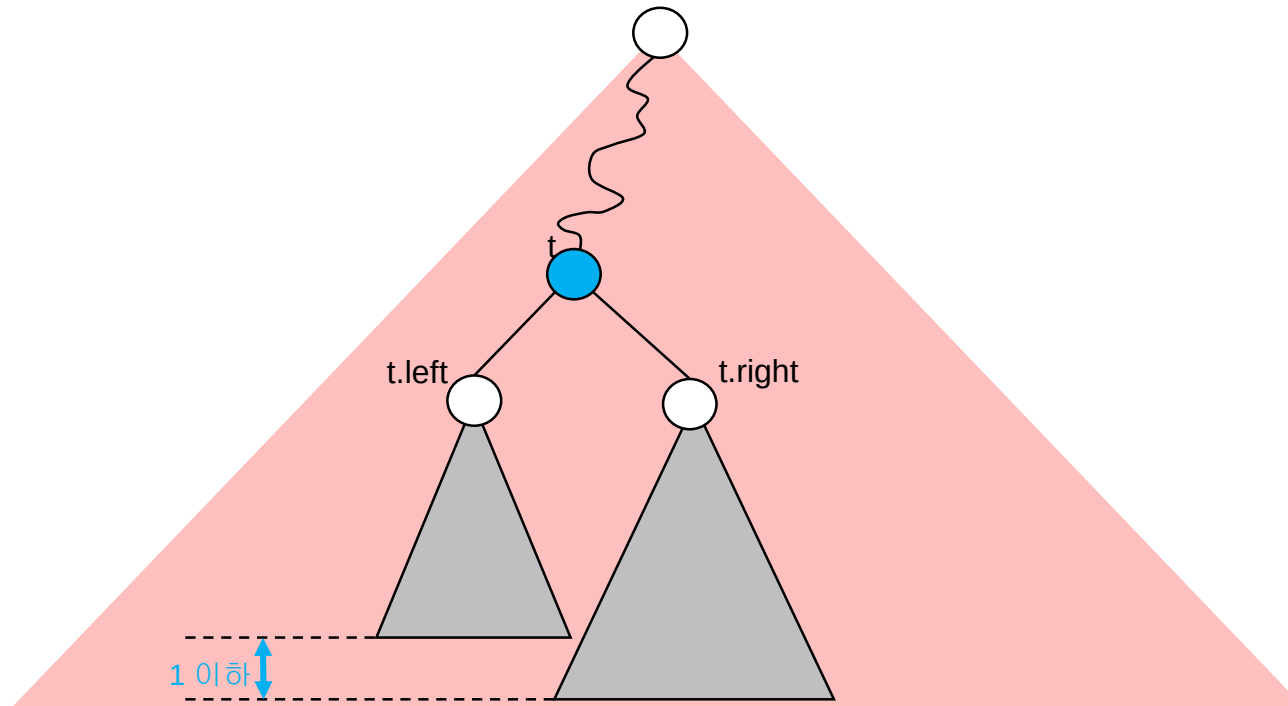
AVL Tree

- Adelson-Velskii와 Landis에 의해 고안됨
- 모든 노드에 대해 좌서브 트리의 높이(깊이)와 우서브 트리의 높이의 차가 1을 넘지 않는다

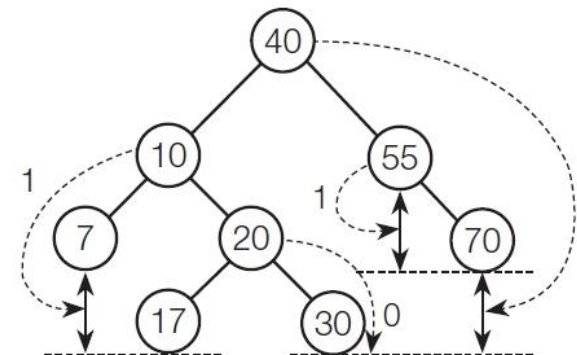


- AVL 트리는 트리가 불균형 상태로 되면 스스로 노드들을 재배치하여 균형 상태로 만든다.
- AVL트리는 균형 트리가 항상 보장된다.
- 자가 균형 이진 탐색 트리이다. 스스로 균형을 잡는 데이터 구조 중 처음으로 발명되었다.

AVL Tree

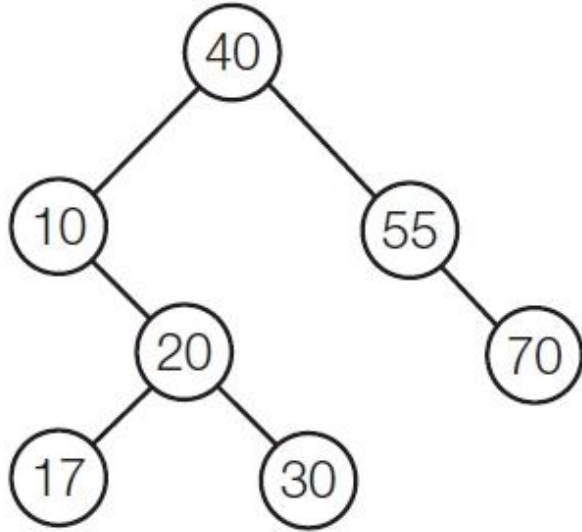


(a) AVL 트리의 예

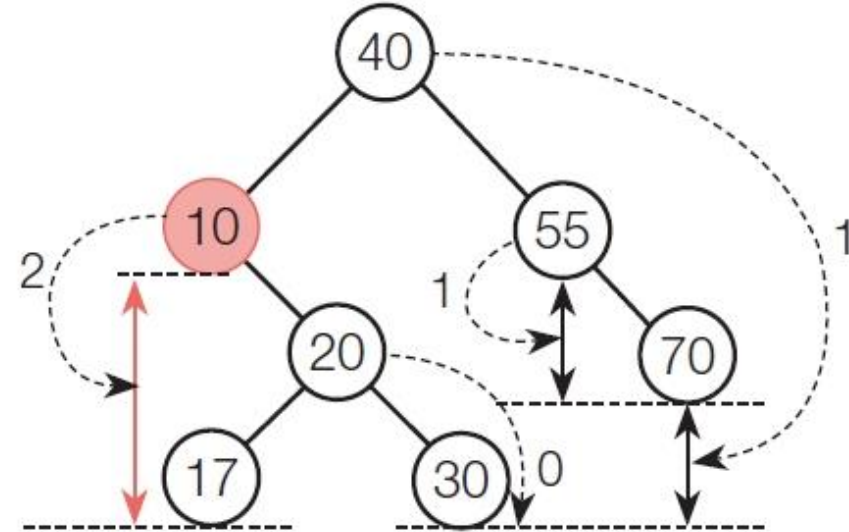


(b) 서브 트리의 높이 차

AVL Tree



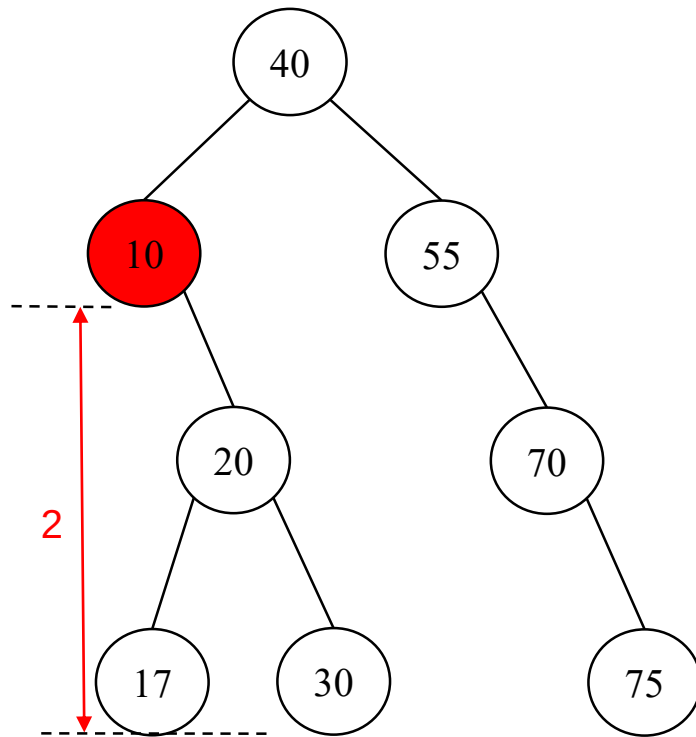
(a) AVL 트리가 아닌 예



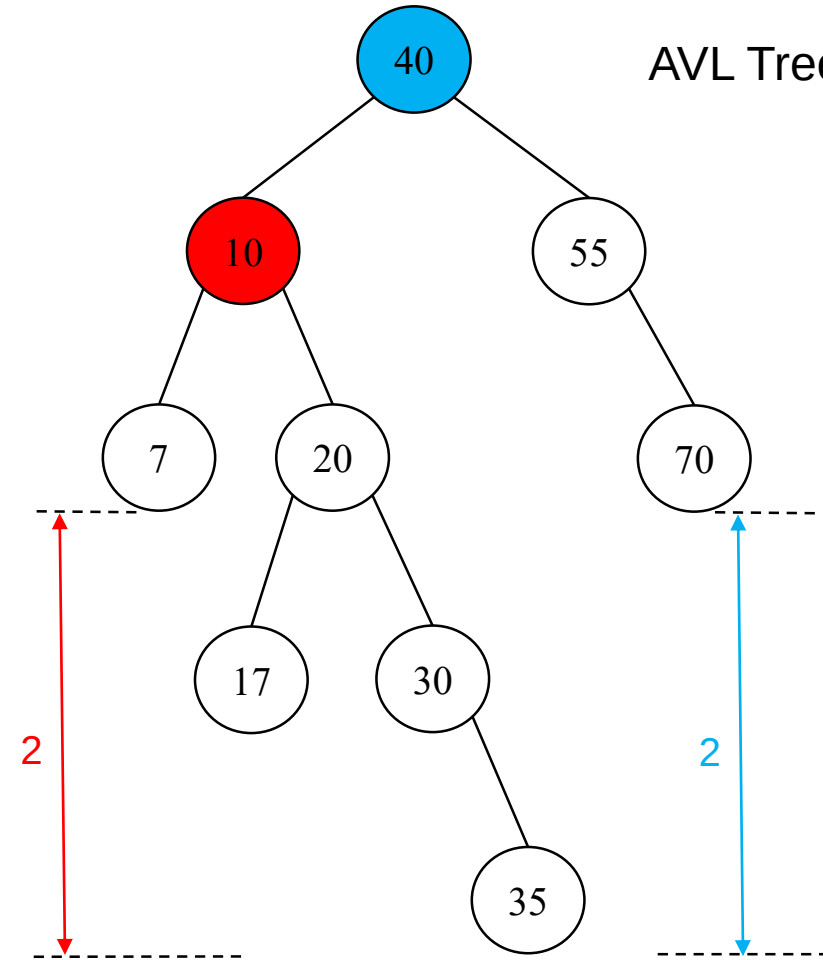
(b) AVL 트리가 아닌 이유: 높이 차 2

repair(수선)

- 서브 트리에서 균형이 깨진 예



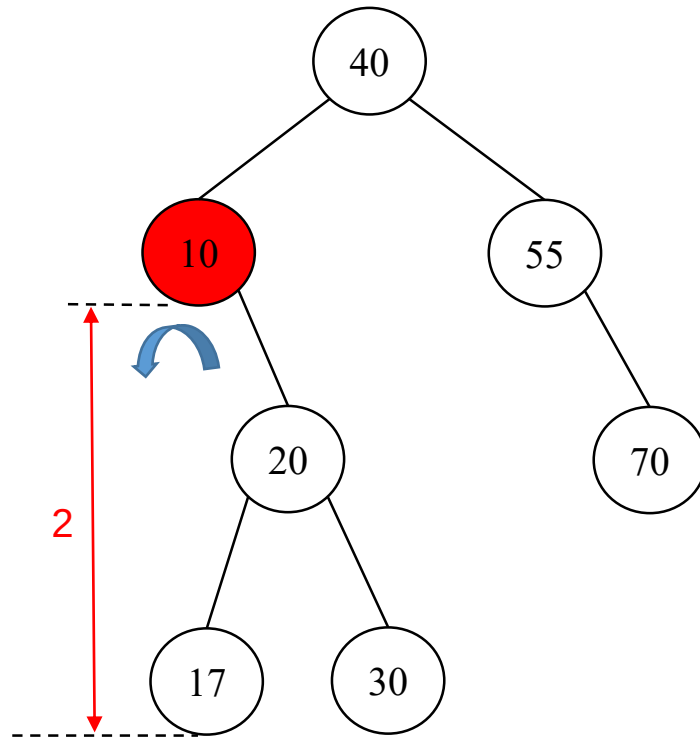
AVL Tree가 아니다



AVL Tree가 아니다

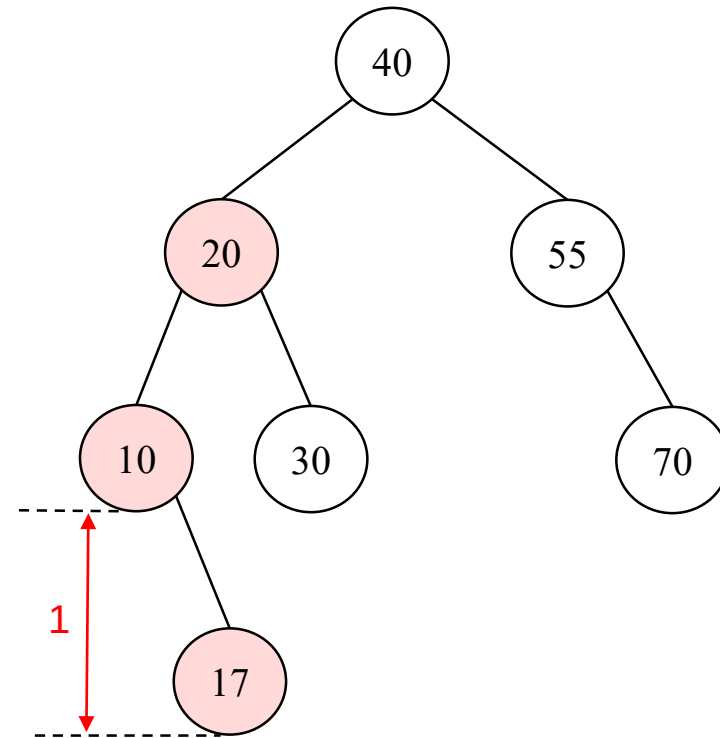
repair(수선)

- 좌회전으로 균형을 맞추는 예



AVL Tree가 아닌 예

repair

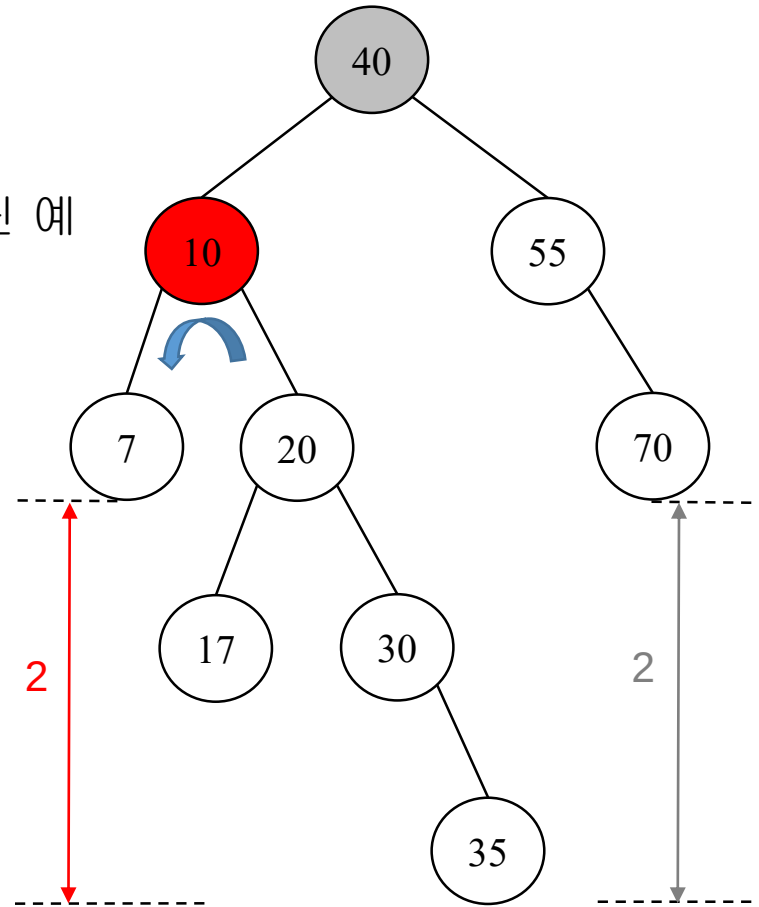


좌회전해서 AVL Tree로 수선됨

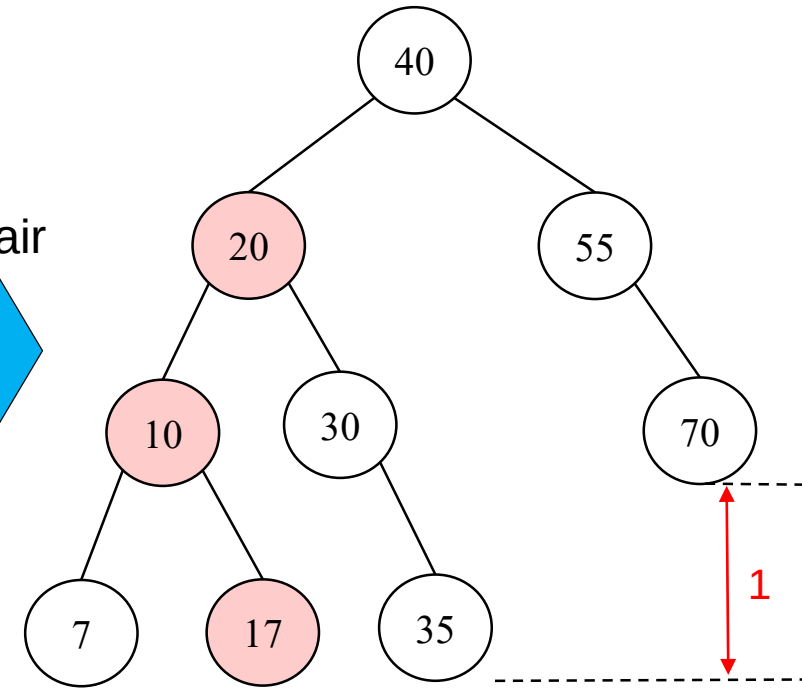
repair(수선)

- 좌회전으로 두 곳의 불균형을 해결하는 예

AVL Tree가 아닌 예



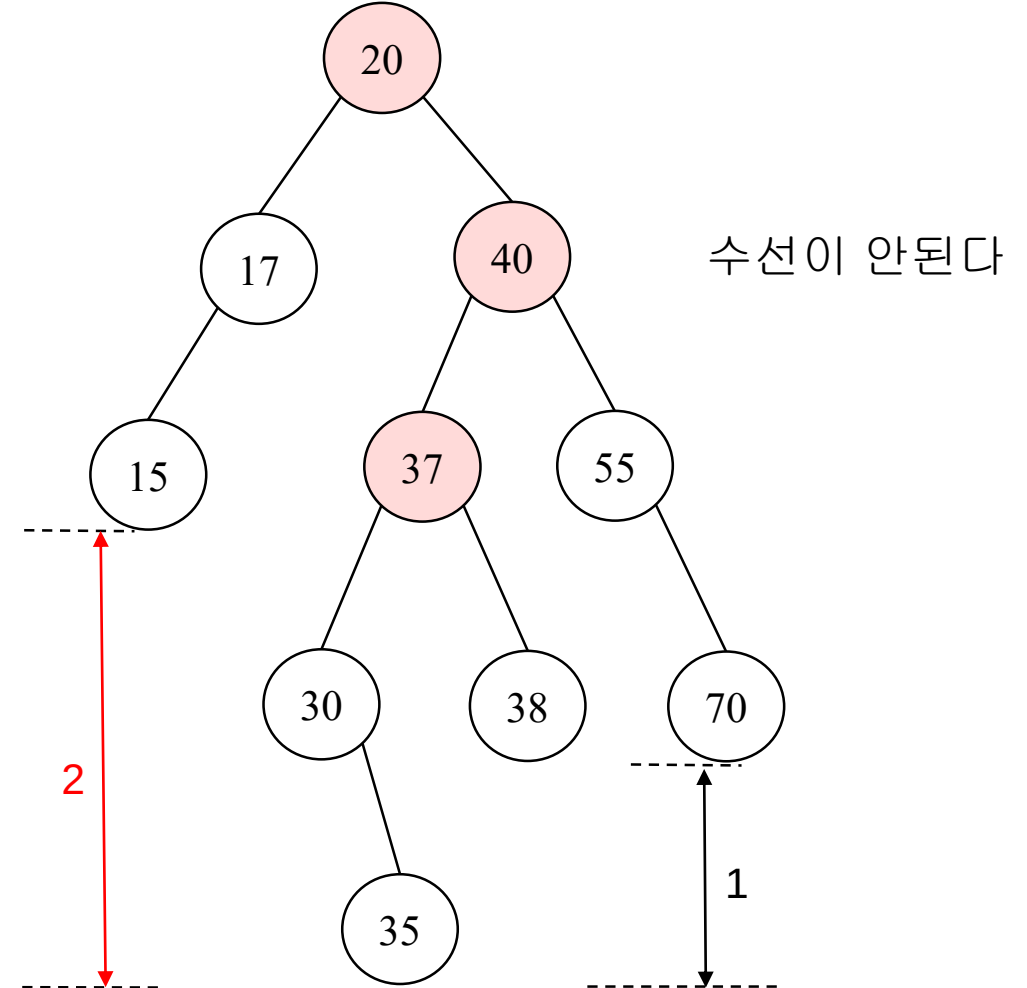
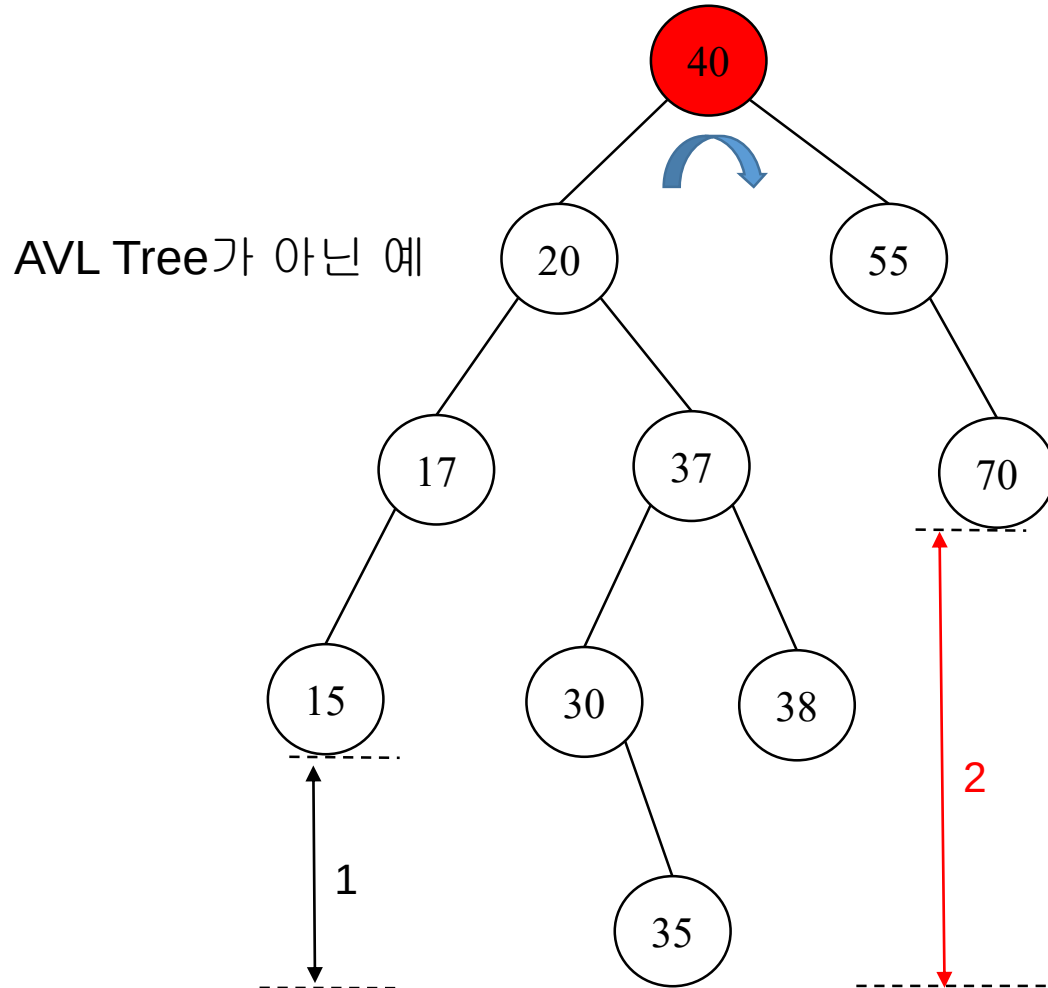
repair



좌회전해서 AVL Tree로 수선됨

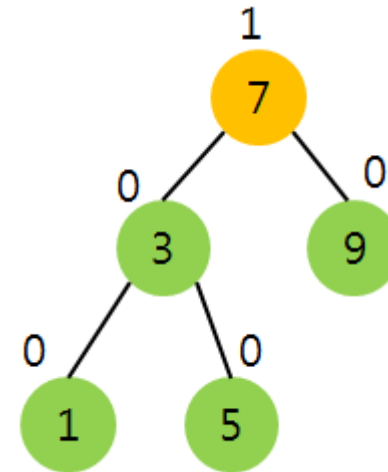
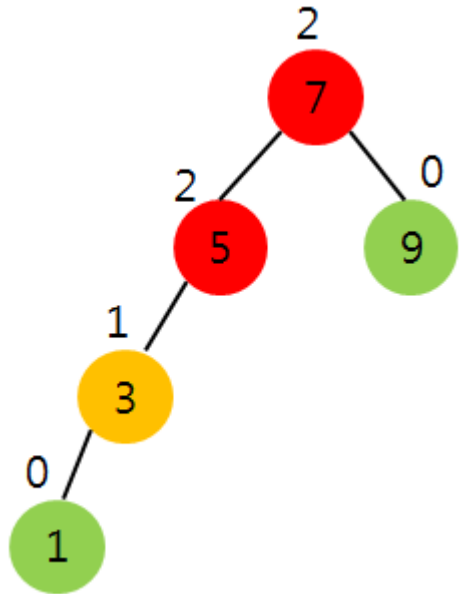
repair(수선)

- 단순한 회전으로 해결이 안 되는 예



AVL Tree

- 새로운 노드부터 균형 인수가 +2혹은 -2가 된 가장 가까운 조상 노드까지 회전시키는 것이다.
- 노드 1, 노드 3, 노드 5를 오른쪽으로 회전 시키면 된다.

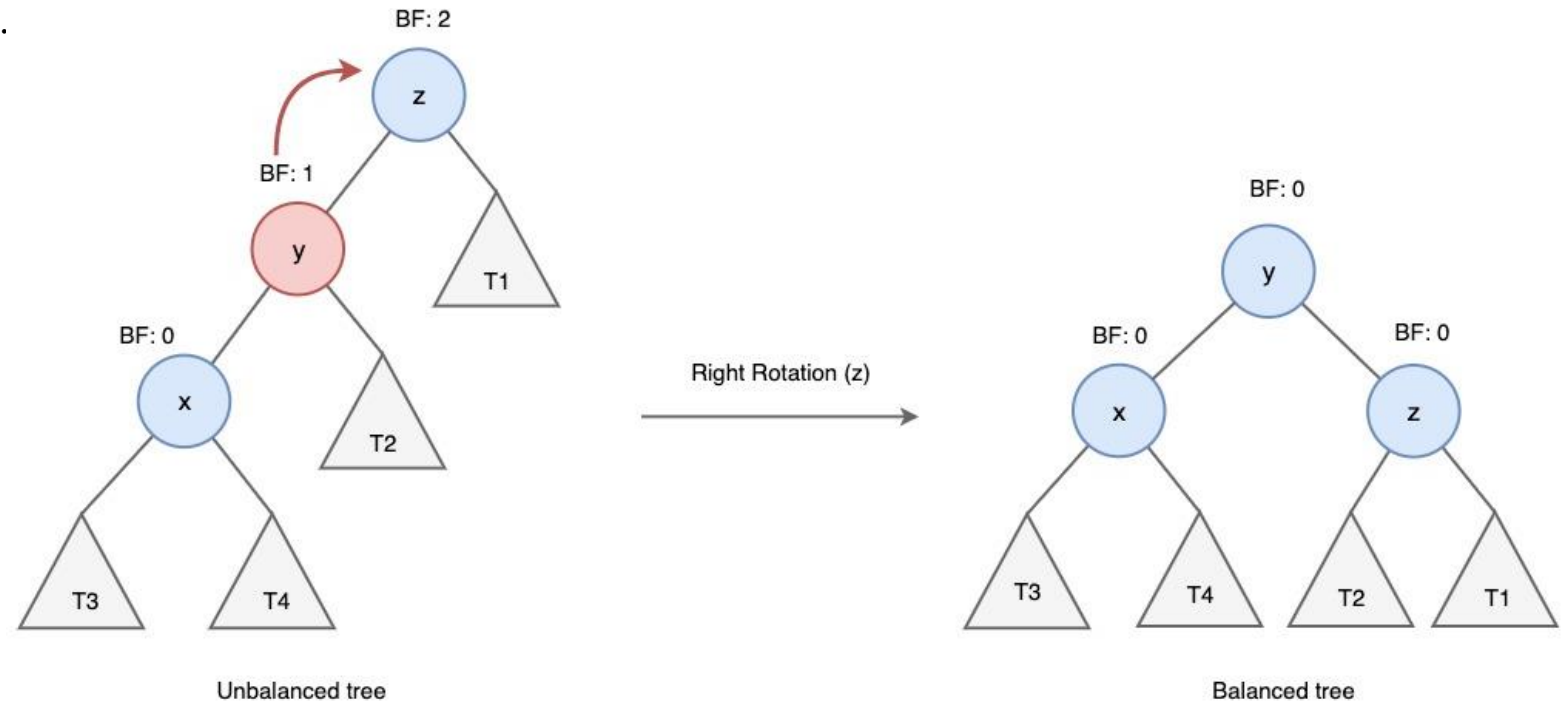


AVL Tree

- 새로 삽입된 노드N으로부터 가장 가까우면서 균형 인수가 +2혹은 -2가 된 조상 노드를 A라고 하자.
- AVL 트리에서 균형이 깨지는 경우에는 아래와 같이 4가지의 경우가 있다.
 1. LL타입 : N이 A의 왼쪽 서브 트리의 왼쪽 서브 트리에 삽입된다.
 2. RR타입 : N이 A의 오른쪽 서브 트리의 오른쪽 서브 트리에 삽입된다.
 3. RL타입 : N이 A의 오른쪽 서브 트리의 왼쪽 서브 트리에 삽입된다.
 4. LR타입 : N이 A의 왼쪽 서브 트리의 오른쪽 서브 트리에 삽입된다.
- 균형 트리로 다시 만드는 방법
 1. LL회전 : A부터 N까지의 경로상의 노드들을 오른쪽으로 회전시킨다.
 2. RR회전 : A부터 N까지의 경로상의 노드들을 왼쪽으로 회전시킨다.
 3. RL회전 : A부터 N까지의 경로상의 노드들을 오른쪽 - 왼쪽으로 회전시킨다.
 4. LR회전 : A부터 N까지의 경로상의 노드들을 왼쪽 - 오른쪽으로 회전시킨다.

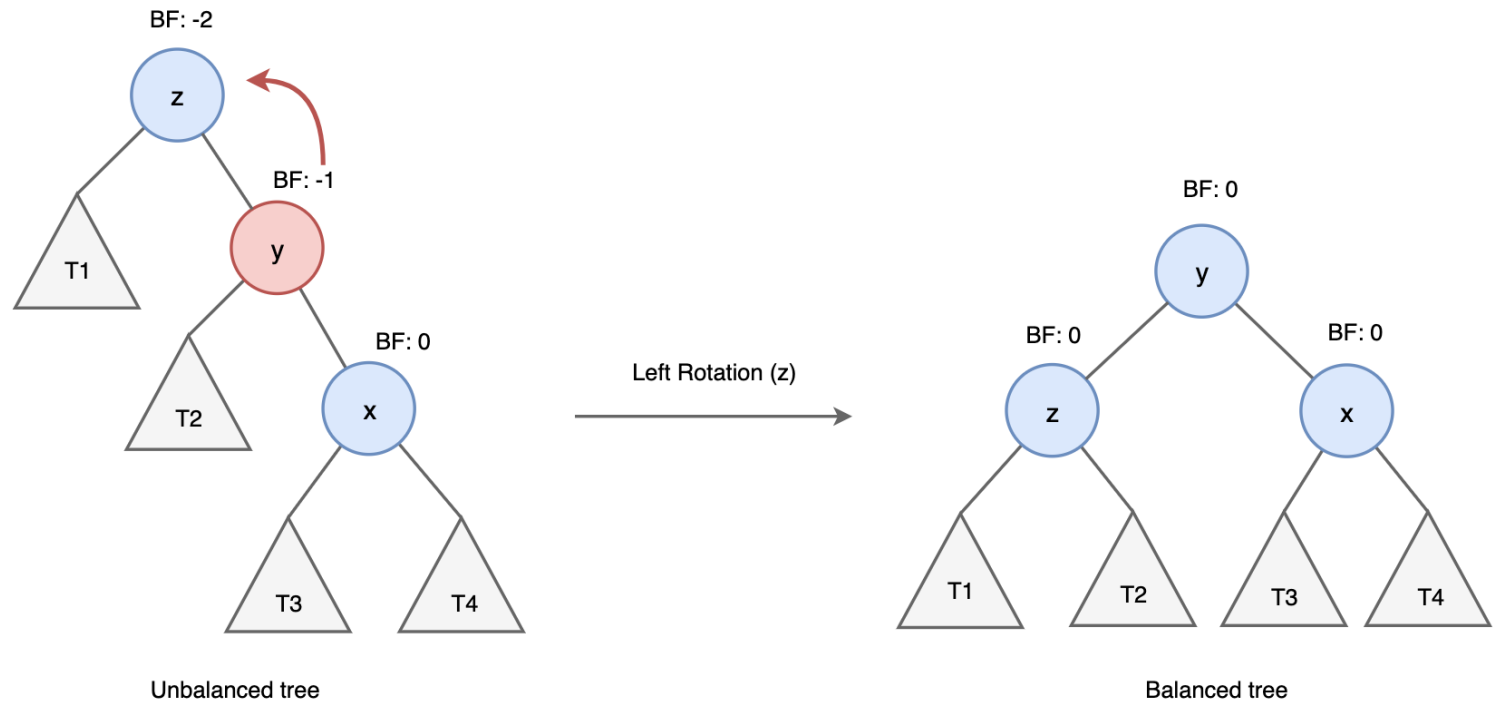
AVL Tree

- LL(Left Left) case
 - y는 z의 왼쪽 자식 노드이고, x는 y의 왼쪽 자식 노드인 경우 right rotation을 수행하여 균형을 맞춘다.
- right rotation 수행 과정
 1. y노드의 오른쪽 자식 노드를 z노드로 변경.
 2. z노드 왼쪽 자식 노드를 y노드 오른쪽 서브트리(T2)로 변경다.
 3. y는 새로운 루트 노드.



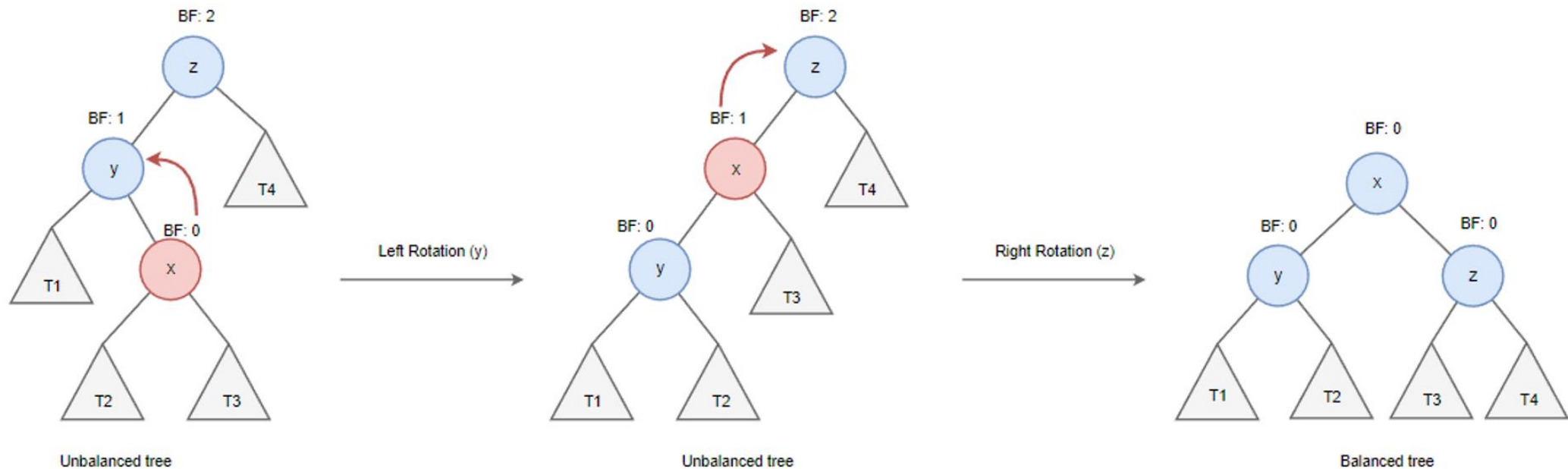
AVL Tree

- RR(Right Right) case
 - y는 z의 오른쪽 자식 노드이고, x는 y의 오른쪽 자식 노드인 경우 left rotation을 수행하여 균형을 맞춘다.
- left rotation 수행 과정
 - y노드의 왼쪽 자식 노드를 z노드로 변경.
 - z노드 오른쪽 자식 노드를 y노드 왼쪽 서브트리(T2)로 변경.
 - y는 새로운 루트 노드..



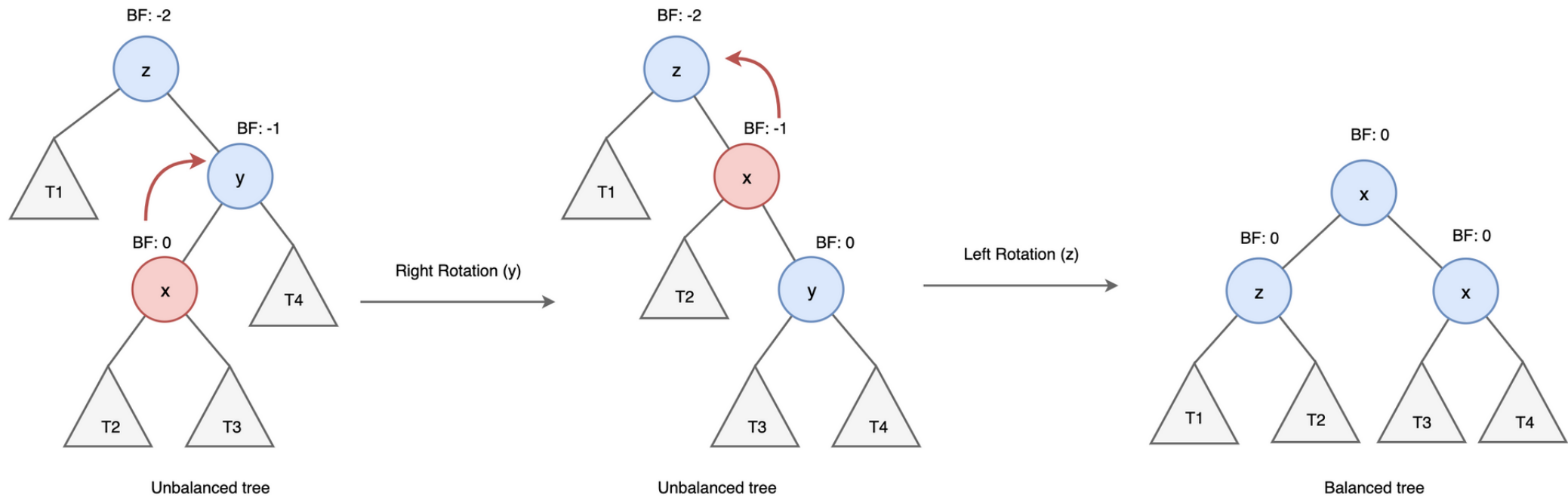
AVL Tree

- LR(Left Right) case
 - y는 z의 왼쪽 자식 노드이고, x는 y의 오른쪽 자식 노드인 경우 left , right 순으로 총 두 번의 rotation을 수행하여 균형을 맞춤.



AVL Tree

- RL(Right Left) case
 - y는 z의 오른쪽 자식 노드이고, x는 y의 왼쪽 자식 노드인 경우, right, left 순으로 총 두번의 rotation을 수행하여 균형을 맞춤



AVL Tree 시뮬레이션

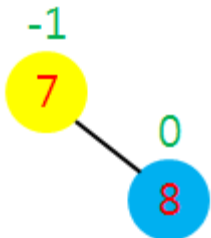
- 시뮬레이션을 진행해보자.
- {7,8,9,2,1,5,3,6,4}가 있다고 하자.
- 데이터를 하나씩 삽입해보자.

(1). 숫자 7 삽입

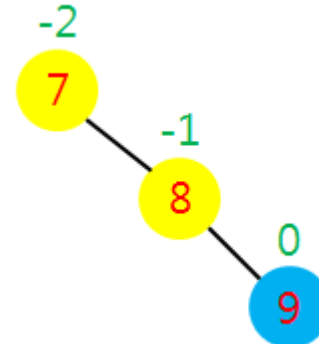
- 현재 아무것도 추가된 데이터 없이 숫자7이 삽입이 되었으므로, 아래와 같은 그림이 된다.



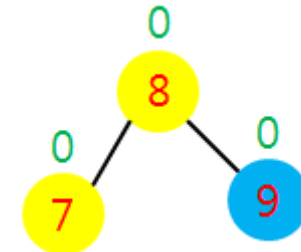
(2). 숫자 8 삽입



(3). 숫자 9 삽입

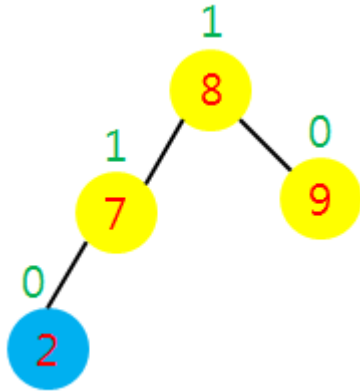


위에서 균형 인수가 -2가 됐으므로, 트리의 재조정이 필요하며 RR회전을 해야 한다.
RR회전을 하자.

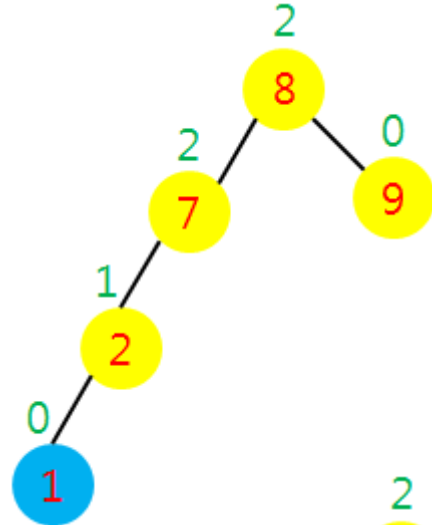


AVL Tree 시뮬레이션

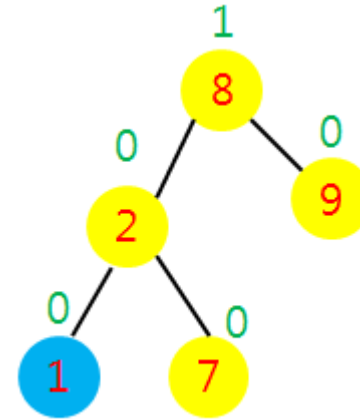
(4). 숫자 2 삽입



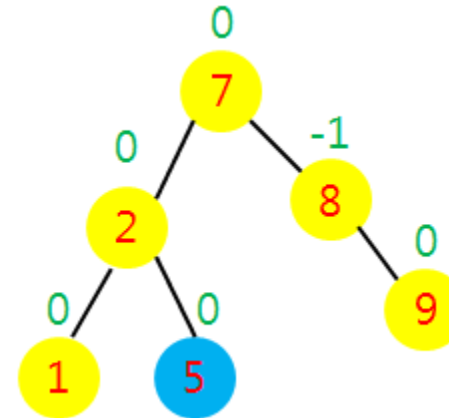
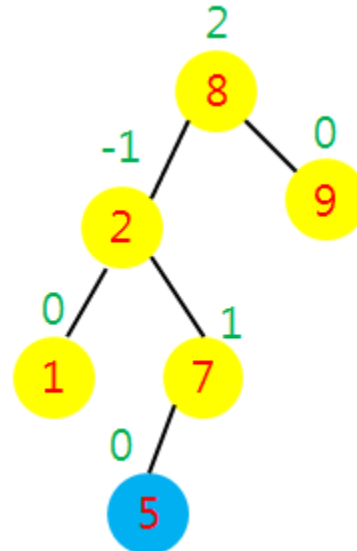
(5). 숫자 1 삽입



위에서 균형 인수가 2가 되었으므로, LL회전을 한다.



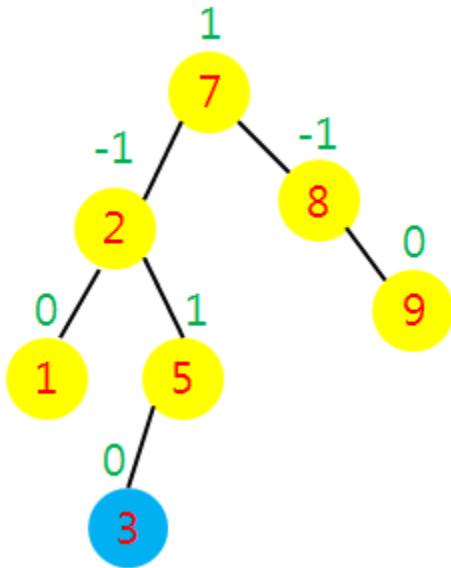
(6). 숫자 5 삽입



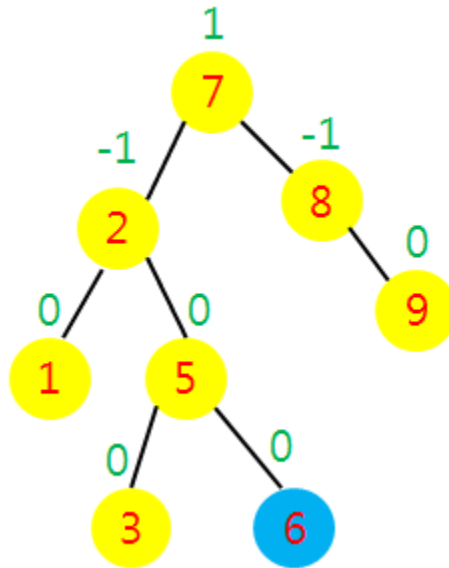
위의 경우는 LR회전을 해야 한다. 아래의 그림이 LR회전 후의 그림.

AVL Tree 시뮬레이션

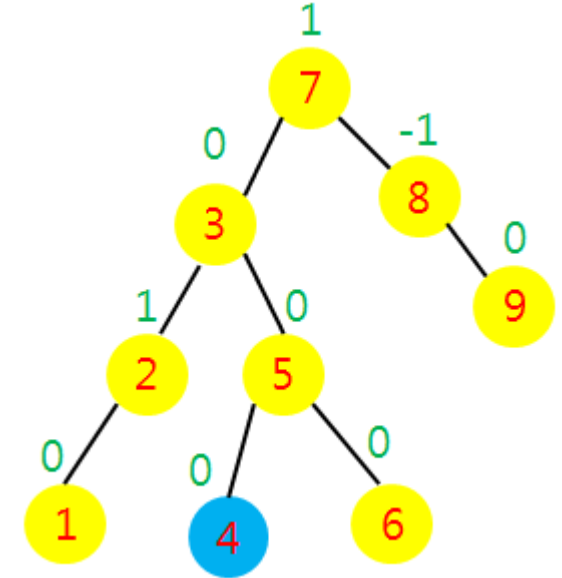
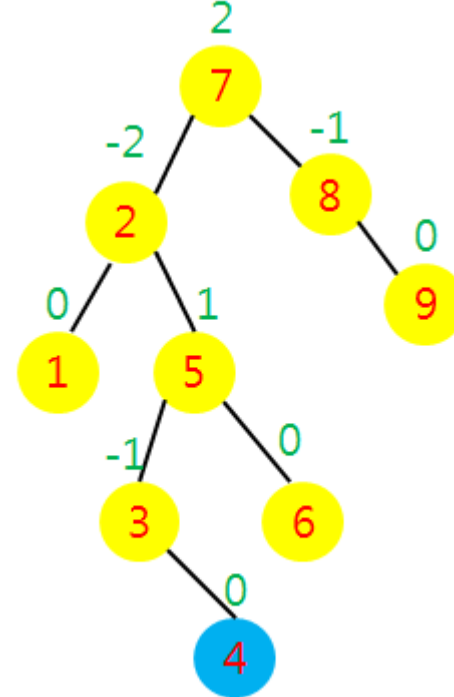
(7). 숫자 3을 삽입



(8). 숫자 6 삽입



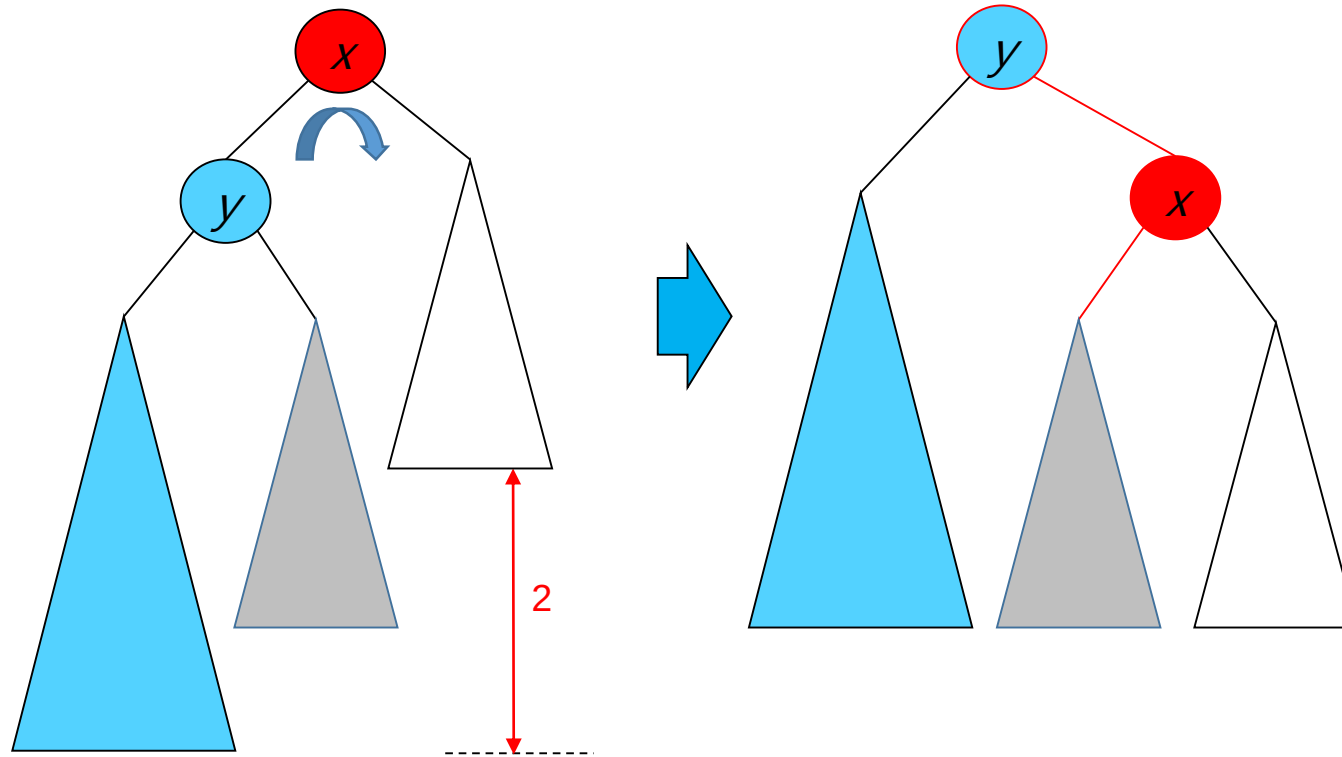
(9). 숫자 4 삽입



그리고 RL회전을 해야 한다.
최종적인 형태는 위의 그림이다.

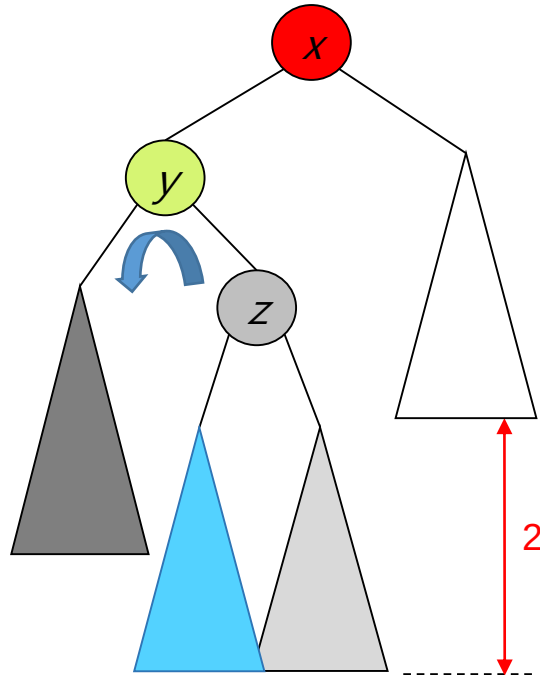
LL 유형의 해결

우회전 **Right rotation**

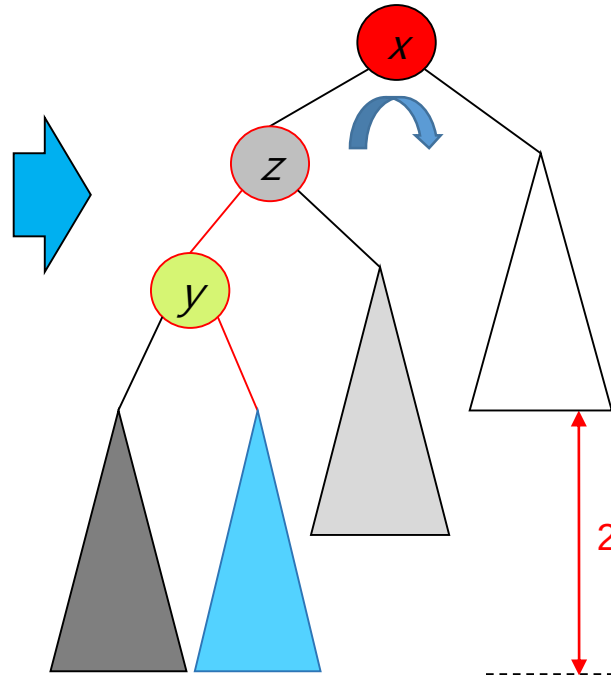


LR 유형의 해결(1)

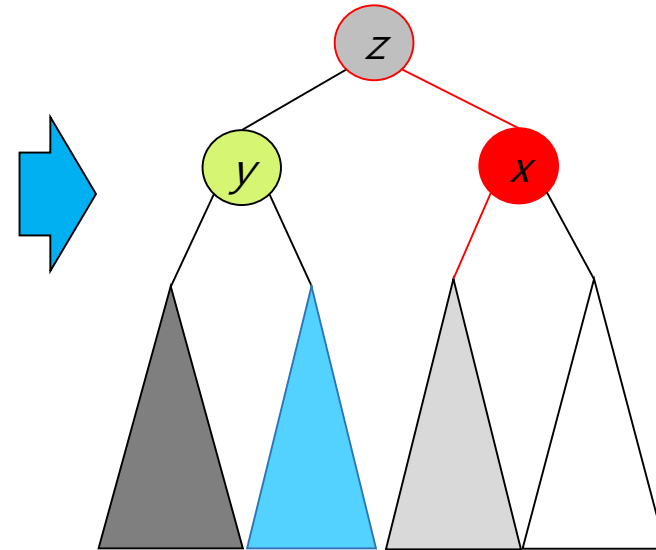
좌회전 후 우회전
(타입 LL로 변환)



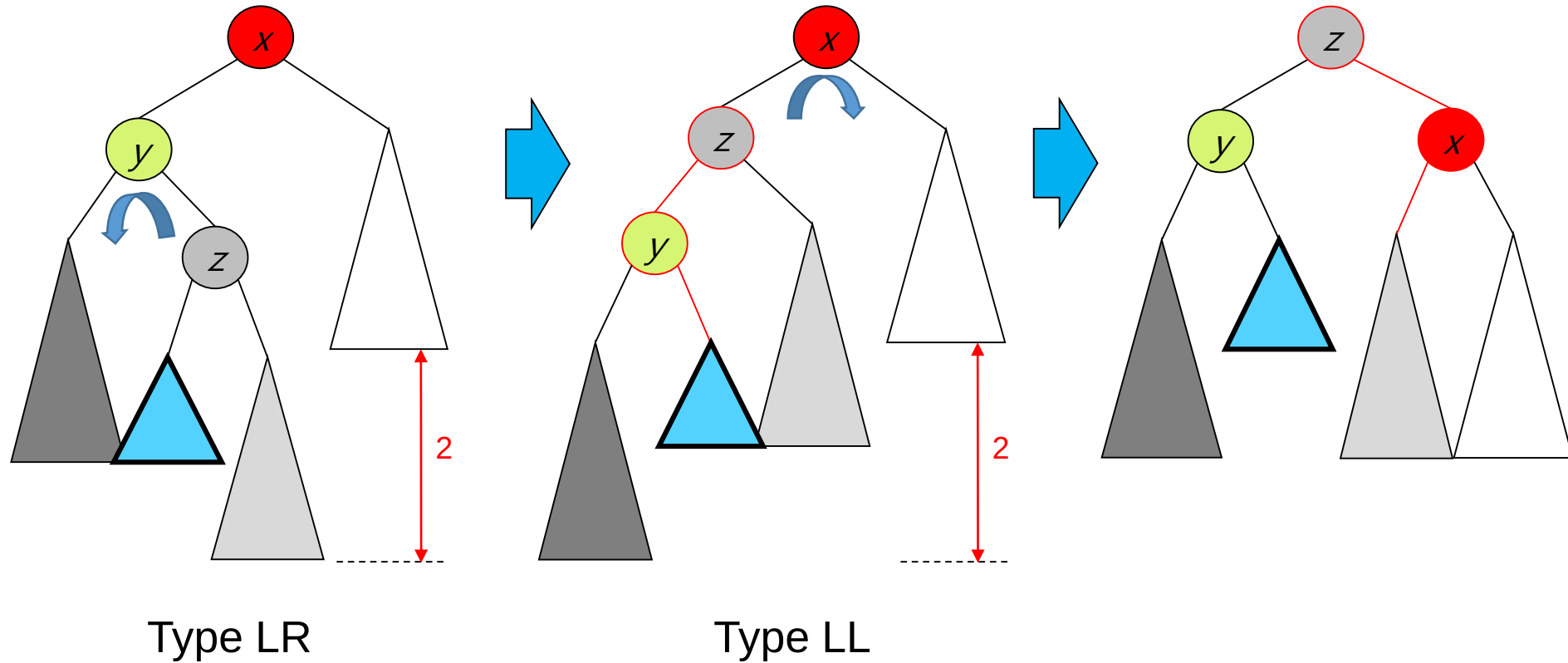
Type LR



Type LL

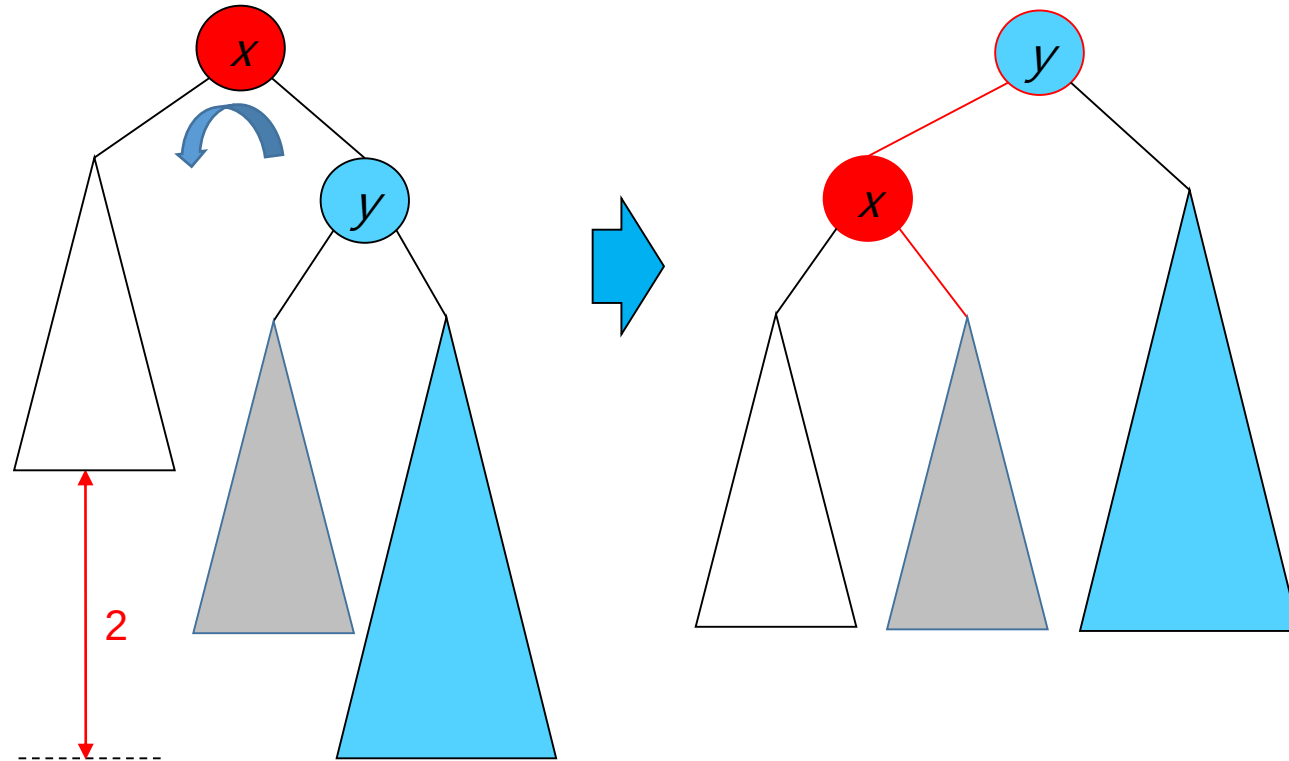


LR 유형의 해결(2)



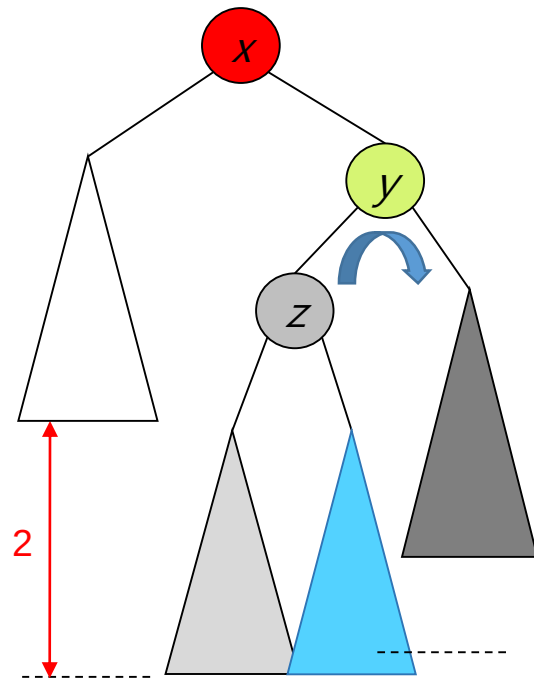
RR 유형의 해결

좌회전

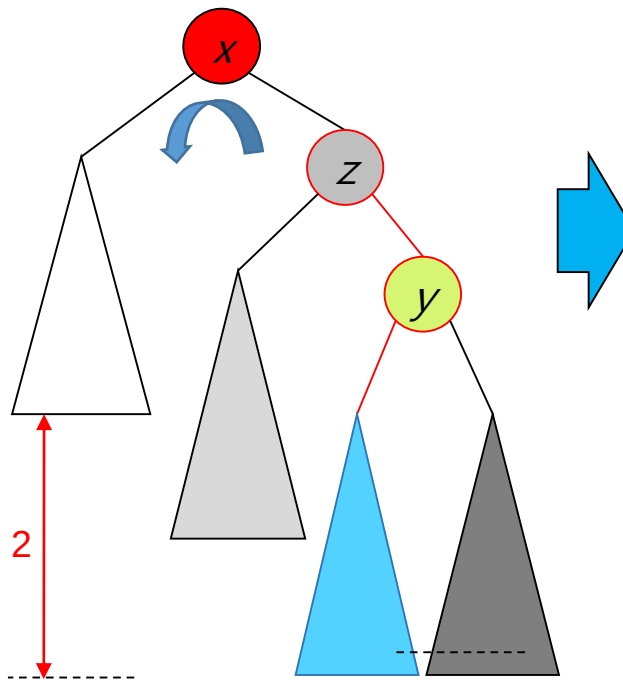


RL 유형의 해결

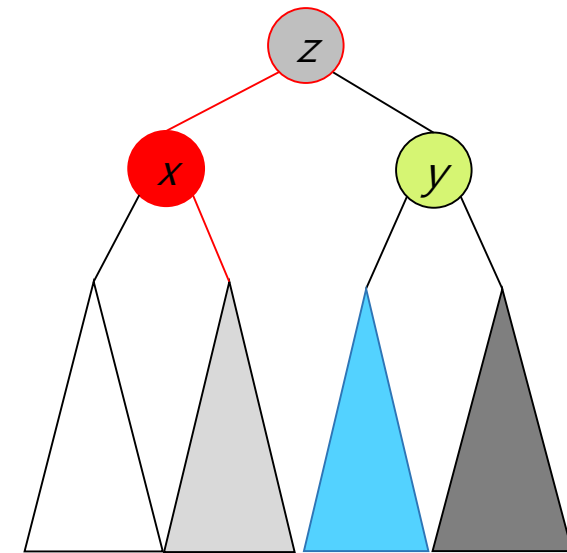
우회전 후 좌회전
 (타입 RR로 변환)



Type RL



Type RR



균형 맞추기

알고리즘 11-3 AVL 트리 균형 잡기(스케치)

```
balanceAVL(t, type):  
    switch type:  
        case LL: 우회전(t)  
        case LR: 좌회전(t.left)  
                    balanceAVL(t, LL)  
        case RR: 좌회전(t)  
        case RL: 우회전(t.right)  
                    balanceAVL(t, RR)
```

Equivalently,

```
balanceAVL(t, type):  
    switch type:  
        case LL: 우회전(t)  
        case LR: 좌회전(t.left)  
                    우회전(t)  
        case RR: 좌회전(t)  
        case RL: 우회전(t.right)  
                    좌회전(t)
```

균형 맞추기

알고리즘 11-3 AVL 트리 균형 잡기(스케치)

```
balanceAVL(t, type):
```

```
    switch type:
```

```
        case LL: 우회전(t)
```

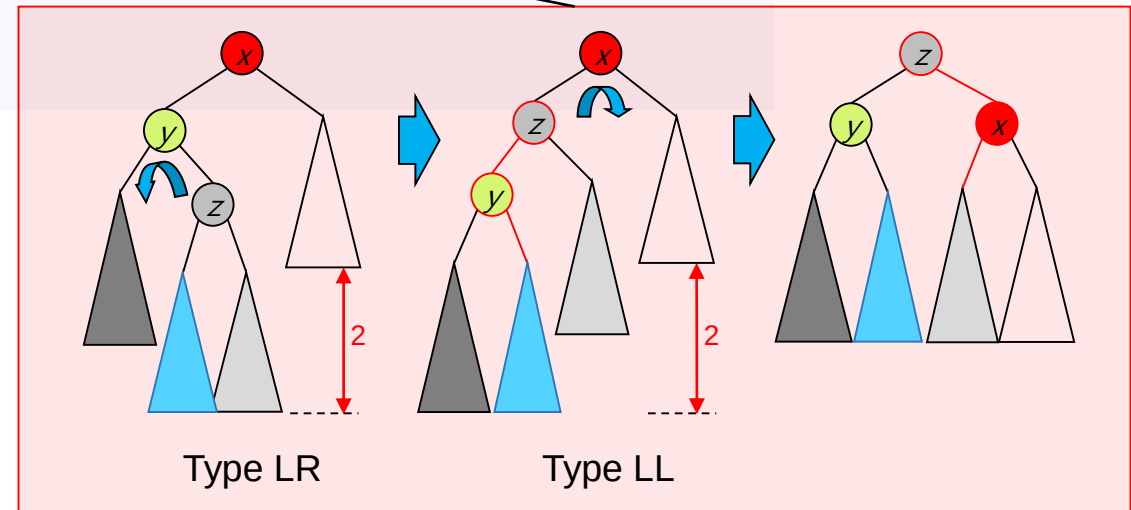
```
        case LR: 좌회전(t.left)
```

```
                balanceAVL(t, LL)
```

```
        case RR: 좌회전(t)
```

```
        case RL: 우회전(t.right)
```

```
                balanceAVL(t, RR)
```



균형 맞추기

알고리즘 11-1 AVL 트리의 좌회전

좌회전(t): ◀ t: 회전의 중심 노드

RChild ← t.right

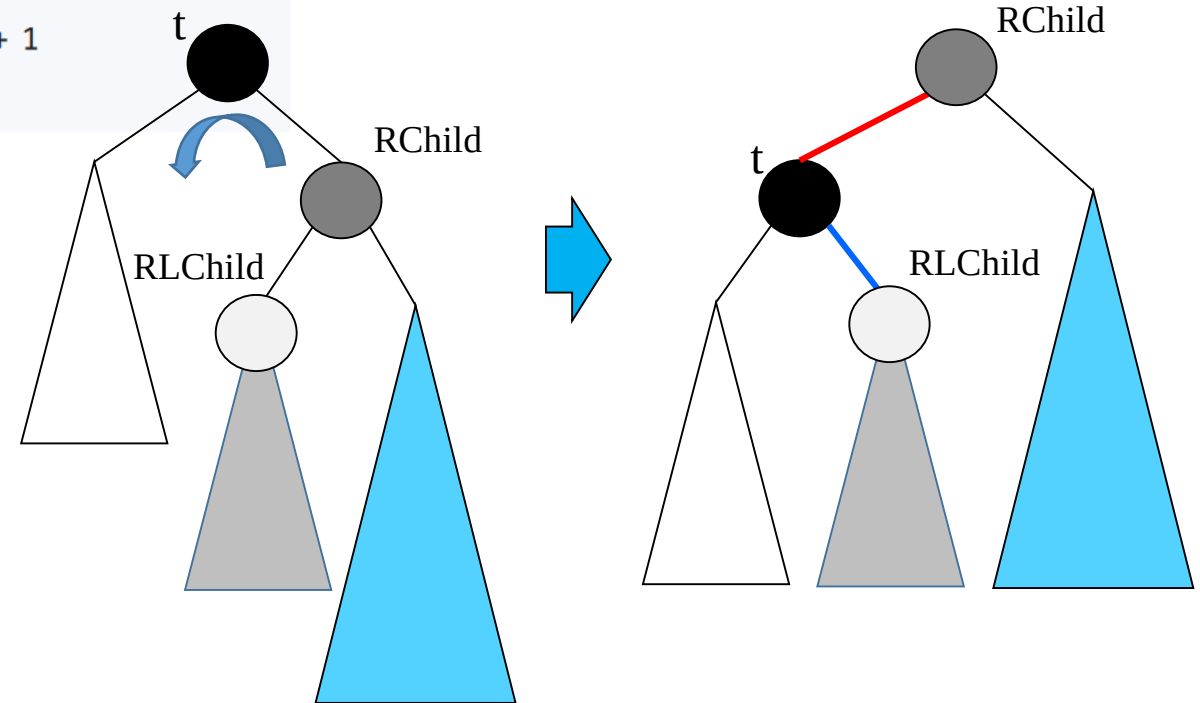
RLChild ← RChild.left

RChild.left ← t

t.right ← RLChild

RChild.height ← max(RChild.right.height, RChild.left.height) + 1

t.height ← max(t.right.height, t.left.height) + 1



균형 맞추기

알고리즘 11-2 AVL 트리의 우회전

우회전(t): ◀ t: 회전의 중심 노드

```
LChild ← t.left;
```

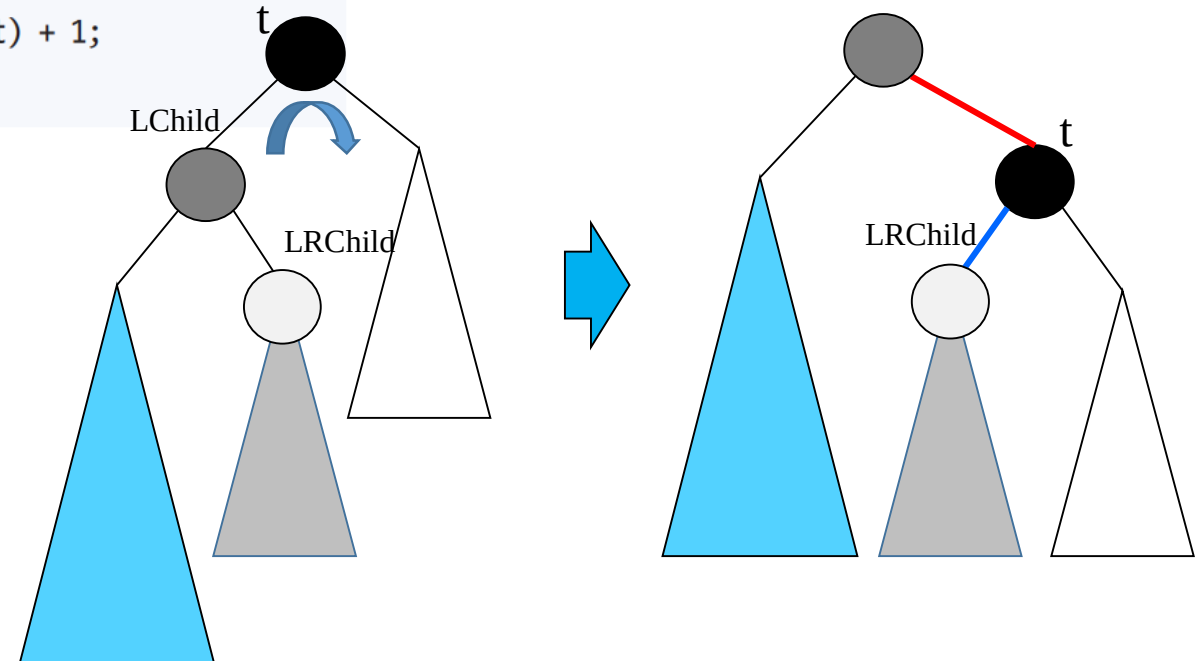
```
LRChild ← LChild.right;
```

```
LChild.right ← t;
```

```
t.left ← LRChild;
```

```
LChild.height ← max(LChild.right.height, LChild.left.height) + 1;
```

```
t.height ← max(t.right.height, t.left.height) + 1;
```



균형 맞추기

좌회전 Left Rotation

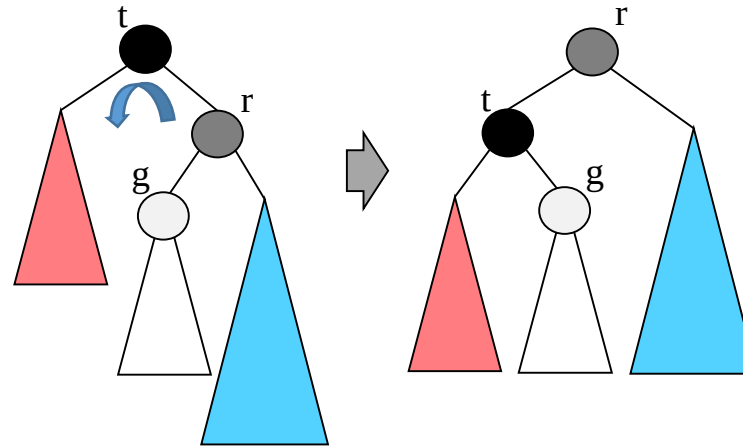
```

r ← t.right
g ← r.left
r.left ← t
t.right ← g
t.height ← max(t.right.height, t.left.height) + 1
r.height ← max(r.right.height, r.left.height) + 1
    
```

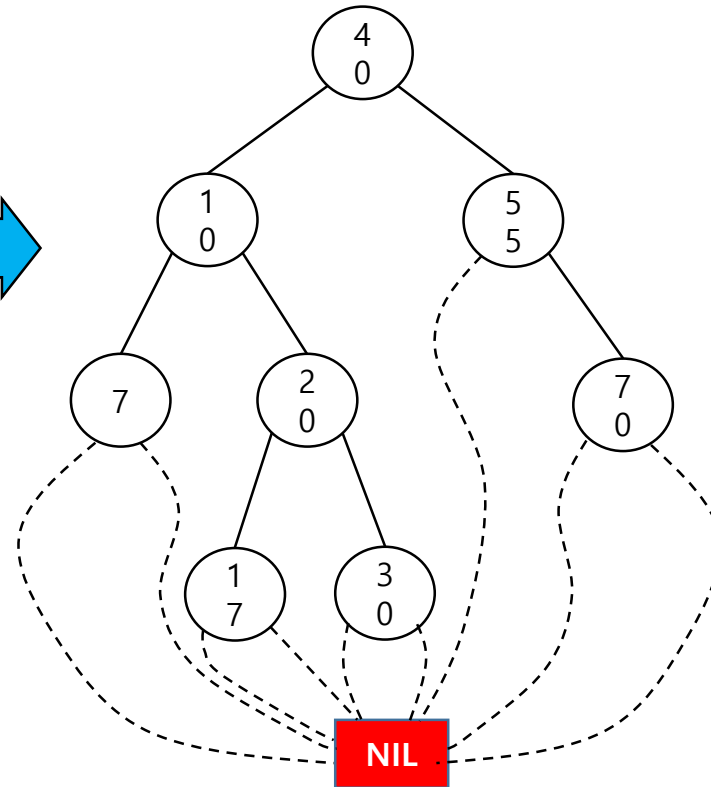
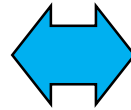
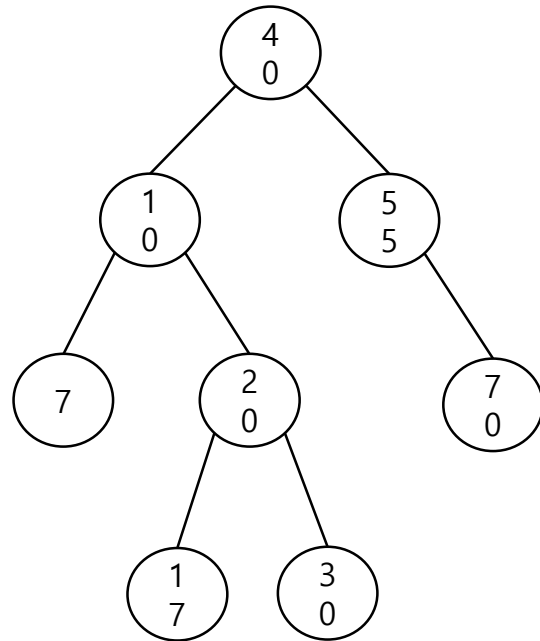
이 코드는 문제를 일으킬 수 있다

$t.right$, $t.left$, $r.right$ 가 null이면 에러 발생

통상적인 방법으로 해결하려면
코드가 지저분해진다



균형 맞추기 : 유용한 수단: 경계 노드 Sentinel



... AVLNode **NIL** = new AVLNode(null, null, null, 0); // sentinel

height 0

NIL이면 에러 안난다

```
...
t.height ← max(t.right.height, t.left.height)
+ 1
r.height ← max(r.right.height, r.left.height)
+ 1
```

null 대신 **NIL** 노드를 사용
NIL 노드는 하나만 만들면 된다
null도 정상 노드처럼 취급할 수 있다
 → 앞 페이지의 코드를 그대로 쓸 수 있다

균형 맞추기 : 끝까지 올라갈 수도 있다

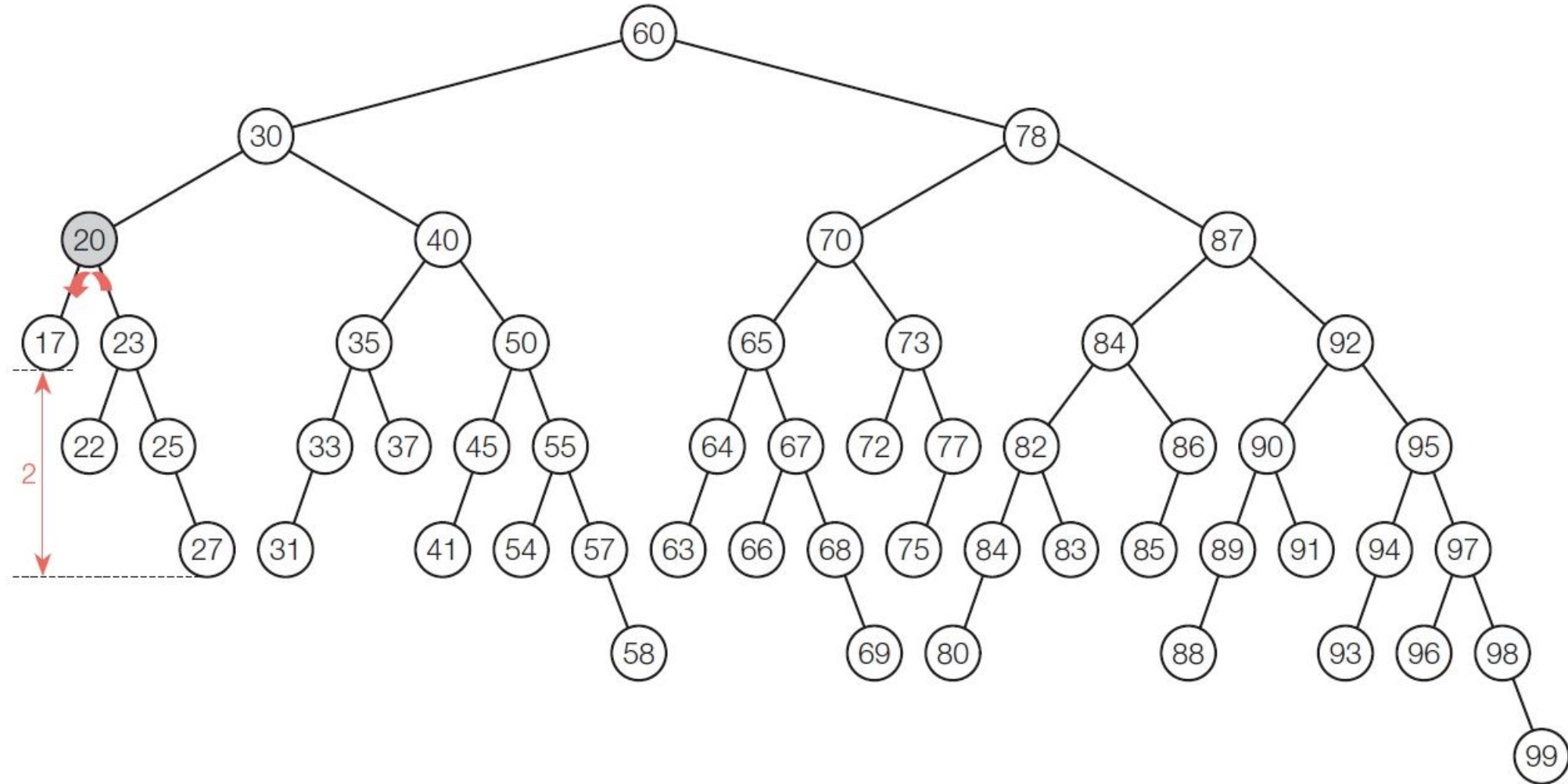


그림 11-18 RR 타입으로 균형이 깨진 예

균형 맞추기 : 끝까지 올라갈 수도 있다

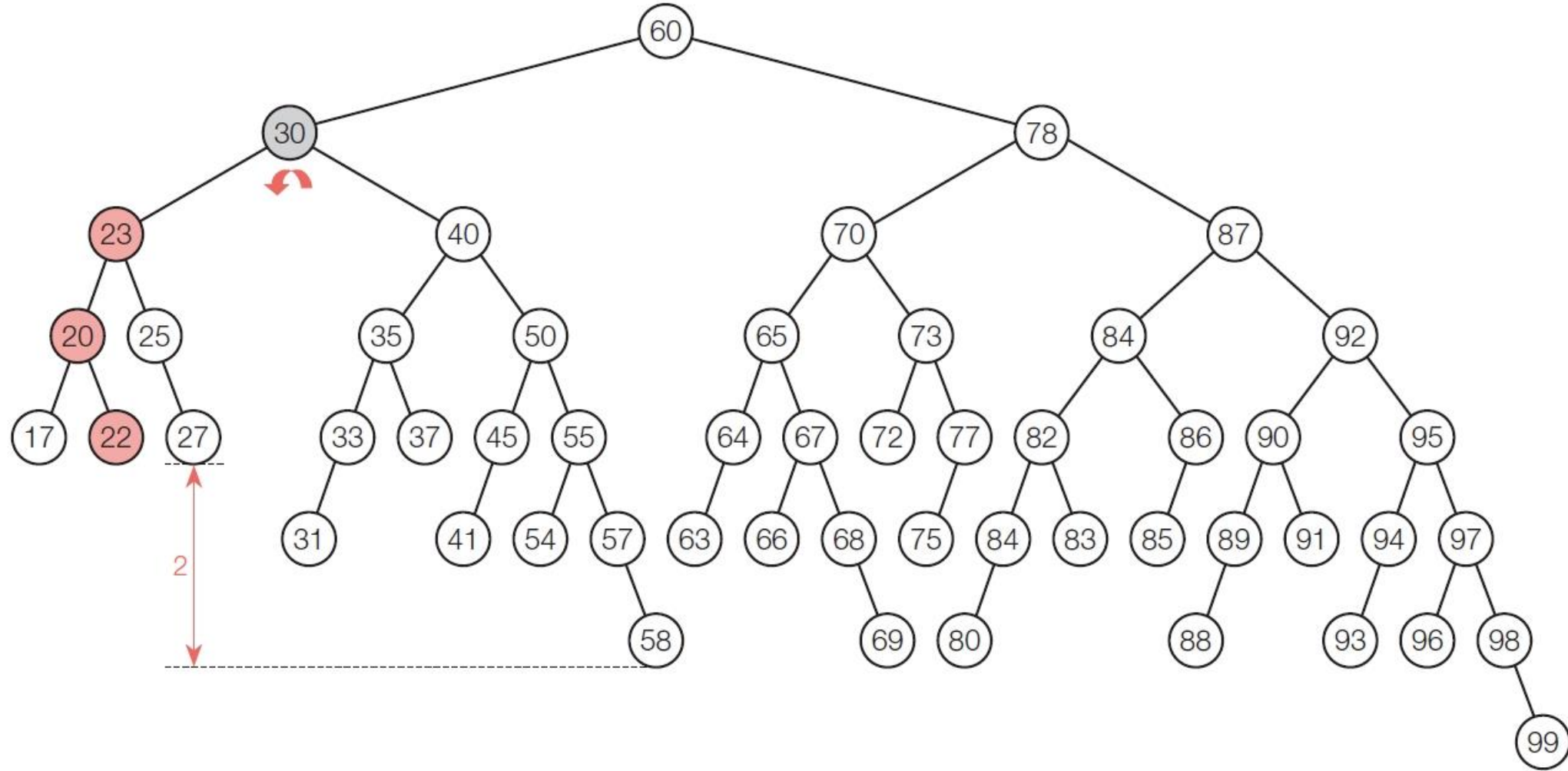


그림 11-19 한 번의 좌회전 후 상위 서브 트리에서 새롭게 균형이 깨진 상태

균형 맞추기 : 끝까지 올라갈 수도 있다

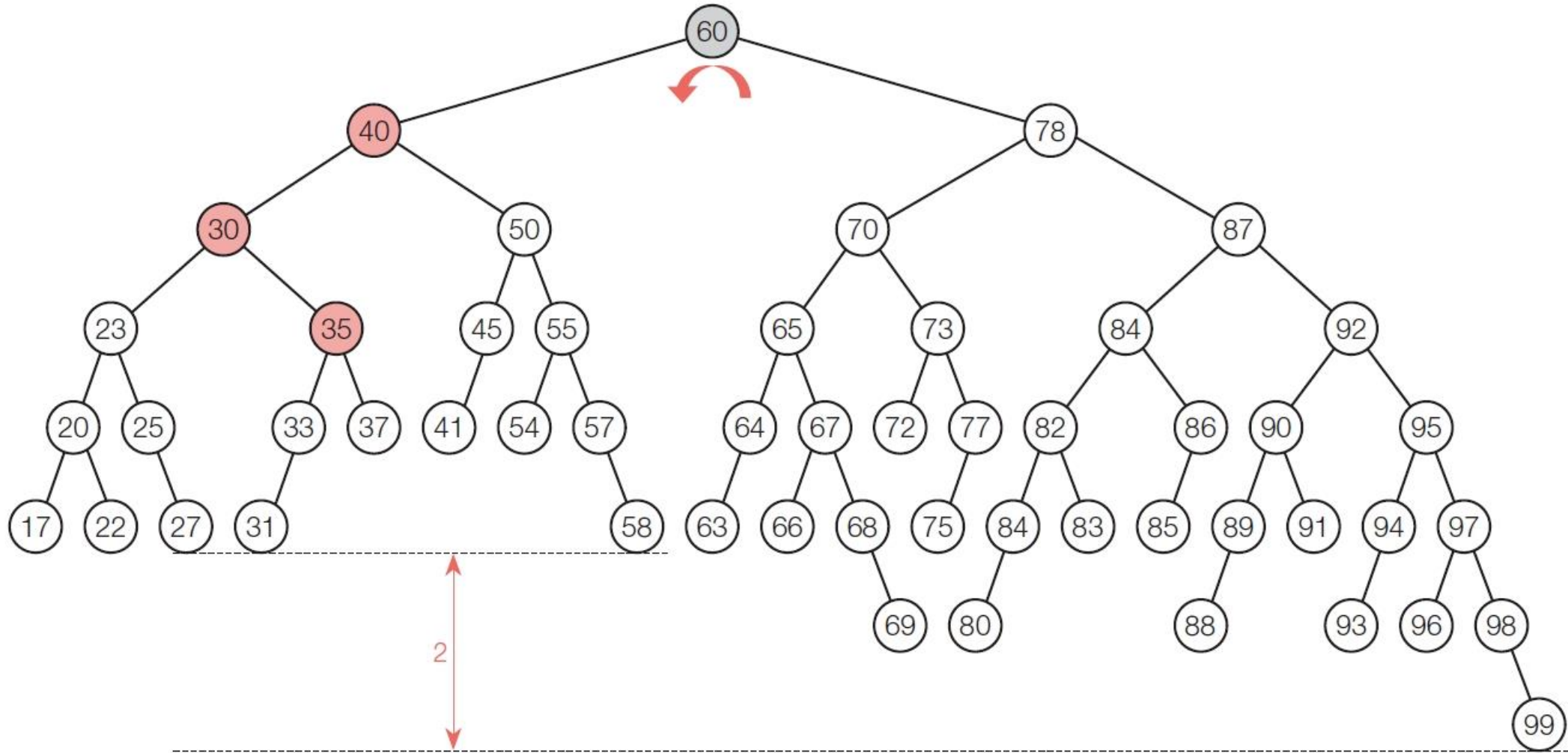


그림 11-20 두 번의 좌회전 후 상위 서브 트리에서 새롭게 균형이 깨진 상태

균형 맞추기 : 끝까지 올라갈 수도 있다

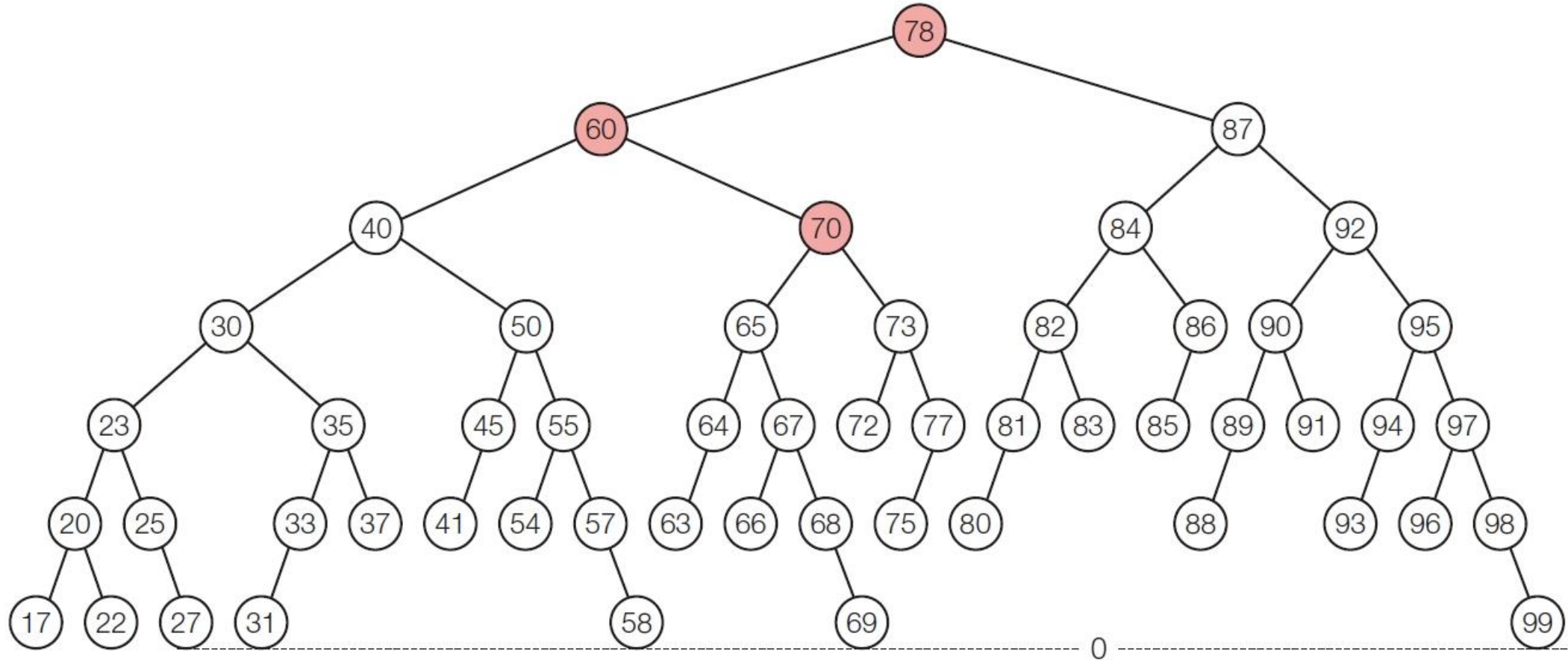


그림 11-21 세 번의 좌회전 후 균형이 해결된 상태

파이썬 코드

```
class AVLNode:
    def __init__(self, newItem, left, right, h):
        self.item = newItem
        self.left = left
        self.right = right
        self.height = h

class AVLTree:
    def __init__(self):
        self.NIL = AVLNode(None, None, None, 0)
        self.__root = self.NIL
        self.LL = 1; self.LR = 2; self.RR = 3; self.RL = 4
        self.NO_NEED = 0;
        self.ILLEGAL = -1
```

파이썬 코드

```
# [알고리즘 10-1] 구현: 검색
def search(x):
    return __searchItem(self.__root, x)

def __searchItem(self, tNode:AVLNode, x) -> AVLNode:
    if tNode == self.NIL:
        return self.NIL
    elif x == tNode.item:
        return tNode
    elif x < tNode.item:
        return __searchItem(tNode.left, x)
    else:
        return __searchItem(tNode.right, x)

# [알고리즘 10-3] 구현: 삽입
def insert(self, x):
    self.__root = self.__insertItem(self.__root, x)

def __insertItem(self, tNode:AVLNode, x) -> AVLNode:
    if tNode == self.NIL: # insert after a leaf(or into an empty tree)
        tNode = AVLNode(x, self.NIL, self.NIL, 1)
    elif x < tNode.item: # branch left
        tNode.left = self.__insertItem(tNode.left, x)
        tNode.height = 1 + max(tNode.right.height, tNode.left.height)
        type = self.__needBalance(tNode)
        if (type != self.NO_NEED):
            tNode = self.__balanceAVL(tNode, type)
    else: # branch right
        tNode.right = self.__insertItem(tNode.right, x)
        tNode.height = 1 + max(tNode.right.height, tNode.left.height)
        type = self.__needBalance(tNode)
        if type != self.NO_NEED:
            tNode = self.__balanceAVL(tNode, type)

    return tNode
```

파이썬 코드

```
# [알고리즘 10-7] 구현: 삭제
def delete(self, x):
    self.__root = self.__deleteItem(self.__root, x)

def __deleteItem(self, tNode:AVLNode, x) -> AVLNode:
    if tNode == self.NIL:
        return self.NIL # item not found!
    else:
        if x == tNode.item: # item found at tNode
            tNode = self.__deleteNode(tNode)
        elif x < tNode.item:
            tNode.left = self.__deleteItem(tNode.left, x)
            tNode.height = 1 + max(tNode.right.height, tNode.left.height)
            type = self.__needBalance(tNode)
            if type != self.NO_NEED:
                tNode = self.__balanceAVL(tNode, type)
        else:
            tNode.right = self.__deleteItem(tNode.right, x)
            tNode.height = 1 + max(tNode.right.height, tNode.left.height)
            type = self.__needBalance(tNode)
            if type != self.NO_NEED:
                tNode = self.__balanceAVL(tNode, type)
        return tNode

def __deleteNode(self, tNode:AVLNode) -> AVLNode:
    # 3가지 case
    # 1. tNode이 리프 노드
    # 2. tNode이 자식이 하나만 있음
    # 3. tNode이 자식이 둘 있음
    if tNode.left == self.NIL and tNode.right == self.NIL: # case 1(자식이 없음)
        return self.NIL
    elif tNode.left == self.NIL: # case 2(오른 자식뿐)
        return tNode.right
    elif tNode.right == self.NIL: # case 2(왼 자식뿐)
        return tNode.left
    else: # case 3(두 자식이 다 있음)
        (rtnItem, rtnNode) = self.__deleteMinItem(tNode.right)
        tNode.item = rtnItem
        tNode.right = rtnNode
        tNode.height = 1 + max(tNode.right.height, tNode.left.height)
        type = self.__needBalance(tNode)
        if type != self.NO_NEED:
            tNode = self.__balanceAVL(tNode, type)
        return tNode # tNode survived
```

파이썬 코드

```
def __deleteMinItem(self, tNode:AVLNode) -> tuple:
    if tNode.left == self.NIL:
        # found min at tNode
        return (tNode.item, tNode.right)
    else: # keep branching left, then backtrack
        (rtnItem, rtnNode) = self.__deleteMinItem(tNode.left)
        tNode.left = rtnNode
        tNode.height = 1 + max(tNode.right.height, tNode.left.height)
        type = self.__needBalance(tNode)
        if type != self.NO_NEED:
            tNode = self.__balanceAVL(tNode, type)
        return (tNode, rtnItem)

# 균형 잡기
def __balanceAVL(self, tNode:AVLNode, type:int) -> AVLNode:
    returnNode = self.NIL
    if type == self.LL:
        returnNode = self.__rightRotate(tNode)
    elif type == self.LR:
        tNode.left = self.__leftRotate(tNode.left)
        returnNode = self.__rightRotate(tNode)
    elif type == self.RR:
        returnNode = self.__leftRotate(tNode)
    elif type == self.RL:
        tNode.right = self.__rightRotate(tNode.right)
        returnNode = self.__leftRotate(tNode)
    else:
        print("Impossible type! Should be one of LL, LR, RR, RL")
    return returnNode
```

파이썬 코드

```
# [알고리즘 11-1] 구현 : 좌회전
def __leftRotate(self, t:AVLNode) -> AVLNode:
    RChild = t.right
    if RChild == self.NIL:
        print(t.item, "'s RChild shouldn't be NIL!")
    RLChild = RChild.left
    RChild.left = t
    t.right = RLChild
    t.height = 1 + max(t.left.height, t.right.height)
    RChild.height = 1 + max(RChild.left.height, RChild.right.height)
    return RChild

# [알고리즘 11-1] 구현 : 우회전
def __rightRotate(self, t:AVLNode) -> AVLNode:
    LChild = t.left
    if LChild == self.NIL:
        print(t.item, "'s LChild shouldn't be NIL!")
    LRChild = LChild.right
    LChild.right = t
    t.left = LRChild
    t.height = 1 + max(t.left.height, t.right.height)
    LChild.height = 1 + max(LChild.left.height, LChild.right.height)
    return LChild

def __needBalance(self, t:AVLNode) -> int:
    type = self.ILLEGAL
    if (t.left.height + 2 <= t.right.height): # R 유형
        if (t.right.left.height) <= t.right.right.height:
            type = self.RR
        else:
            type = self.RL;
    elif (t.left.height) >= t.right.height + 2: # L 유형
        if (t.left.left.height) >= t.left.right.height:
            type = self.LL
        else:
            type = self.LR
    else:
        type = self.NO_NEED
    return type
```

```
# 기타
def isEmpty(self) -> bool:
    return self.__root == self.NIL

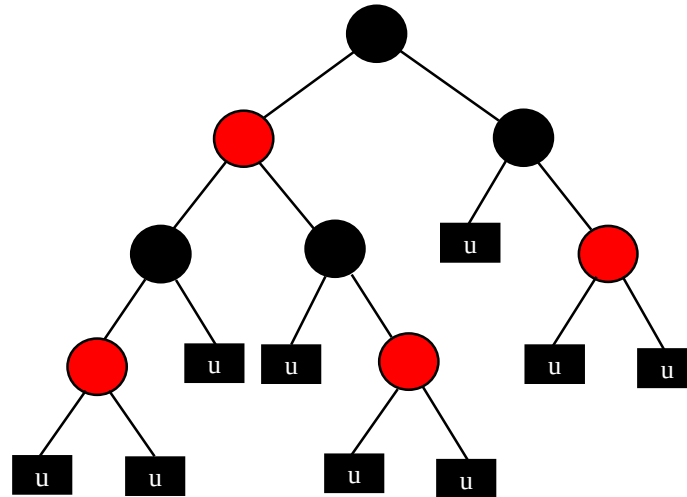
def clear(self):
    self.__root = self.NIL
```

AVL Tree

Red-Black Tree, RB Tree

레드-블랙 트리

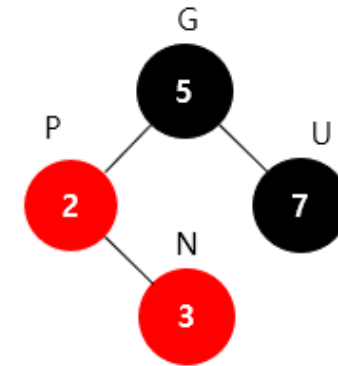
- 레드-블랙 트리는 자가 균형 이진 탐색 트리이다.
- 레드-블랙 트리는 다음과 같은 조건들을 만족한다.
 1. 모든 노드는 빨간색 혹은 검은색이다.
 2. 루트 노드는 검은색이다.
 3. 모든 리프 노드(NIL)들은 검은색이다. (NIL : null leaf, 자료를 갖지 않고 트리의 끝을 나타내는 노드)
 4. 빨간색 노드의 자식은 검은색이다. No Double Red(빨간색 노드가 연속으로 나올 수 없다)
 5. 모든 리프 노드에서 Black Depth는 같다. 리프노드에서 루트 노드까지 가는 경로에서 만나는 검은색 노드의 개수가 같다.



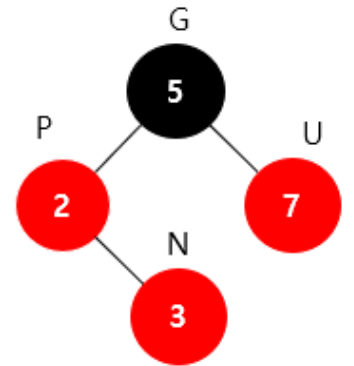
Red/Black Tree

RB Tree의 삽입

- 레드-블랙 트리에 새로운 노드를 삽입할 때 새로운 노드는 항상 빨간색으로 삽입한다.
- 이렇게 되면 4번 조건이 위배되는 상황이 발생한다. 즉, 빨간색 노드가 연속으로 2번 나타날 수 있다(Double Red)
- 앞으로 새로 삽입할 노드를 N(New), 부모 노드를 P(Parent), 조상 노드를 G(Grand Parent), 삼촌 노드를 U(Uncle)라고 하자. 즉, 삼촌 노드는 말 그대로 부모의 형제라고 생각하면 된다.
- Double Red가 발생했을 때
 - 삼촌 노드가 검은색이라면 -> Restructuring을 수행하면 된다.
 - 삼촌 노드가 빨간색이라면 -> Recoloring을 수행하면 된다.



U가 검은색 -> Restructuring



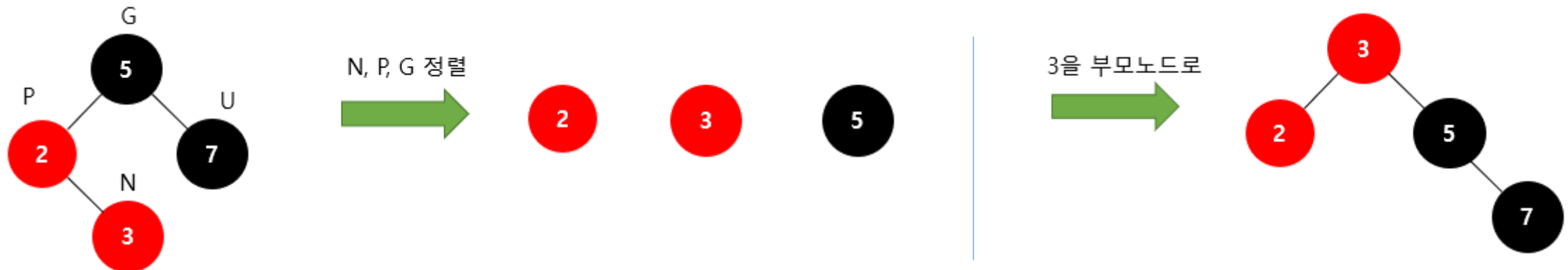
U가 빨간색 -> Recoloring

Red-Black Tree, RB Tree

Restructuring

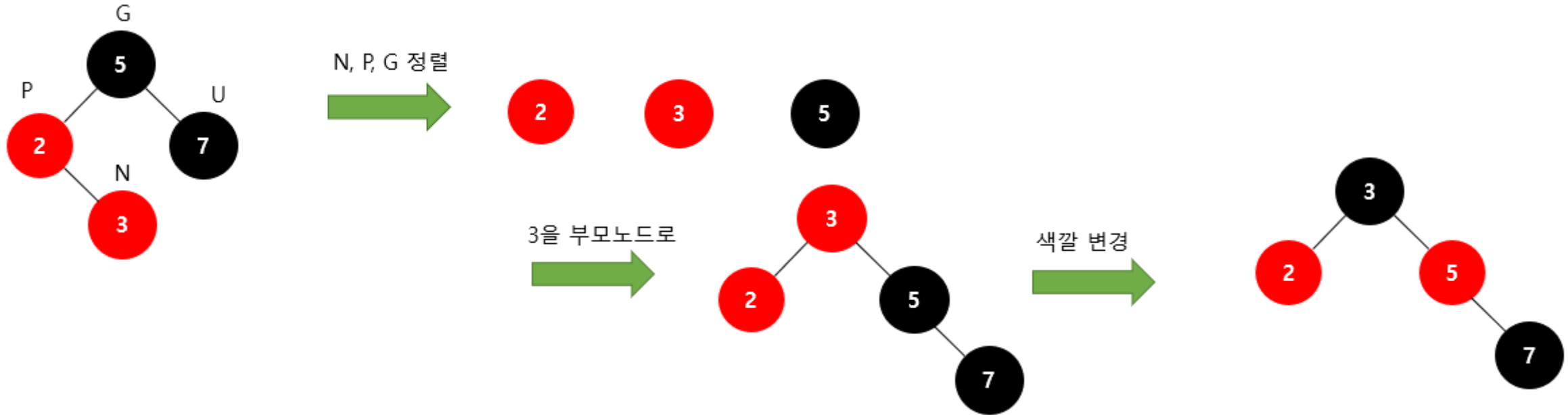


- 위와 같은 상황이 발생했다고 가정하자. Double Red가 발생했는데 삼촌 노드가 검은색이다. 따라서 Restructuring을 수행한다.
- 먼저 새로운 노드 N과 부모 P, 조상 G를 오름차순으로 정렬한다. 그러면 3이 중간값이 된다.
- 따라서 중간값인 3을 부모 노드로 바꾸고 나머지 2와 5를 자식 노드로 바꾼다.
- 당연히 원래 5의 자식 노드였던 7은 딸려가게 된다.



Red-Black Tree, RB Tree

Restructuring



- 마지막으로 새롭게 부모가 된 3을 검은색으로 바꿔주고 나머지 두 자식인 2, 5의 색을 빨간색으로 바꿔주면 Double Red 문제가 해결된다!!
- 여기서 많이들 헷갈리는 게 완성된 트리가 규칙 3(모든 리프 노드는 검은색)을 만족하지 않는 것처럼 보일 수 있다. 값이 2인 노드는 자식 노드 NIL 2개를 가지고 있고 그 NIL들이 검은색이라고 생각하면 된다.

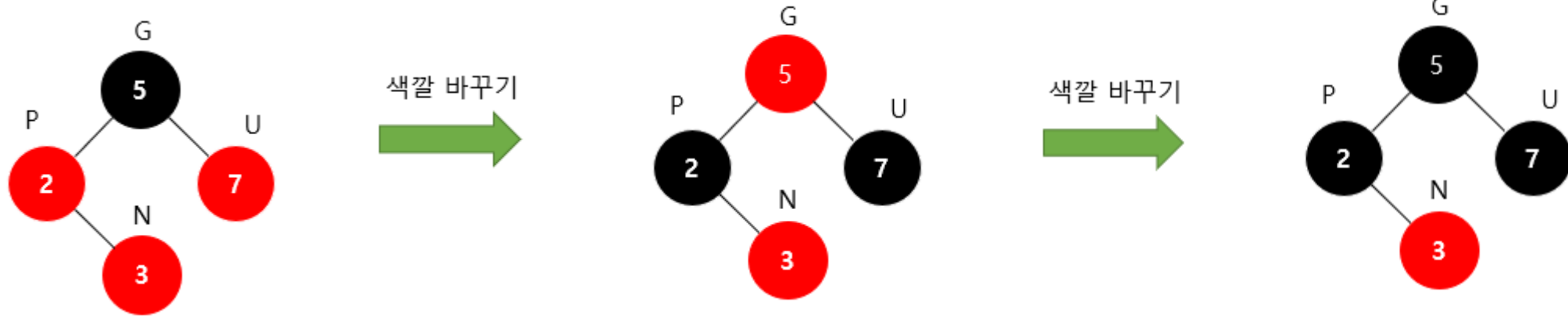
Recoloring



- 위와 같은 상황을 가정하자. Double Red가 발생했는데 삼촌 노드가 빨간색이다. 따라서 Recoloring을 수행한다.
- 먼저 부모(P)와 삼촌(U)을 검은색으로 바꾸고, 조상(G)을 빨간색으로 바꾼다.

Red-Black Tree, RB Tree

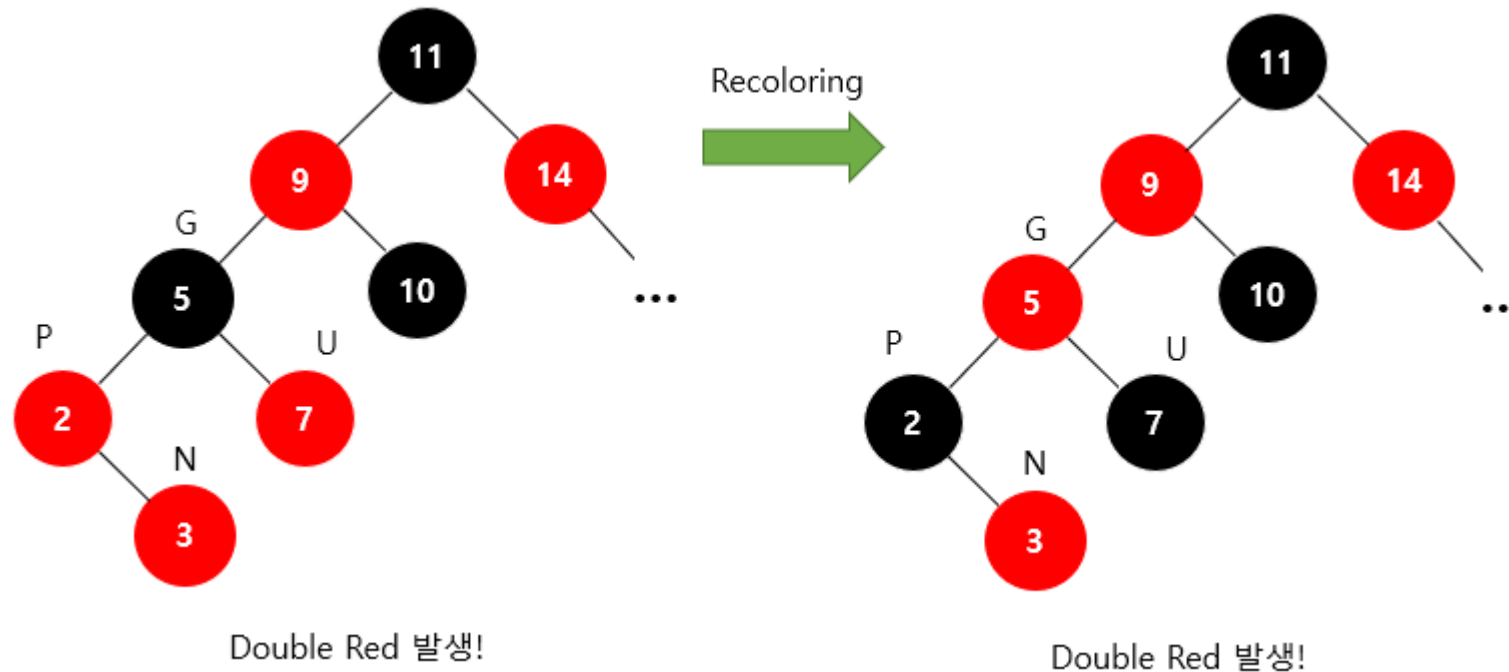
Recoloring



- 하지만 루트 노드는 검은색이라는 조건을 지켜야 하므로, 루트 노드를 검은색으로 바꾼다.
- 이렇게 하면 모든 조건이 지켜지면서 Double Red 문제가 해결된다.
- 검은색 노드는 2번 나와도 되냐고 묻는다면 Yes이다. 빨간색 노드가 2번 나오면 안 되는 것이다.

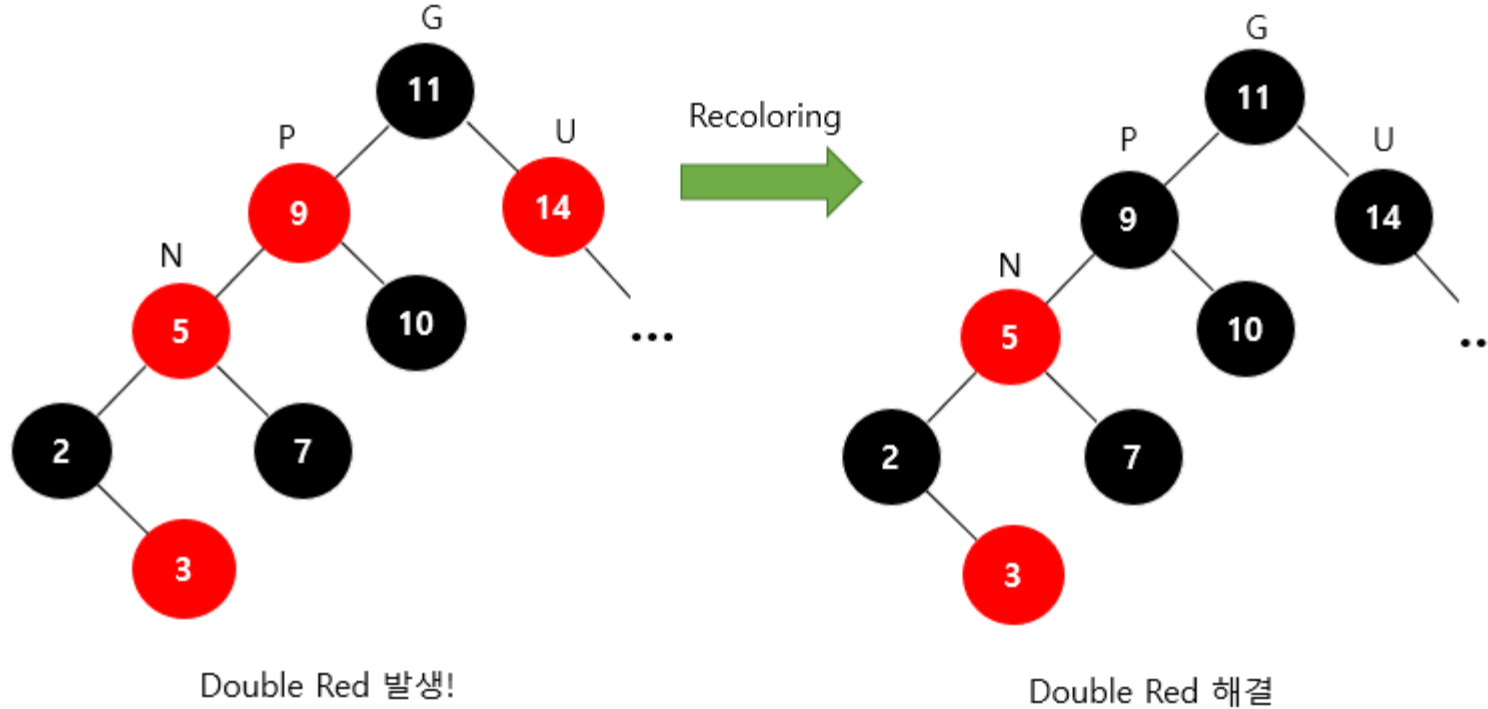
Recoloring

- Recoloring은 간단해 보이지만 문제는 조상 노드(G)가 루트 노드가 아니면서, 조상 노드(G)가 또다시 Double Red 문제가 발생하는 경우이다.



- 위와 같은 상황을 가정하자. 왼쪽 트리에서 Recoloring을 진행하면 오른쪽 트리가 된다.
- 이때 조상 노드(G)가 또다시 Double Red가 발생하게 된다.

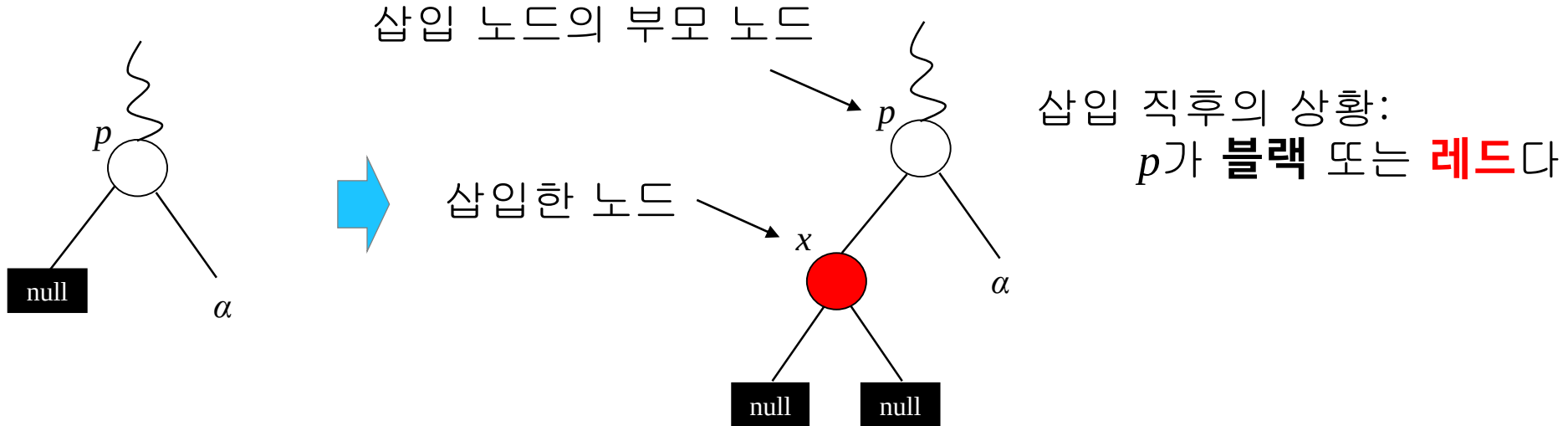
Recoloring



- Double Red 문제가 발생한 "값이 5인 노드"를 기준으로 다시 한번 살펴보자.
- 해당 노드의 삼촌(U)이 빨간색이므로 다시 Recoloring을 진행해주면 Double Red 문제를 해결할 수 있다!
- 만약 해당 노드의 삼촌(U)가 검은색이었다면 Restructuring을 진행해주면 된다.

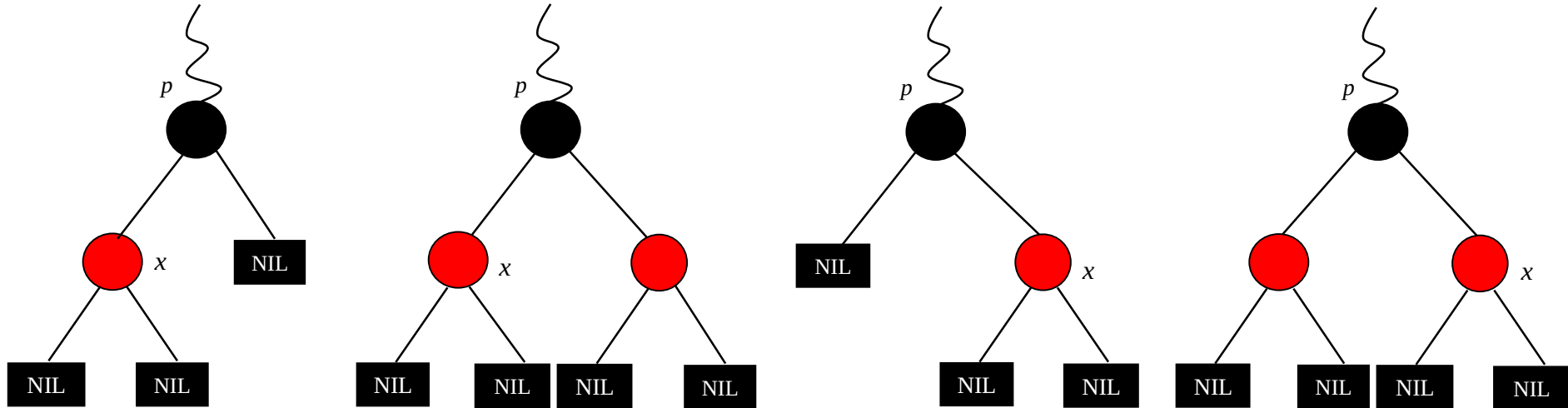
RB Tree의 삽입

- 일반적인 BST의 삽입 작업 후, 삽입 노드에 레드를 칠하고, 삽입한 노드의 좌우에 null 리프를 달아 준다



RB Tree의 삽입

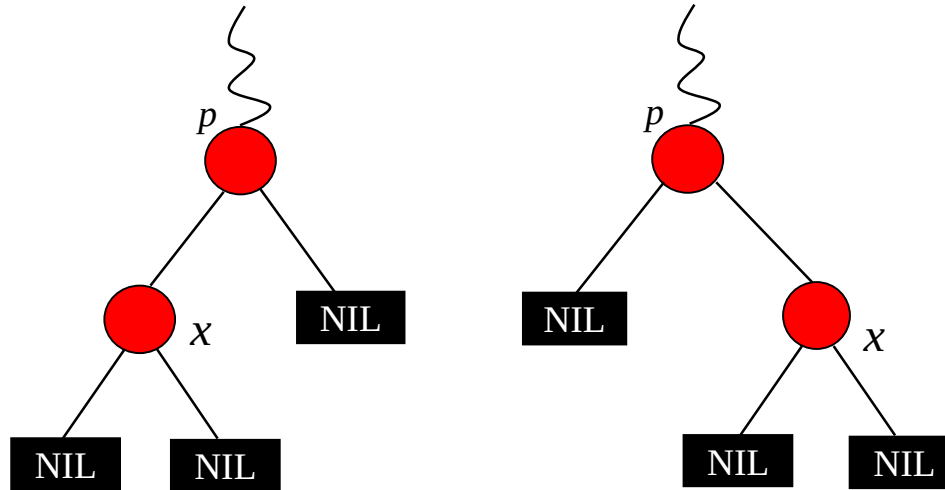
- 삽입 직후의 상황: x 의 부모 p 가 블랙 또는 레드다
- If p 가 블랙
 - RB 특성 다 만족. 완료!



가능한 모양은 이 4가지 뿐이다

RB Tree의 삽입

- If p 가 레드
 - RB 특성이 깨졌다 → 수선 (다음 페이지 이후)



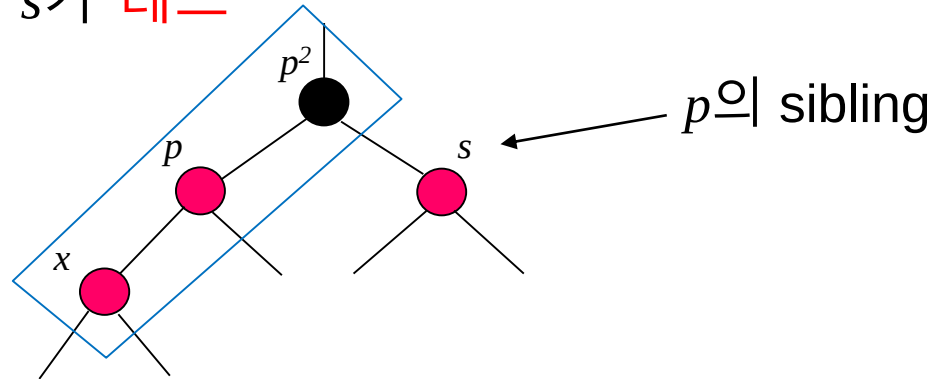
가능한 모양은 이 2가지 뿐이다

Red-Black Tree, RB Tree

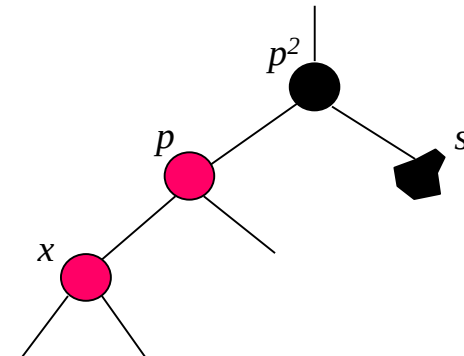
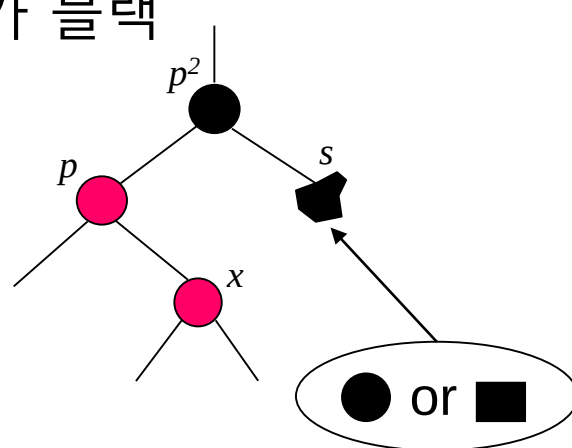
RB Tree의 삽입

- If p 가 레드
 - p 의 형제 sibling 노드 s 에 따라 두 가지로 나눈다

Case 1: s 가 레드



Case 2: s 가 블랙

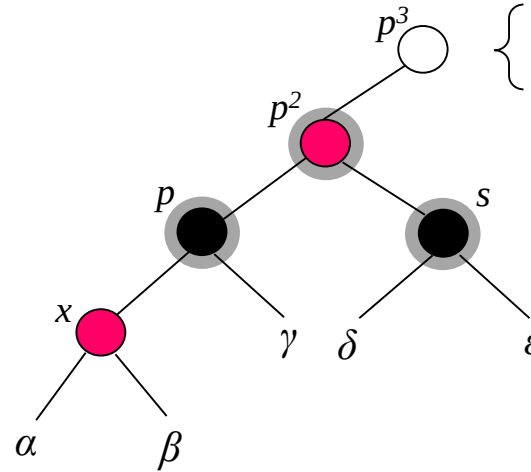
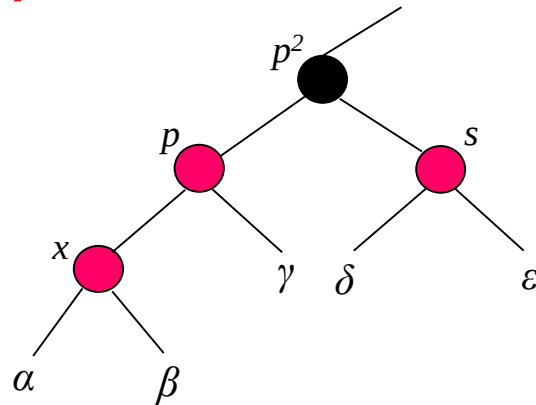


Red-Black Tree, RB Tree

RB Tree의 삽입

- If p 가 레드

Case 1: s 가 **레드**



$\begin{cases} p^3 \text{가 블랙이면 완료} \\ p^3 \text{가 레드이면 } p^2 \text{가 새 } x : \text{Recursive!} \end{cases}$

● : 색이 바뀐 노드

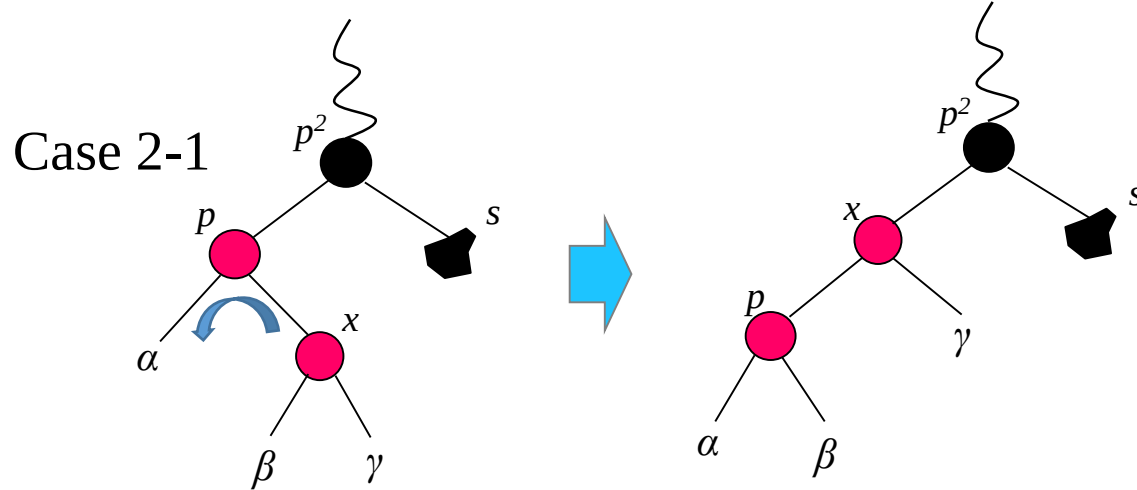
p 와 s 를 **블랙**으로 바꾸고,
 p^2 을 **레드**로 바꾼다.

Red-Black Tree, RB Tree

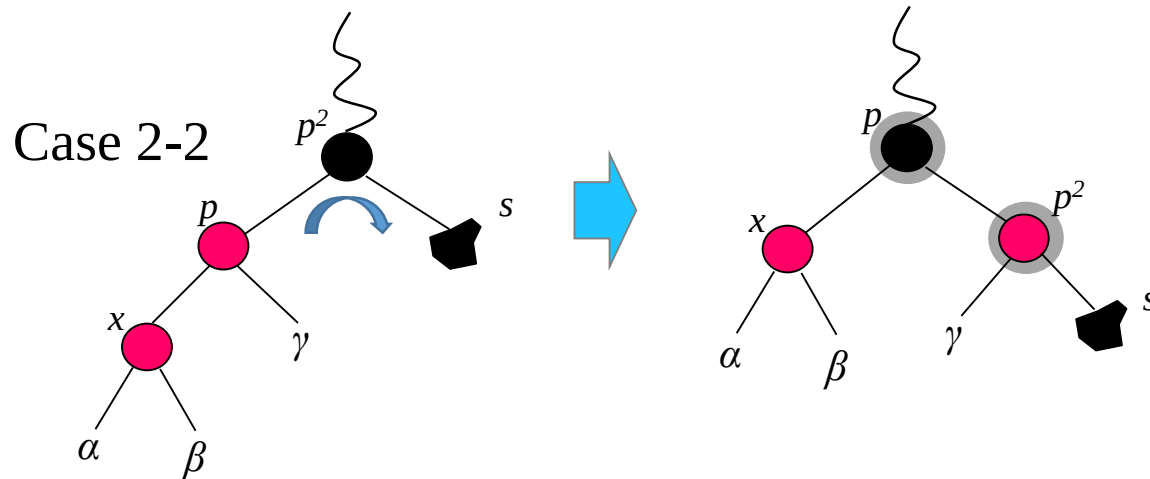
RB Tree의 삽입

- If p 가 레드

Case 2: s 가 블랙



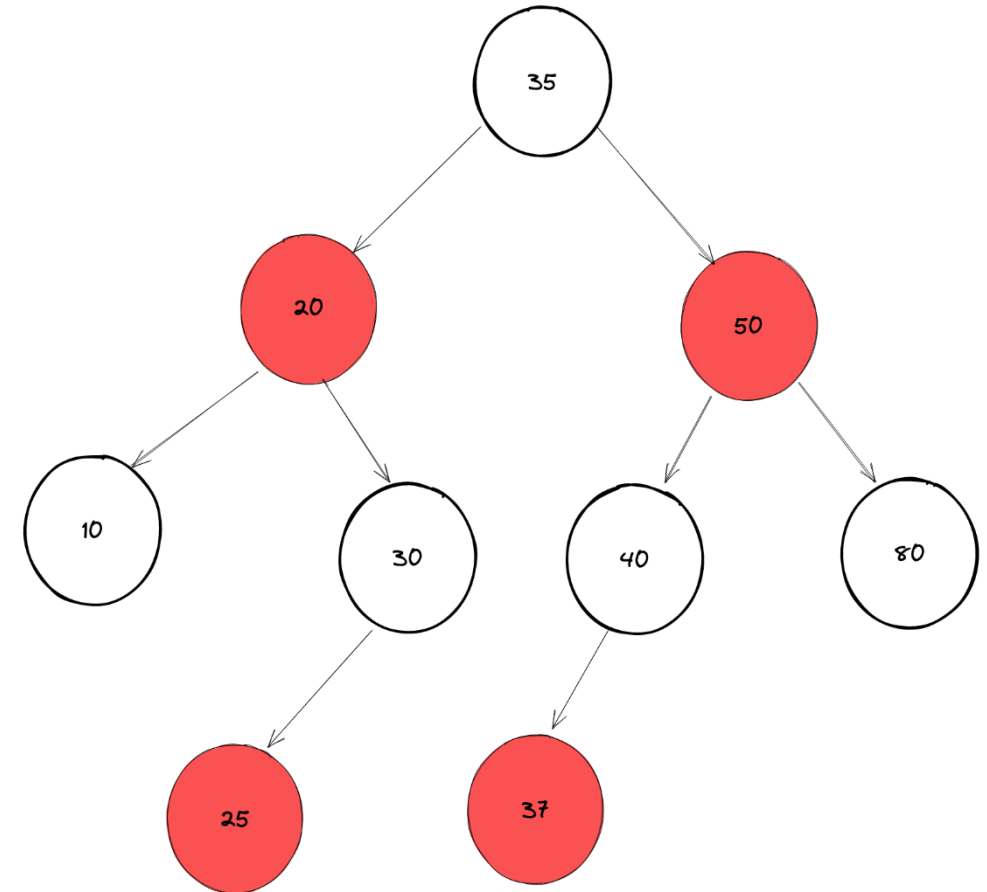
Case 2-2로 변환



우회전하고, p 와 p^2 의 색을 바꾼다.
수선 끝.

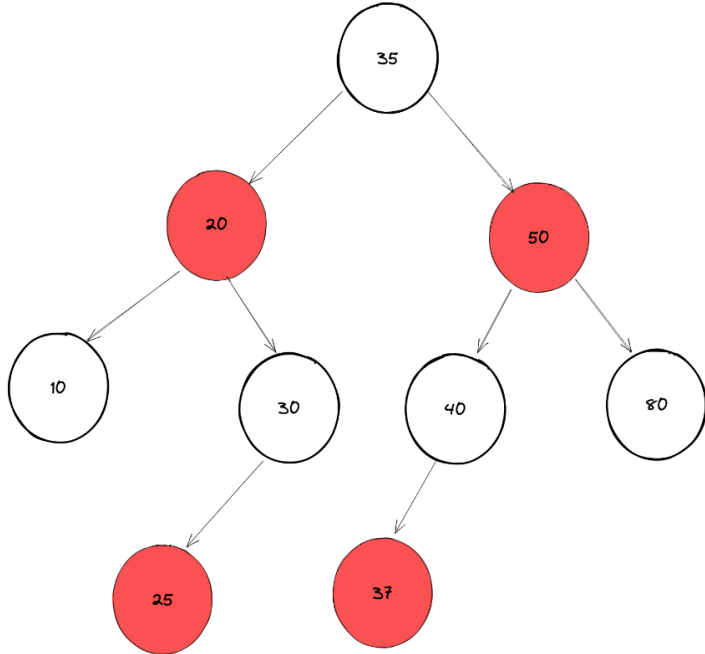
RB Tree의 삭제

- 과정
 1. 삭제 전 rb 트리 속성을 만족한 상태
 2. 삭제 방식은 일반적인 bst와 동일
 3. 삭제 후 rb 트리 속성 위반 여부를 확인
 - 위반했다면 재조정
- 속성 위반 여부를 확인하는 방법은 삭제되는 색을 통해

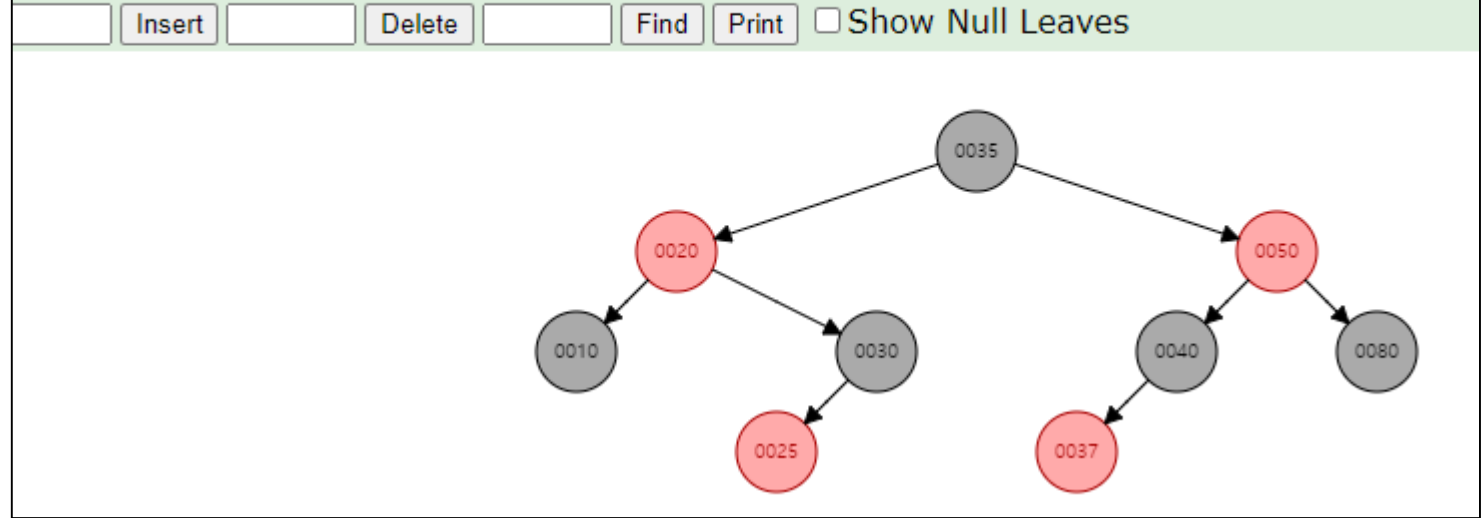


Red-Black Tree, RB Tree

RB Tree의 삭제



Red/Black Tree



- 삭제하려는 노드의 자녀가 둘이라면
 → 삭제되는 색은 삭제되는 노드의 successor의 색
 successor: 삭제하려는 노드의 오른쪽 서브트리에서 가장 작은 값
 20 삭제 → successor 25 → red 삭제
 35 삭제 → successor 37 → red 삭제
 50 삭제 → successor 80 → black 삭제

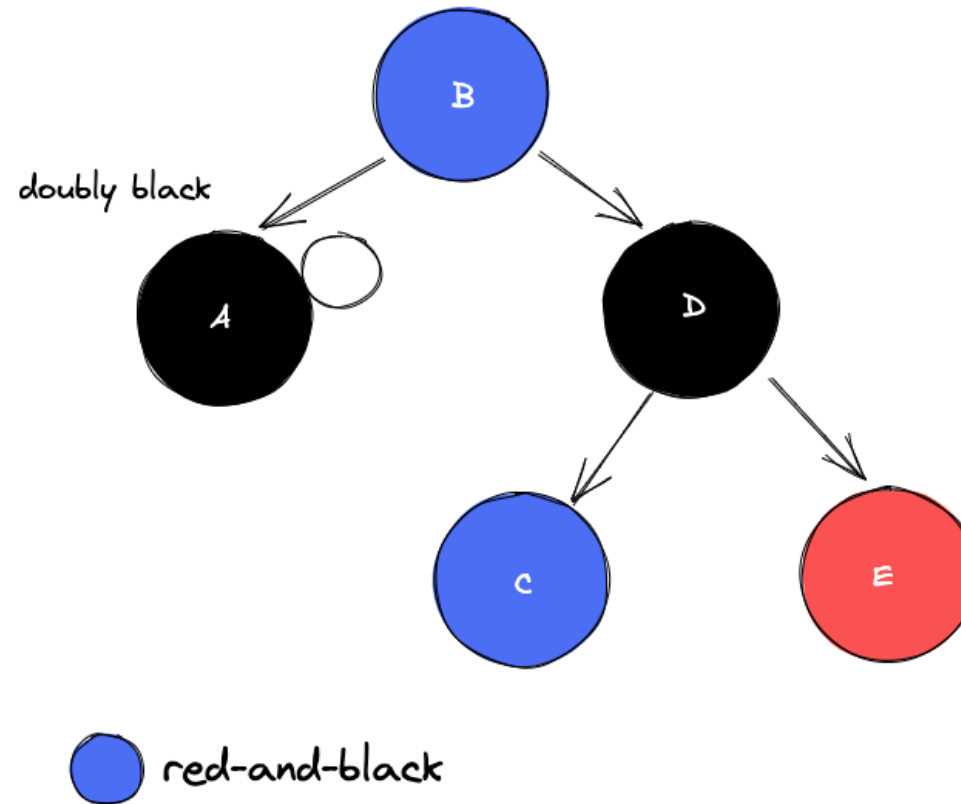
RB Tree의 삭제

- 속성 위반 여부 확인
- 삭제되는 색이 red라면 어떠한 속성도 위반하지 않는다.
- 삭제되는 색이 black이라면 2,4,5번 속성을 위반할 수 있다.
 - 2번 속성을 위반했을 경우 루트 노드를 black으로 변경하면 됨
 - black노드를 삭제한 경우 대부분 5번 속성을 위반하게 됨 →
 - 빠르게 5번 속성을 만족시키기 위해 삭제된 노드를 대체한 노드에 extra black 속성을 부여한다.
 - ✓ extra black: 경로에서 black 수를 카운트 할 때 하나의 black으로 카운드된다.
 - ✓ doubly black: extra black이 부여된 black 노드
 - ✓ red-and-black | extra black이 부여된 red 노드
- 해결방법
 - red-and-black일 경우: black으로 바꾸면 해결
 - doubly black일 경우: 4가지의 경우가 있음
 - 4가지로 분류된 기준은 doubly black의 형제의 색과 그 형제의 자녀들의 색

Red-Black Tree, RB Tree

RB Tree의 삭제 - case 4

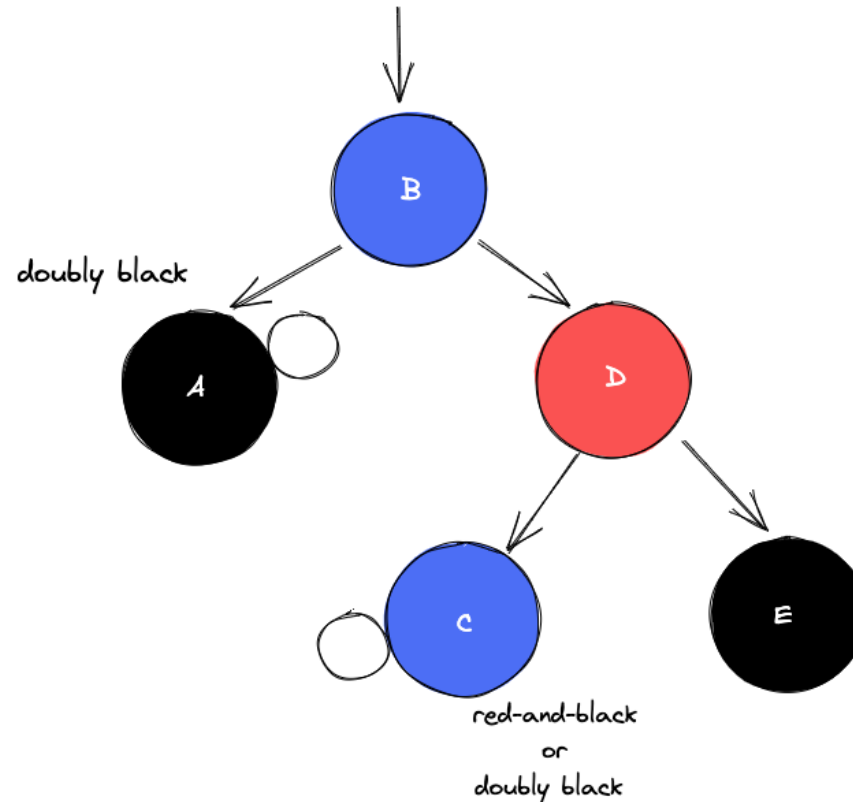
- case 4: doubly black의 오른쪽 형제가 black & 그 형제의 오른쪽 자녀가 red



Red-Black Tree, RB Tree

RB Tree의 삭제 - case 4

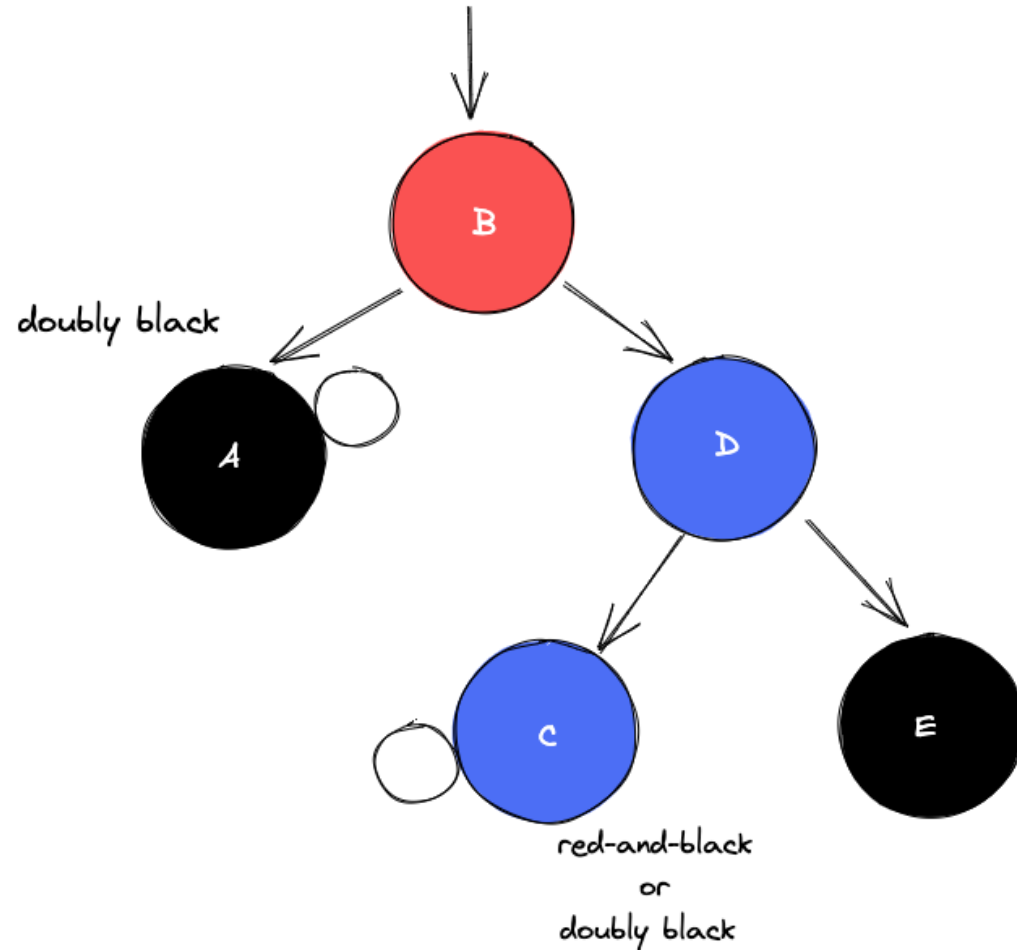
- red를 doubly black 위로 옮기고 옮긴 red로 extra black을 전달해서 red-and-black으로 만들면 red-and-black을 black으로 바꿔서 해결
- red를 왼쪽으로 보내기 위해서는 d의 위치가 red가 되어야함



Red-Black Tree, RB Tree

RB Tree의 삭제 - case 4

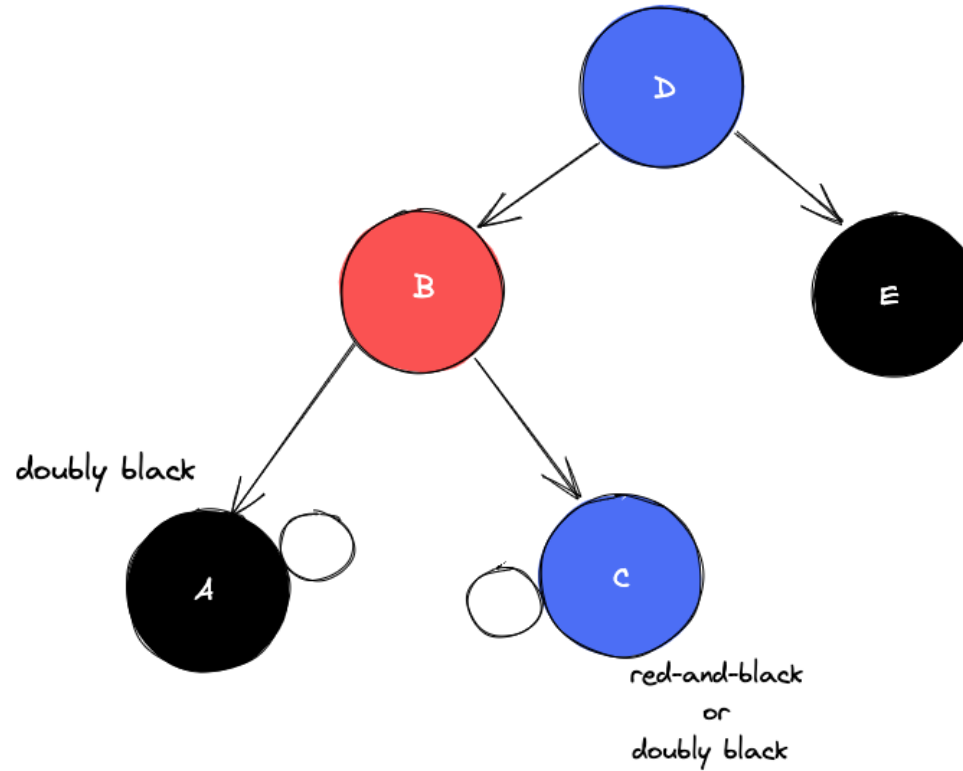
- red가 왼쪽으로 넘어갈 수 있도록 왼쪽으로 회전 전에 b와 d의 색을 바꿔준다.



Red-Black Tree, RB Tree

RB Tree의 삭제 - case 4

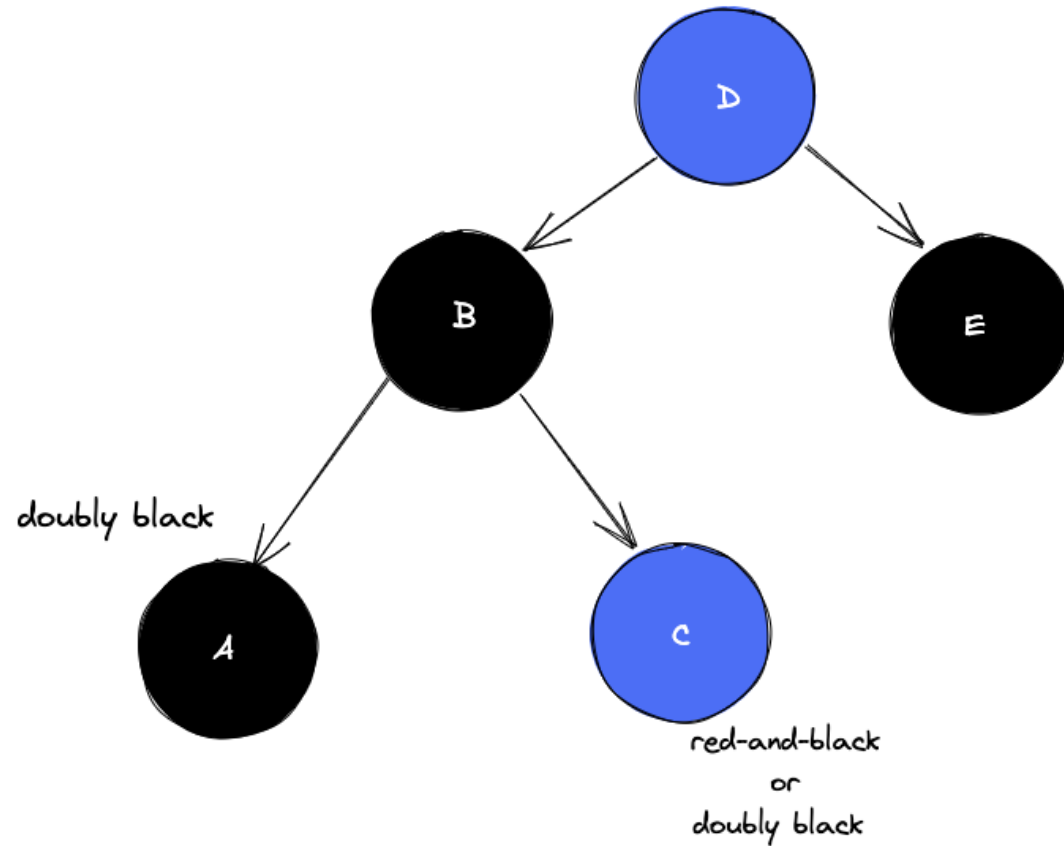
- B를 기준으로 왼쪽으로 회전한다.



Red-Black Tree, RB Tree

RB Tree의 삭제 - case 4

- A와 C의 extra black을 모아 부모로 전달해 B를 black으로 만들어 문제를 해결한다.



RB Tree의 삭제 - case 4

- 결과론적으로
 - 오른쪽 형제(D)는 부모의 색으로
 - 오른쪽 형제의 오른쪽 자녀(E)는 black으로
 - 부모는 black으로 바꾼 후에 부모를 기준으로 왼쪽 회전하면 해결
- 오른쪽 왼쪽을 바꿔도 성립

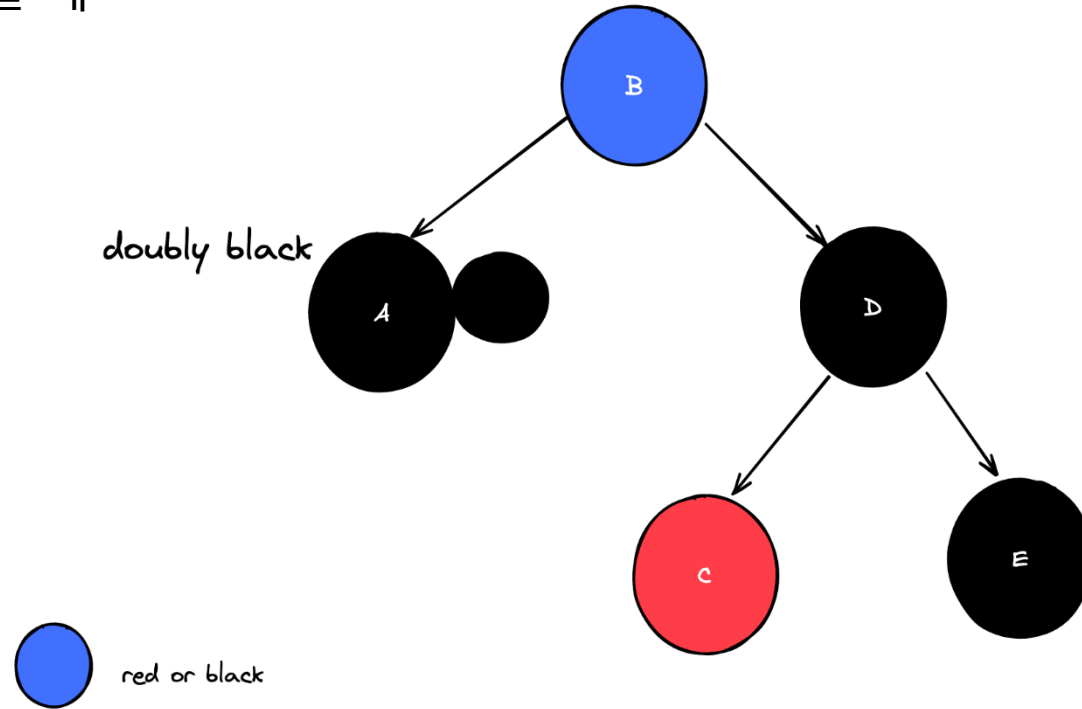
RB Tree의 삭제 - case 4

- 결과론적으로
 - 오른쪽 형제(D)는 부모의 색으로
 - 오른쪽 형제의 오른쪽 자녀(E)는 black으로
 - 부모는 black으로 바꾼 후에 부모를 기준으로 왼쪽 회전하면 해결
- 오른쪽 왼쪽을 바꿔도 성립

Red-Black Tree, RB Tree

RB Tree의 삭제 - case 3

- case 3: doubly black의 오른쪽 형제가 black & 그 형제의 왼쪽 자녀가 red & 그 형제의 오른쪽 자녀가 black일 때

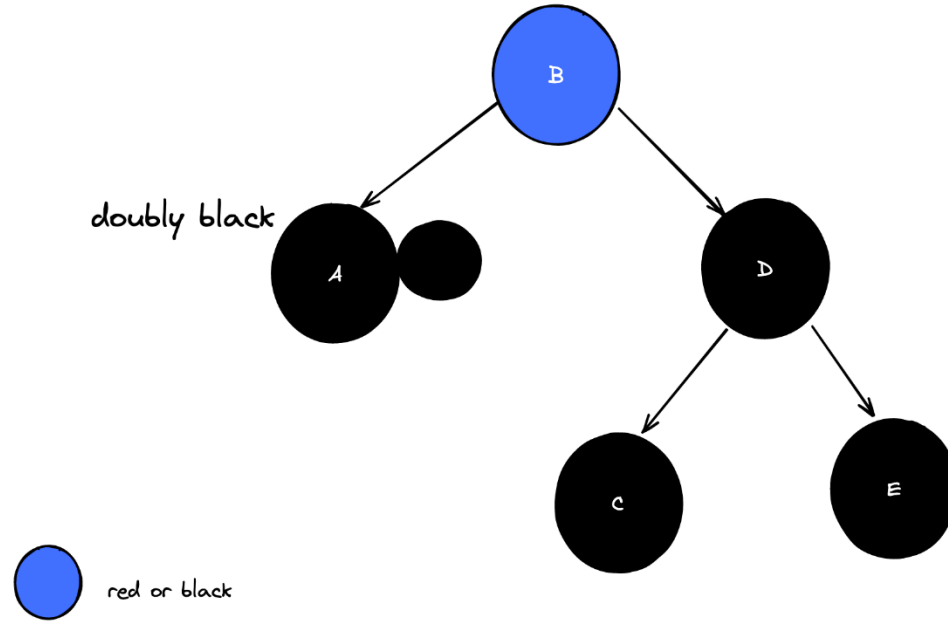


- doubly black의 형제의 오른쪽 자녀가 red가 되게 만들어서 이후에 case 4를 적용하면 해결!
- E 위치에 red가 오게 만들려면 C와 D의 색을 바꾼 후 D를 기준으로 오른쪽 회전하면 된다.

Red-Black Tree, RB Tree

RB Tree의 삭제 - case 2

- case 2: doubly black의 형제가 black & 그 형제의 두 자녀 모두 black

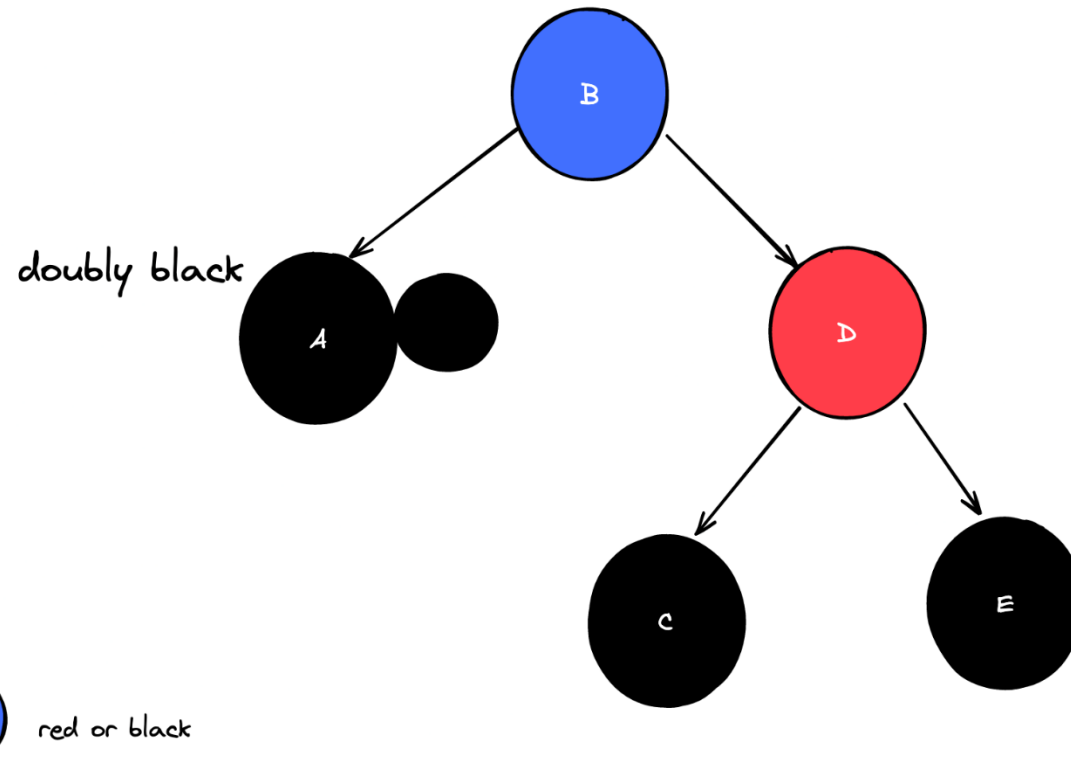


- doubly black과 그 형제의 black을 모아서 부모에게 전달 후 부모가 extra black을 해결하도록 위임
 - B가 red-and-black이면 black으로 변경 후 끝
 - B가 doubly black이면
 - B가 루트이면 black으로 바뀌서 해결
 - 아니라면 case 1,2,3,4 중 하나로 해결

Red-Black Tree, RB Tree

RB Tree의 삭제 - case 1

- case 1: doubly black의 형제가 red일 때

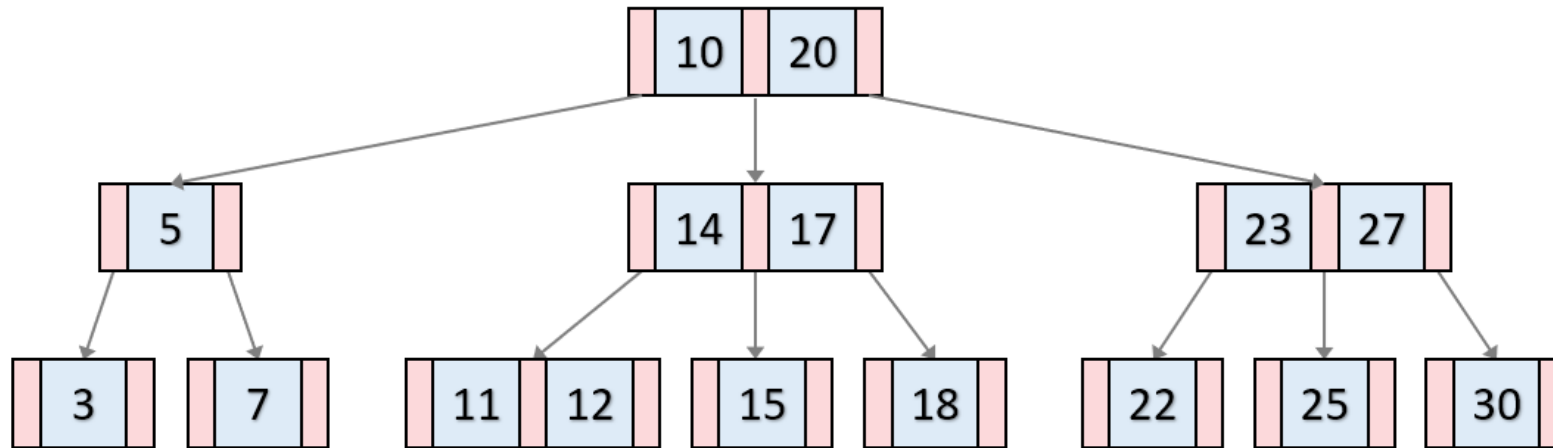


- 부모와 형제의 색을 바꾸고 부모를 기준으로 왼쪽 회전한 뒤 doubly black을 기준으로 case 2,3,4 중 하나로 해결

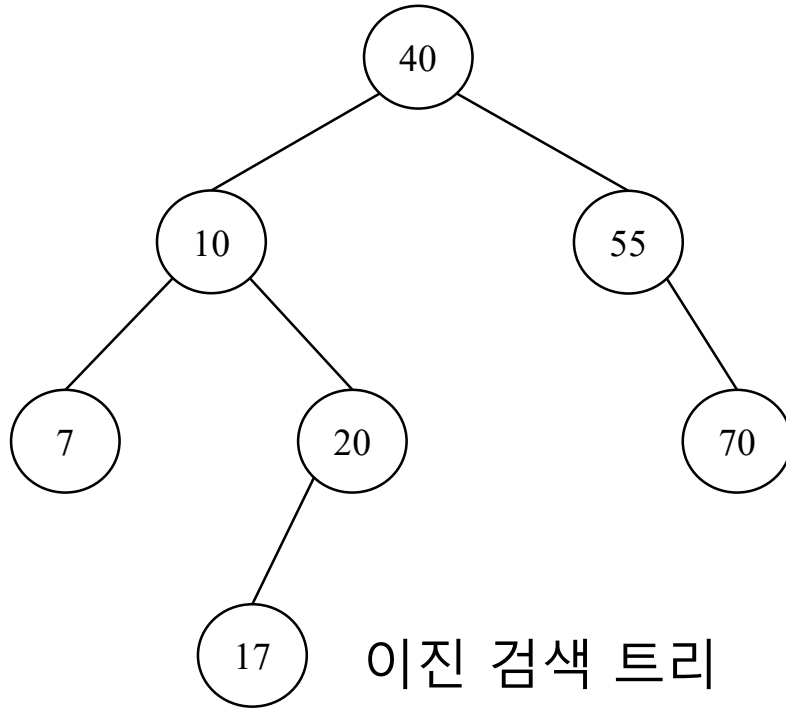
Red-Black Tree, RB Tree

B-트리란?

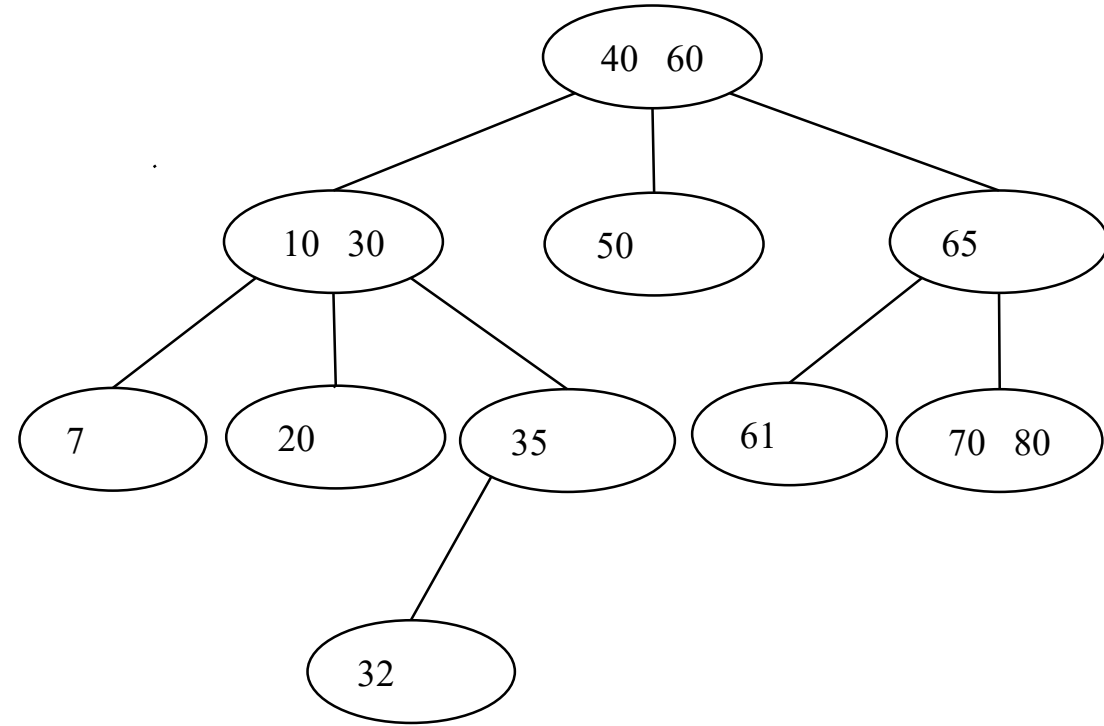
- B-트리(B-tree)는 데이터베이스와 파일 시스템에서 널리 사용되는 트리 자료구조의 일종으로, 이진 트리를 확장해 하나의 노드가 가질 수 있는 자식 노드의 최대 숫자가 2보다 큰 트리 구조이다.



K-진 검색 트리



K=2, 최대 2개 분기



삼진 검색 트리

K=3, 최대 3개 분기

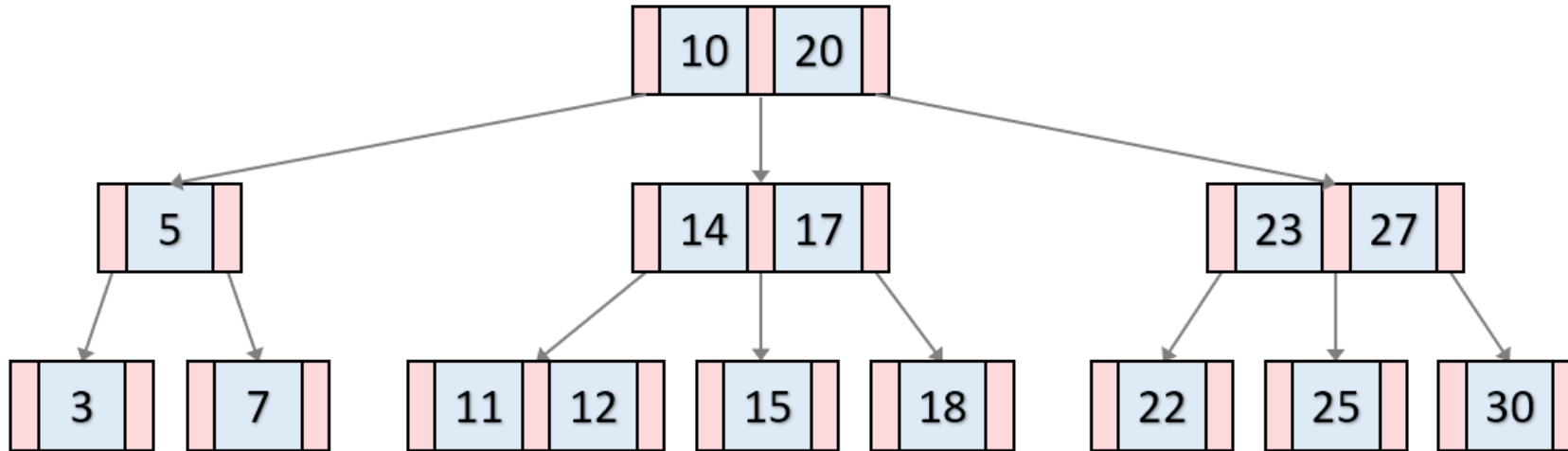
B-Trees

B-트리 특징

- B트리는 이진트리와 다르게 하나의 노드에 많은 수의 정보를 가지고 있을 수 있다. 최대 M 개의 자식을 가질 수 있는 B트리를 M 차 B트리라고 하며 다음과 같은 특징을 가진다.
- 노드는 최대 M 개 부터 $M/2$ 개 까지의 자식을 가질 수 있다.
- 노드에는 최대 $M-1$ 개 부터 $[M/2]-1$ 개의 키가 포함될 수 있다.
- 노드의 키가 x 개라면 자식의 수는 $x+1$ 개 이다.
- 최소 차수는 자식 수의 하한 값을 의미하며, 최소 차수가 t 라면 $M=2t-1$ 을 만족합니다.
(최소차수 t 가 2라면 3차 B트리이며, key의 하한은 1개 이다.)

B-트리 특징

- 다음은 차수가 3인 B트리 입니다. 파란색 부분은 각 노드의 key를 나타내며, 빨간색 부분은 자식 노드들을 가르키는 포인터.
- key들은 노드 안에서 항상 정렬된 값을 가지며, 이진검색 트리처럼 각 key들의 왼쪽 자식들은 항상 key보다 작은 값을, 오른쪽은 큰 값을 가진다.



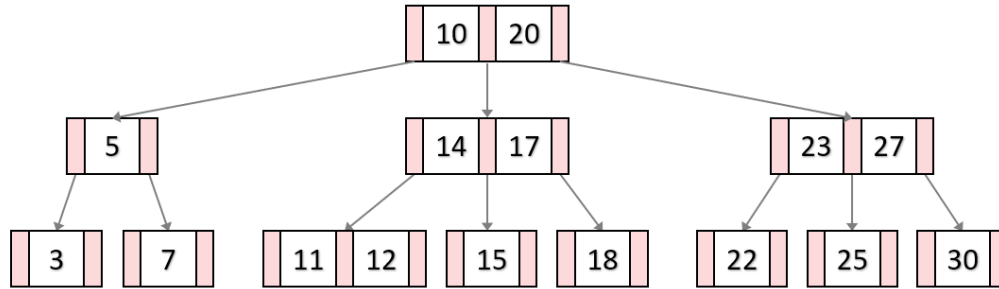
B-트리 검색 과정

- 루트노드에서 시작하여 하향식으로 검색을 수행.
- 검색하고자 하는 key를 k라고 하였을 때 검색 과정.
 1. 루트 노드에서 시작하여 key들을 순회하면서 검사.
 - 1-1. 만일 k와 같은 key를 찾았다면 검색을 종료.
 - 1-2. 검색하는 값과 key들의 대소 관계를 비교.

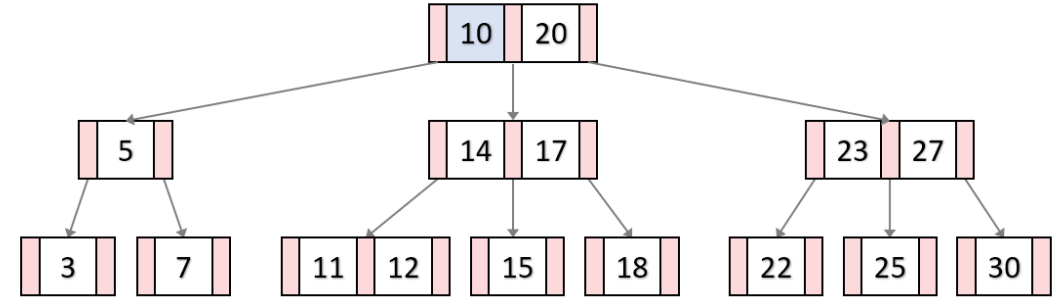
어떠한 key들 사이에 k가 들어간다면 해당 key들 사이의 자식노드로 내려감.
 2. 해당 과정을 리프노드에 도달할 때까지 반복. 만일 리프노드에도 k와 같은 key가 없다면 검색을 실패.

B-트리 검색 과정

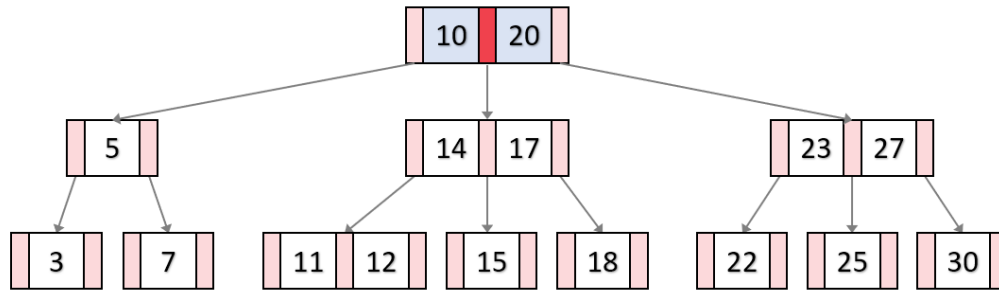
1 18 검색 시작, 루트노드의 key를 순회하면서 검색 시작



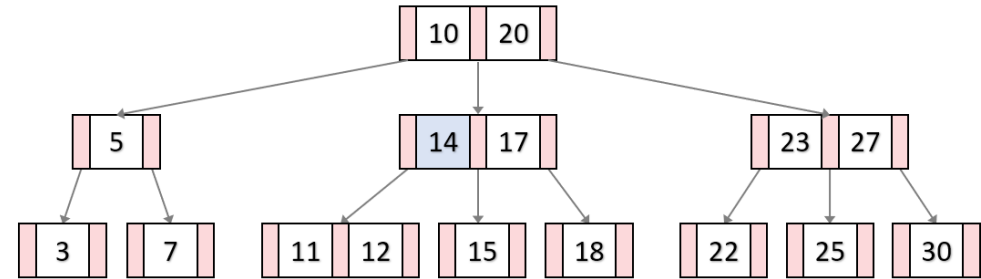
2 18은 10보다 크기 때문에 다음 key를 검사



3 18은 10보다 크고, 20보다 작기 때문에 10과 20 사이의 자식 노드로 이동

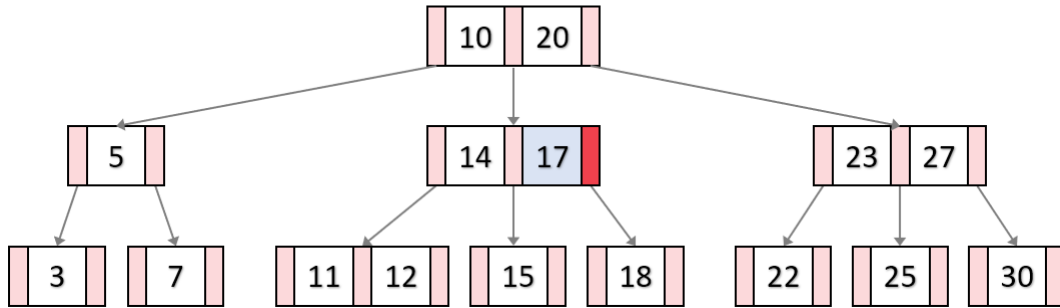


4 자식노드에서 다시 검색 시작. 18은 14보다 크기 때문에 다음 key를 검사

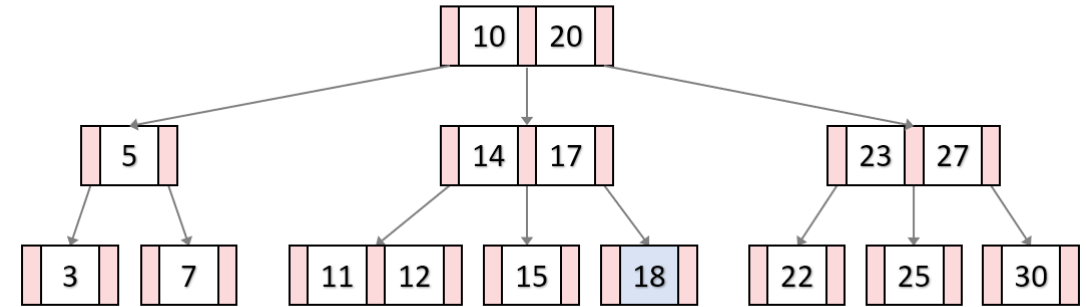


B-트리 검색 과정

5 18은 노드의 가장 마지막 key인 17보다 크기 때문에 노드의 가장 마지막 자식노드로 이동



6 18 검색 완료



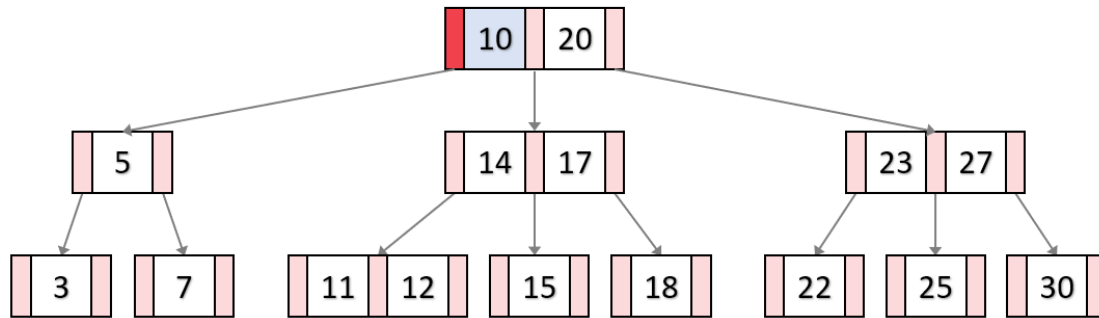
B-트리 key 삽입과정

- key를 삽입하기 위해서는 1. 요소 삽입에 적절한 리프 노드를 검색하고, 2. 필요한 경우 노드를 분할해야 한다.
- 리프노드 검색은 하향식이지만 노드 분할의 과정은 상향식으로 이루어진다고 볼 수 있다.
- 삽입하고자 하는 값을 k 로 하였을 때 삽입 과정이다.
 1. 트리가 비어있으면 루트 노드를 할당하고 k 를 삽입. 만일 루트노드가 가득 찼다면, 노드를 분할하고 리프노드가 생성된다.
 2. 이후부터는 삽입하기에 적절한 리프노드를 찾아 k 를 삽입한다. 삽입위치는 노드의 key값과 k 값을 검색 연산과 동일한 방법으로 비교하면서 찾는다.
- 이후 두가지 케이스로 나뉜다.

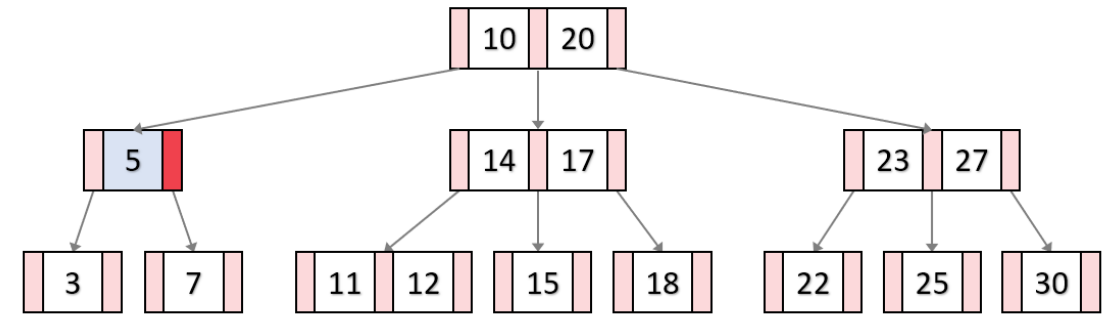
B-트리 key 삽입과정

- Case 1. 분할이 일어나지 않는 경우
리프노드가 가득 차지 않았다면, 오름차순으로 k를 삽입.

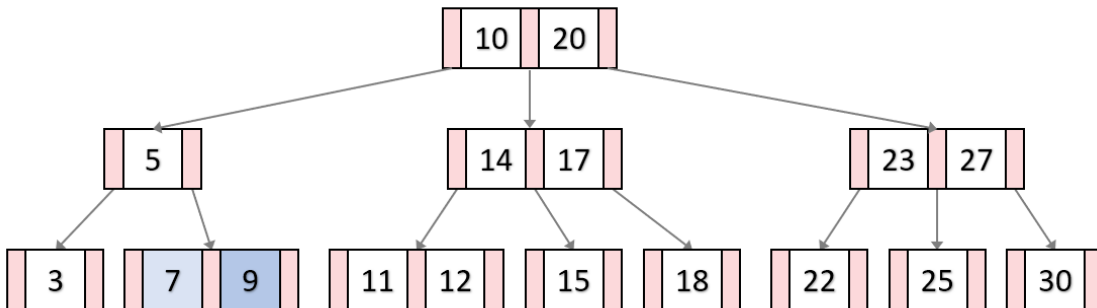
1 9 삽입 시작, 루트노드에서 key 10이 9보다 크기 때문에 가장 왼쪽 자식노드로 이동



2 9는 5보다 크기 때문에 가장 오른쪽 자식노드로 이동



3 리프노드에 도달, 9는 7보다 크기 때문에 7 오른쪽에 9 삽입 완료

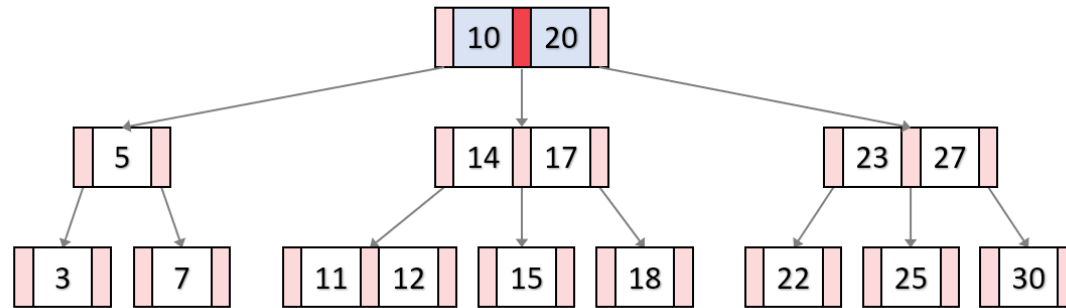


B-트리 key 삽입과정

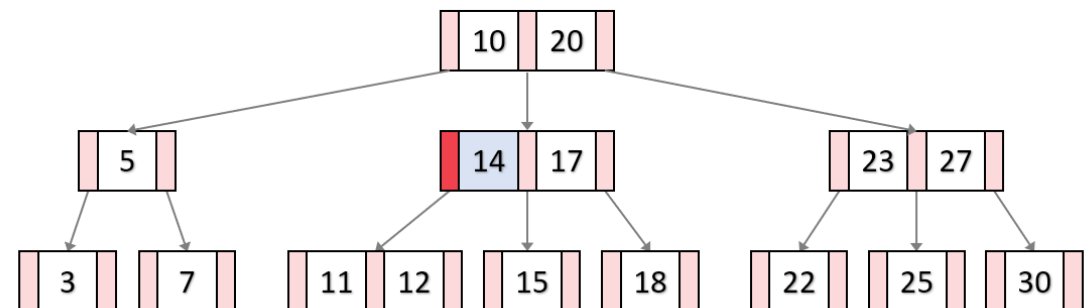
- Case 2. 분할이 일어나는 경우
- 만일 리프노드에 key노드가 가득 찬 경우, 노드를 분할해야 한다.
 1. 오름차순으로 요소를 삽입합니다. 노드가 담을 수 있는 최대 key 개수를 초과하게 된다.
 2. 중앙값에서 분할을 수행한다. 중앙값은 부모 노드로 병합하거나 새로 생성된다. 왼쪽 키들은 왼쪽 자식으로, 오른쪽 키들은 오른쪽 자식으로 분할된다.
 3. 부모 노드를 검사해서 또 다시 가득 찼다면, 다시 부모노드에서 위 과정을 반복한다.

B-트리 key 삽입과정

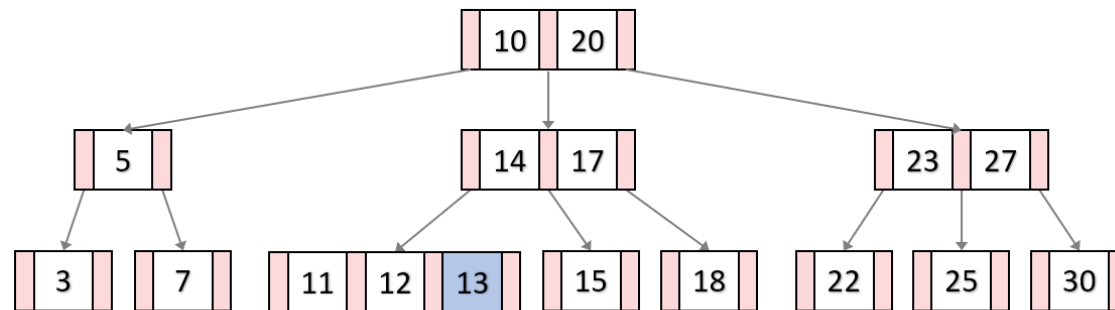
1 13 삽입 시작, 루트노드의 key 중 10보다 크고 20보다 작기 때문에 중간 자식노드로 이동



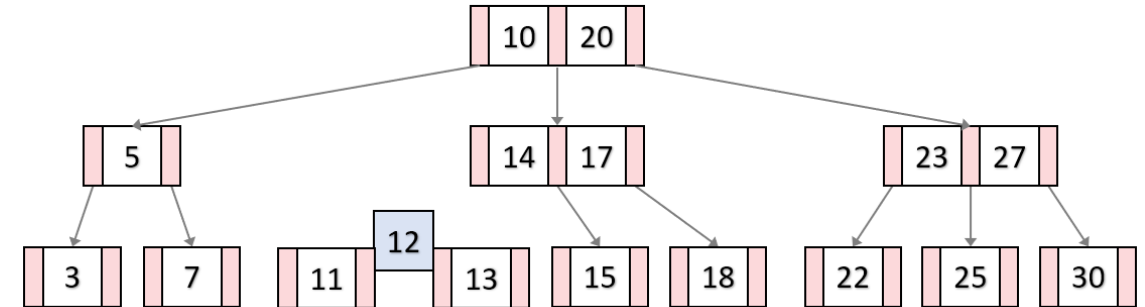
2 13은 14보다 작기 때문에, 가장 왼쪽 자식노드로 이동



3 13은 12보다 크기 때문에 가장 마지막에 삽입. 해당 노드가 **최대로 가질 수 있는 key의 개수를 초과**했기 때문에 분할을 수행

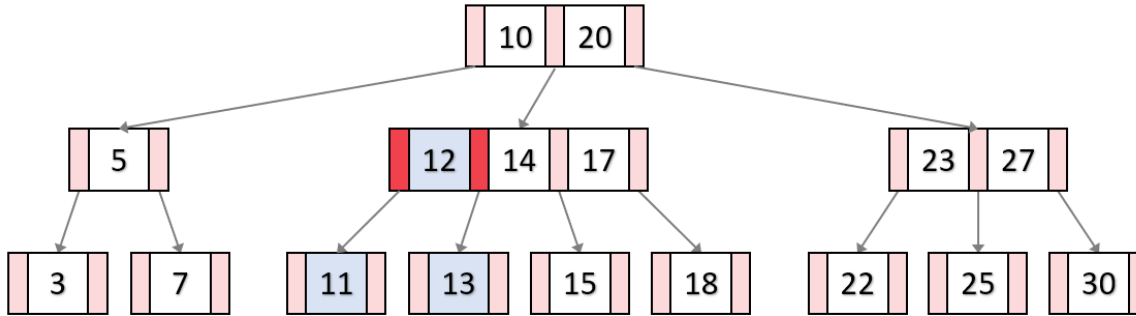


4 중앙값 12를 기준으로 분할을 수행

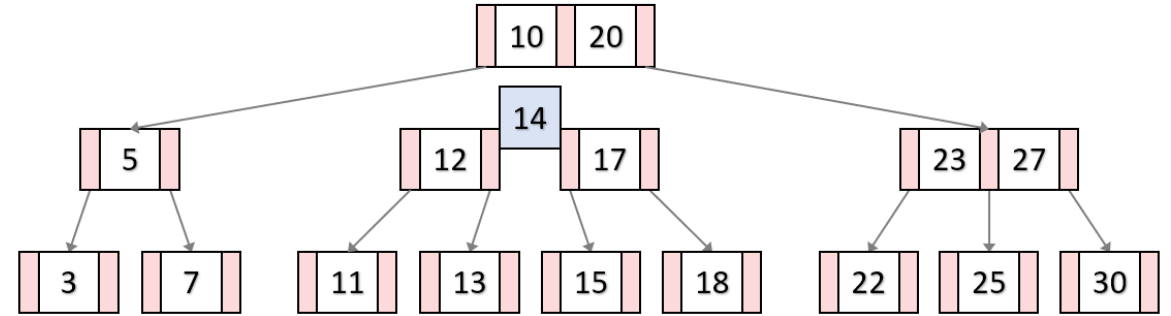


B-트리 key 삽입과정

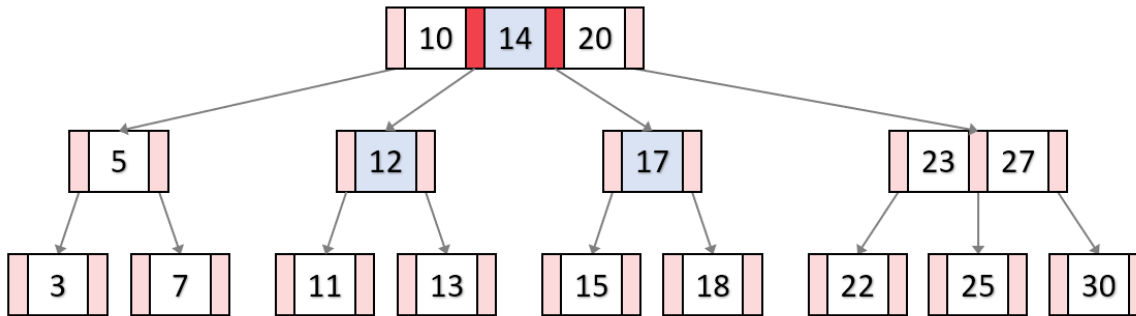
- 5 12는 부모노드에 오름차순으로 삽입, 11과 13은 12의 왼쪽자식, 오른쪽 자식으로 설정



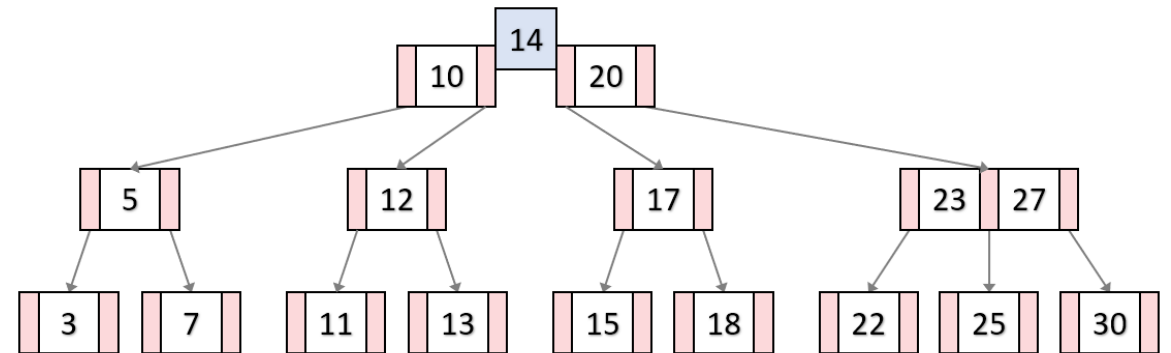
- 6 12가 병합된 노드가 **최대로 가질 수 있는 key의 개수를 초과**, 중앙값 14를 기준으로 분할을 수행



- 7 14는 부모노드에 오름차순으로 삽입, 12와 17은 14의 왼쪽자식, 오른쪽 자식으로 설정

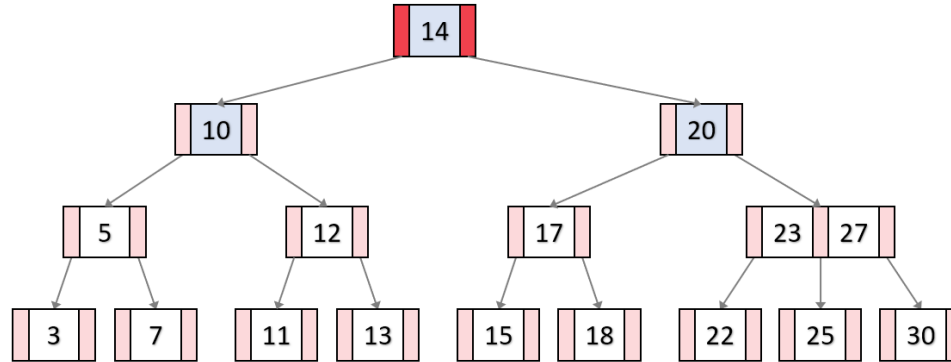


- 8 14가 병합된 노드가 **최대로 가질 수 있는 key의 개수를 초과**, 중앙값 14를 기준으로 분할을 수행



B-트리 key 삽입과정

- 9 14가 새로운 루트노드가 되고, 10과 20은 14의 왼쪽자식, 오른쪽 자식으로 설정. 삽입과정 완료



B-트리 key 삭제 과정

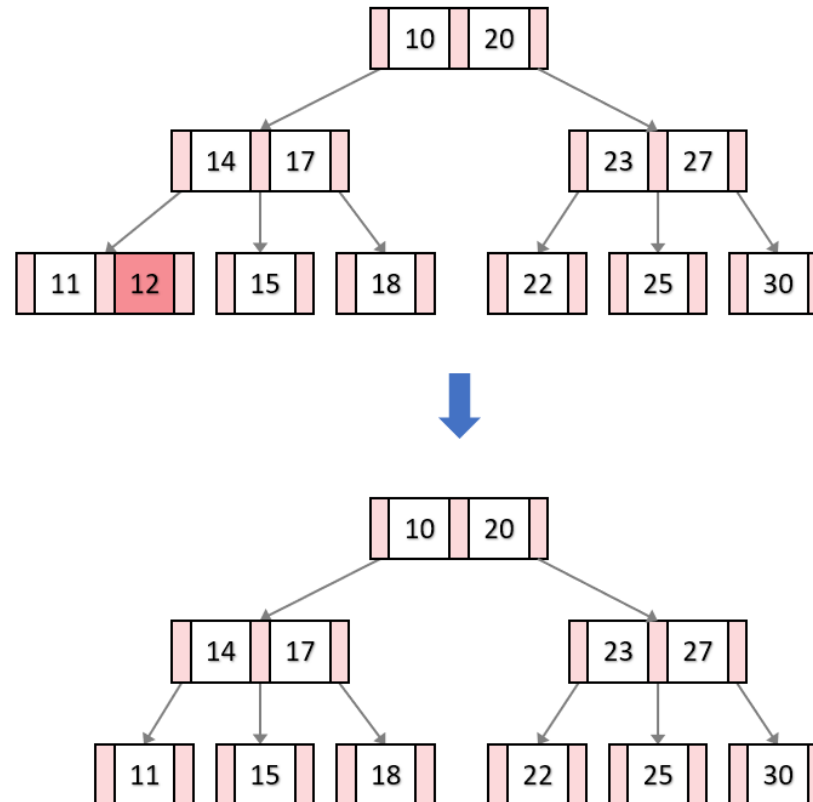
- 요소를 삭제하기 위해선 1. 삭제할 키가 있는 노드 검색, 2. 키 삭제, 3. 필요한 경우, 트리 균형 조정을 해야한다.
- 삭제 과정을 이해하기 위해서 몇가지 용어를 정의하였다.
 - ❖ inorder predecessor : 노드의 왼쪽 자손에서 가장 큰 key
 - ❖ inorder successor : 노드의 오른쪽 자손에서 가장 작은 key
 - ❖ 부모key: 부모노드의 key들 중 왼쪽 자식으로 본인 노드를 가지고 있는 key값.
 - ❖ 단, 마지막 자식노드의 경우에는 부모의 마지막 key.

B-트리 key 삭제 과정

- Case 1. 삭제할 key k가 리프에 있는 경우

Case 1.1) 현재 노드의 key 개수가 최소 key 개수보다 크다면 다른 노드들에 영향 없이 해당 k를 단순 삭제.

Case 1.1) 12 삭제, 해당 노드의 key 개수는 최소 key의 개수보다 크므로 바로 삭제



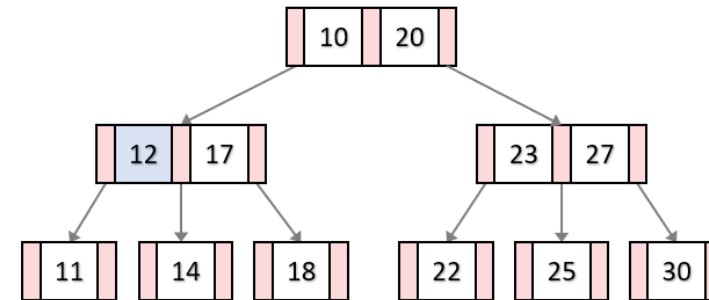
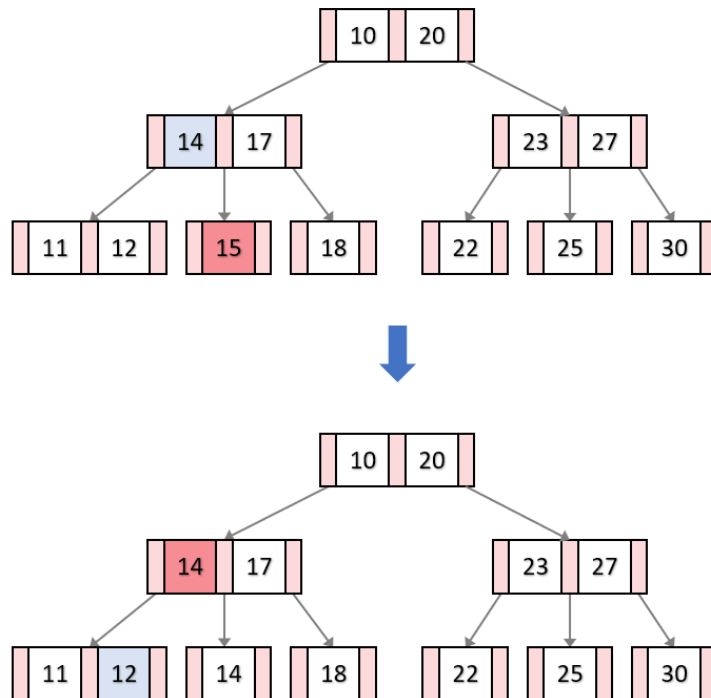
B-트리 key 삭제 과정

- Case 1. 삭제할 key k가 리프에 있는 경우

Case 1.2) 왼쪽 또는 오른쪽 형제 노드의 key가 최소 key 개수 이상이라면 부모 key 값으로 k를 대체.

최소키 개수 이상의 키를 가진 형제 노드가 왼쪽 형제라면 가장 큰 값을, 오른쪽 형제라면 가장 작은 값을 부모key로 대체.

Case 1.2) 15 삭제, 해당 노드를 부모 key로 대체, 왼쪽 형제 노드에서 가장 큰 값을 부모key로 대체



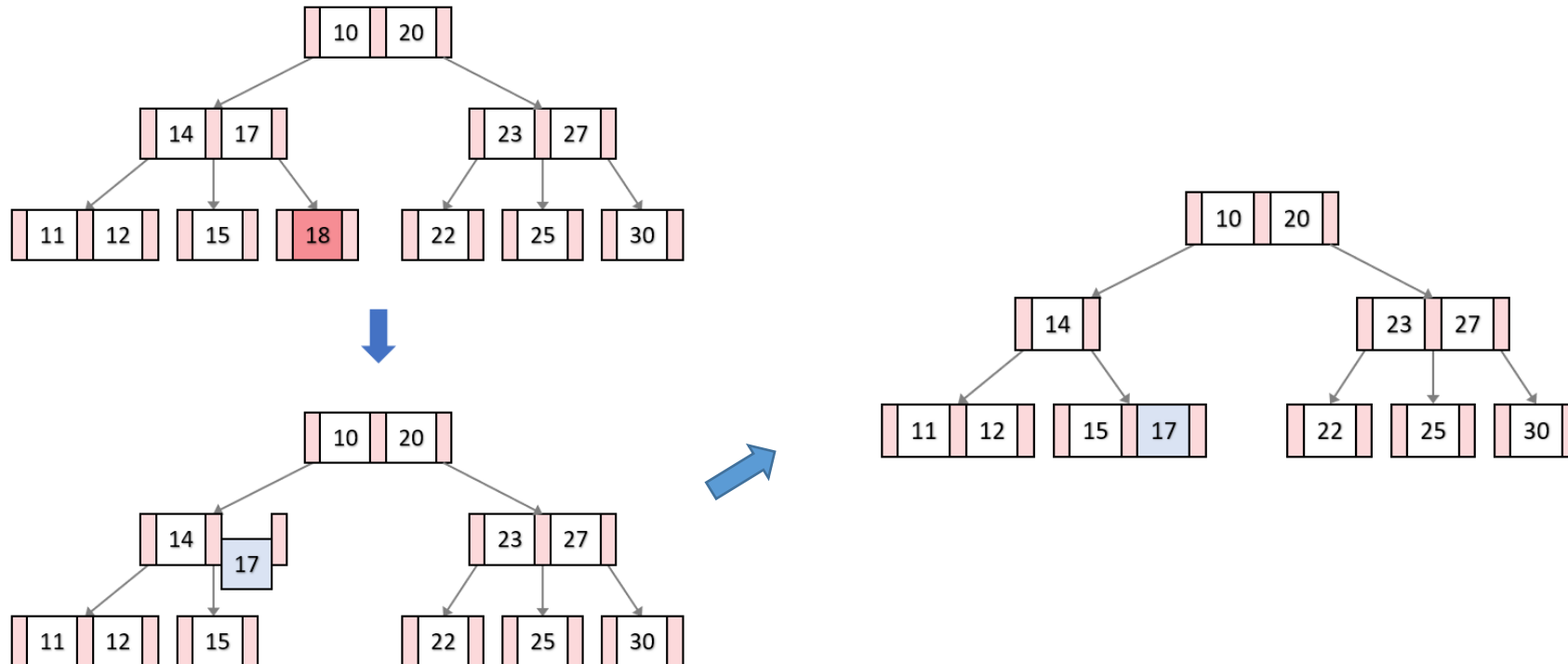
B-트리 key 삭제 과정

- Case 1. 삭제할 key k가 리프에 있는 경우

Case 1.3) 왼쪽, 오른쪽 형제 노드의 key가 최소 key 개수이고, 부모노드의 key가 최소개수 이상이면 k를 삭제한 후, 부모key를 형제 노드와 병합.

부모노드의 key개수를 하나 줄이고, 자식 수 역시 하나를 줄여 B-Tree를 유지.

Case 1.3) 18 삭제, 부모 key를 내려 형제 노드에 병합



B-트리 key 삭제 과정

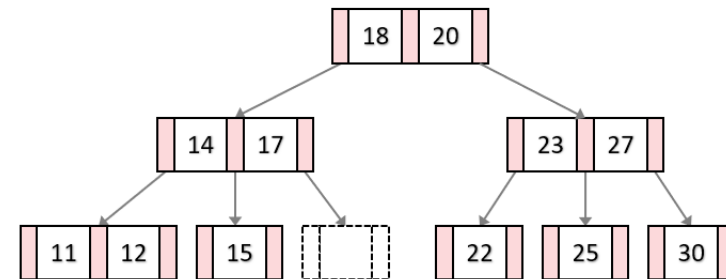
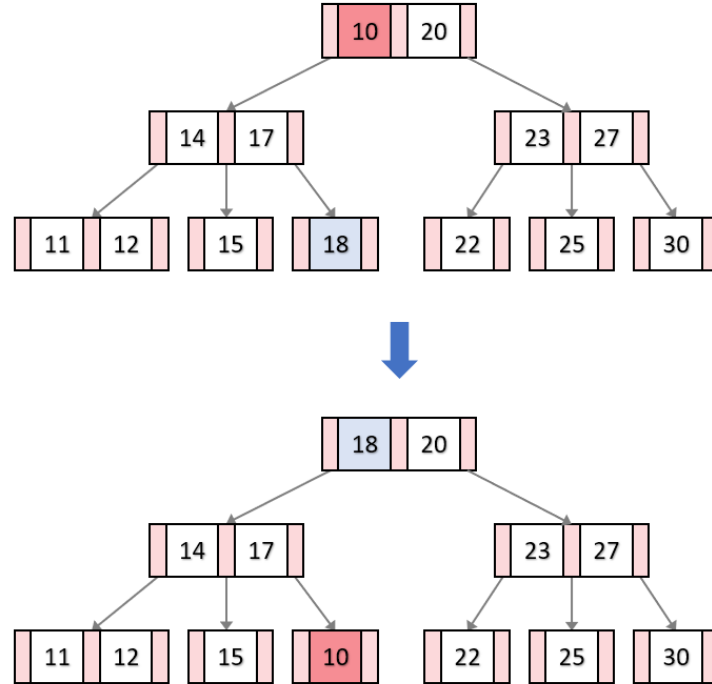
- Case 1. 삭제할 key k가 리프에 있는 경우

Case 1.4) 자신과 형제, 부모 노드의 key 개수가 모두 최소 key 개수라면 부모노드를 루트노드로 한 부분 트리의 높이가 줄어드는 경우이기 때문에 재구조화의 과정이 일어난다. Case3의 2번 과정으로 이동.

B-트리 key 삭제 과정

- Case 2. 삭제할 key k가 내부 노드에 있고, 노드나 자식에 키가 최소 키 수보다 많을 경우
 현재 노드의 inorder predecessor 또는 inorder successor와 k의 자리를 바꾼다.
 리프노드의 k를 삭제하게 되면, 리프노드가 삭제 되었을 때의 조건으로 변한다. 삭제한 리프노
 드에 대해서 case 1 조건으로 이동.

Case 2) 10 삭제, inorder predecessor인 18을 찾아 서로의 key를 교환,



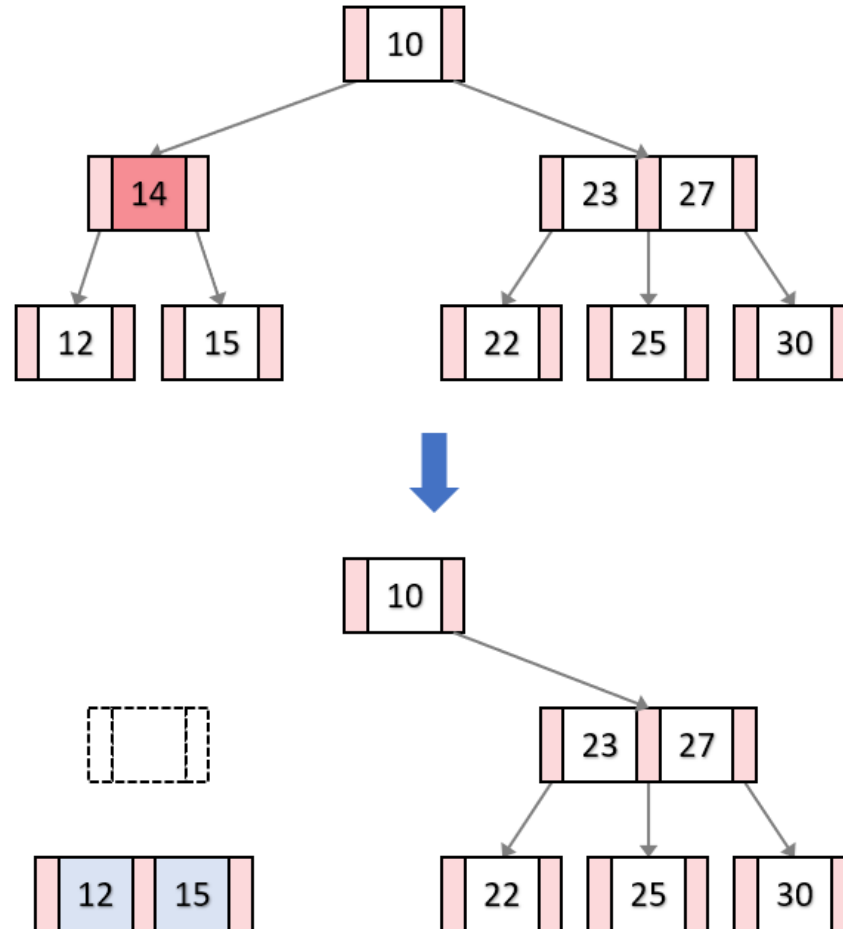
리프노드가 삭제되었을 때의 조건으로 변환

B-트리 key 삭제 과정

- Case 3. 삭제할 key k가 내부 노드에 있고, 노드에 key 개수가 최소key 개수만큼, 노드의 자식 key 개수도 모두 최소 key 개수인 경우
- 삭제할 key k가 있는 노드도 최소, 자식노드들도 최소의 key 개수를 가지므로, k를 삭제하면 트리의 높이가 줄어들어 재구조화가 일어나는 케이스. 재구조화의 과정은 다음과 같다.
 1. k를 삭제하고, k의 양쪽 자식을 병합하여 하나의 노드로 만든다.
 2. K의 부모key를 인접한 형제 노드에 붙입니다. 이후, 이전에 병합했던 노드를 자식노드로 설정.
 3. 해당 과정을 수행하였을 때 부모노드의 개수가 e 따라 이후 수행 과정이 달라진다. 3-1. 만일 새로 구성된 인접 형제노드의 key가 최대 key 개수를 넘어갔다면, 삽입 연산의 노드 분할 과정을 수행. 3-2. 만일 인접 형제노드가 새로 구성되더라도 원래 k의 부모 노드가 최소 key의 개수보다 작아진다면, 부모 노드에 대하여 2번 과정부터 다시 수행.

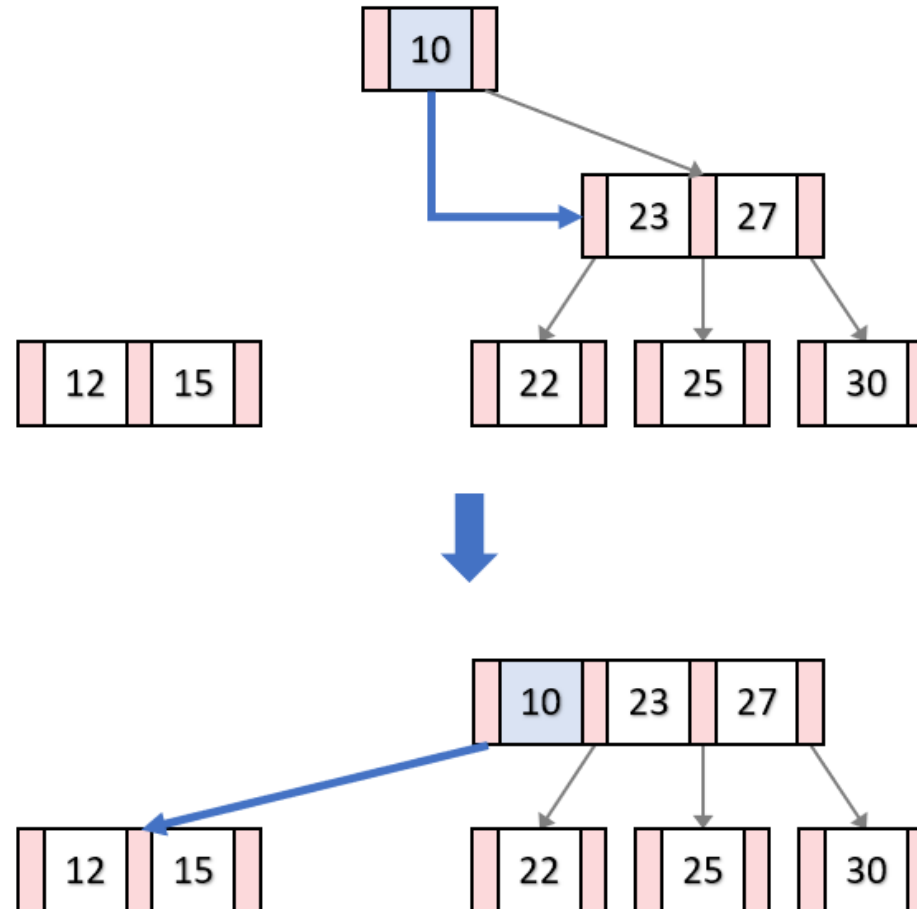
B-트리 key 삭제 과정

Case 3-1) 14 삭제, 양쪽 자식노드를 병합하고, 14는 삭제



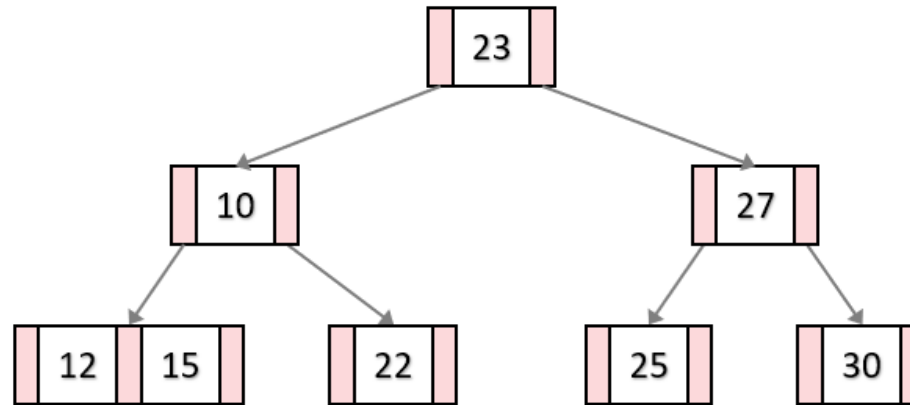
B-트리 key 삭제 과정

Case 3-2) 부모 key인 10을 형제 노드에 붙이고, 이전에 병합한 자식노드를 왼쪽 자식으로 가져옴



B-트리 key 삭제 과정

Case 3-3-1) 최대 key수를 넘어간 노드가 생겼기 때문에 노드분할 수행

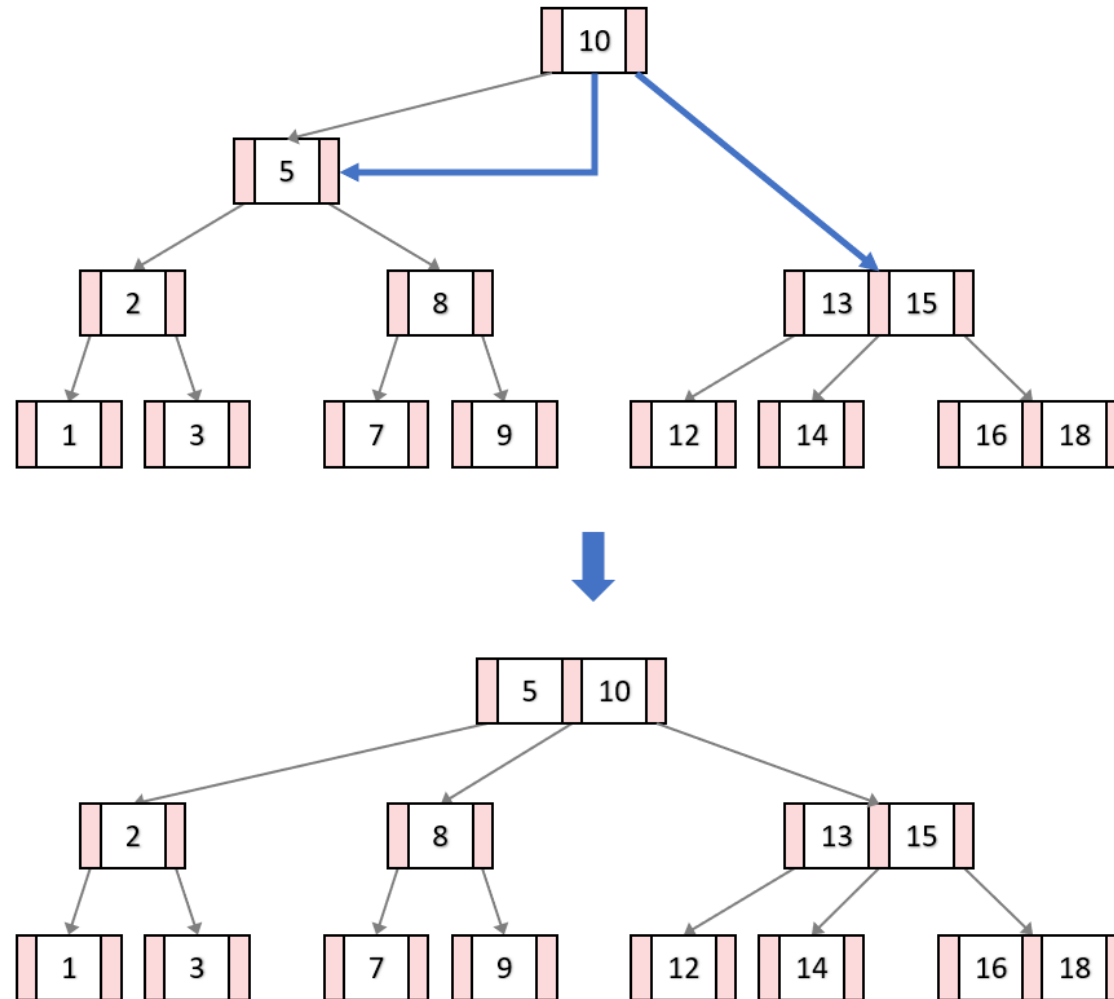


- Case 3-3-2)** 17 삭제 시, 연산을 수행하다 보면 17의 부모노드의 key가 최소 key 개수 이하로 내려감



B-트리 key 삭제 과정

17의 부모노드를 기준으로 하여 Case 3과정을 다시한번 수행



Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://wikidocs.net/195736>
- <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>